

DSA Notes in C++

Beginner to Advanced — Comprehensive Study Notes

STL · Time & Space Complexity · Arrays · Strings · Searching · Sorting · Hashing · Two Pointers · Sliding Window · Linked Lists · Stacks · Queues & Deques · Recursion · Backtracking · Trees · BST · Heaps · Greedy · Graphs · Shortest Paths · MST & Topo Sort · Dynamic Programming · Tries · Bit Manipulation · DSU · Fenwick · Segment Trees · Advanced Graph Algorithms

Every topic: what it is · why it's useful · how it works · step-by-step examples · clean C++ implementations · complexity · common mistakes · Easy / Medium / Challenging problems

Problem-solving patterns · separated hints & solutions · mastery checklist · complexity cheat sheet

Table of Contents

DSA Notes in C++ — Beginner to Advanced	10
How to use this course	10
File index (study order)	10
Golden rules for learning DSA	11
01 — C++ STL (Standard Template Library)	12
What it is	12
Why it's useful	12
Containers at a glance	12
Key operations (with complexity)	12
Must-know algorithms (<algorithm>)	13
Step-by-step example — counting word frequencies	14
When to use what — decision guide	14
Common mistakes	15
Practice questions	15
02 — Time & Space Complexity (Big-O)	16
What it is	16
Why it's useful	16
The growth ladder (memorize the shape, not the formula)	16
How to analyze code	16
Step-by-step example — analyzing three solutions to "sum of array"	17
Space complexity notes	17
Common mistakes	18
Practice questions	18
03 — Arrays	19
What it is	19
Why it's useful	19
How it works / key vocabulary	19
Pattern 1 — Prefix sums (the single most reused trick)	19
Pattern 2 — Kadane's algorithm (maximum subarray sum)	19
Step-by-step example — "two-sum with sorted array"	20
Step-by-step example — rotate array in-place by k	20
Common array mistakes	20
Practice questions	20
04 — Strings	22
What it is	22
Why it's useful	22
Key std::string operations	22
Pattern 1 — Frequency counting (anagrams, first-unique-char)	22
Pattern 2 — Palindrome checks & longest palindromic substring	22
Pattern 3 — String matching (naive → hashing/KMP)	23

Step-by-step example — group anagrams	23
Common mistakes	24
Practice questions	24
05 — Searching	25
What it is	25
Why it's useful	25
Binary search — the classic	25
Lower bound / upper bound (finding boundaries with duplicates)	25
Binary search on answer (the pattern that wins interviews)	26
Step-by-step example — sqrt via binary search	26
Common mistakes	27
Practice questions	27
06 — Sorting	28
What it is	28
Why it's useful	28
Comparison table	28
The two you must deeply understand	28
Counting sort (non-comparison, $O(n + k)$)	29
Custom comparators and stability	29
Step-by-step example — merge intervals (sorting unlocks the solution)	30
Common mistakes	30
Practice questions	30
07 — Hashing (Hash Tables)	32
What it is	32
Why it's useful	32
How it works	32
Core patterns	32
Hashable vs not	33
Step-by-step example — first repeating element	33
Common mistakes	33
Practice questions	34
08 — Two Pointers	35
What it is	35
Why it's useful	35
Pattern 1 — Opposite direction (sorted array pair problems)	35
Pattern 2 — Same direction (fast/slow, in-place rewriting)	35
Pattern 3 — Merging / comparing two sequences	36
When to reach for two pointers	36
Common mistakes	36
Practice questions	36
09 — Sliding Window	38
What it is	38

Why it's useful	38
Two kinds	38
Pattern 1 — Fixed window: maximum sum of subarrays of size k	38
Pattern 2 — Variable window: longest subarray with sum $\leq k$ (all positive)	38
Pattern 3 — Variable window with a *condition counter: longest substring with k distinct chars	39
Pattern 4 — Monotonic deque: max in every window of size k ($O(n)$)	39
Common mistakes	40
Practice questions	40
10 — Linked Lists	41
What it is	41
Why it's useful	41
Core operations	41
Pattern 1 — Reversal (in place, iterative)	42
Pattern 2 — Floyd's cycle detection (tortoise & hare)	42
Pattern 3 — Fast/slow for the middle	42
Pattern 4 — Merge two sorted lists	42
Common mistakes	42
Practice questions	43
11 — Stacks	44
What it is	44
Why it's useful	44
Pattern 1 — Balanced parentheses	44
Pattern 2 — Monotonic stack: next greater element	44
Pattern 3 — Expression evaluation	45
Step-by-step example — "simplify path" style (undo semantics)	45
Common mistakes	45
Practice questions	46
12 — Queues & Deques	47
What they are	47
Why they're useful	47
Implementing a queue with an array (circular buffer)	47
Pattern 1 — Level-order traversal of a binary tree (BFS)	47
Pattern 2 — Deque as a monotonic structure (sliding window max)	48
Pattern 3 — Queue problems that aren't BFS	48
Common mistakes	48
Practice questions	49
13 — Recursion	50
What it is	50
Why it's useful	50
How to write recursion — the "leap of faith"	50
Pattern 1 — Linear recursion: Fibonacci	50
Pattern 2 — Divide & conquer: merge sort	50
Pattern 3 — Recursion on trees	51

Pattern 4 — Recursive linked-list reversal	51
Tail recursion & iteration	51
Common mistakes	51
Practice questions	52
14 — Backtracking	53
What it is	53
Why it's useful	53
Pattern 1 — All subsets (powerset)	53
Pattern 2 — Permutations	53
Pattern 3 — N-Queens (the canonical pruning example)	54
Pattern 4 — Word search (grid DFS + backtracking)	54
Pruning — the difference between $O(2^n)$ that dies and $O(2^n)$ that runs	54
Common mistakes	55
Practice questions	55
15 — Trees (Binary Trees)	56
What it is	56
Why it's useful	56
Node definition & building	56
Traversals — the core skill	56
Pattern 1 — Height / depth	57
Pattern 2 — Diameter (longest path between any two nodes)	57
Pattern 3 — LCA (lowest common ancestor)	57
Pattern 4 — Same tree / subtree / symmetry (mirror-recursion)	58
Pattern 5 — Serialize / reconstruct	58
Common mistakes	58
Practice questions	58
16 — Binary Search Trees (BST)	60
What it is	60
Why it's useful	60
Search	60
Insert	60
Delete (the tricky one — 3 cases)	60
Validate a BST (the classic gotcha)	61
Pattern — k-th smallest / k-th largest (inorder + early exit)	61
Pattern — floor / ceil, and "closest value"	61
Balanced vs unbalanced — and self-balancing BSTs	61
Common mistakes	62
Practice questions	62
17 — Heaps & Priority Queues	63
What it is	63
Why it's useful	63
How a heap works (two operations)	63
std::priority_queue — the practical interface	64

Pattern 1 — Top-K elements (min-heap of size k)	64
Pattern 2 — Merge k sorted lists/arrays	64
Pattern 3 — Median of a stream (two heaps)	65
Heap sort (bonus)	65
Common mistakes	65
Practice questions	65
18 — Greedy Algorithms	67
What it is	67
Why it's useful	67
When is greedy correct?	67
Pattern 1 — Interval scheduling (maximum non-overlapping intervals)	67
Pattern 2 — Fractional knapsack / value-density	67
Pattern 3 — Huffman coding (optimal prefix-free codes)	68
Pattern 4 — Jump game / candy / gas station ($O(n)$ greedy scans)	68
Greedy fails here — know these traps	68
Common mistakes	69
Practice questions	69
19 — Graphs: Representations, BFS, DFS	70
What it is	70
Representations	70
DFS (depth-first) — go deep before going wide	70
BFS (breadth-first) — explore in layers, shortest paths in unweighted graphs	71
The 6 problems DFS/BFS solve	71
Choosing DFS vs BFS	72
Common mistakes	72
Practice questions	72
20 — Shortest Paths (Dijkstra, Bellman-Ford, Floyd-Warshall, 0-1 BFS)	74
What it is	74
Dijkstra — the workhorse	74
Bellman-Ford — handles negative weights, detects negative cycles	75
Floyd-Warshall — all-pairs in $O(V^3)$	75
0-1 BFS — weights are only 0 or 1	75
Choosing the right tool	76
Common mistakes	76
Practice questions	76
21 — Topological Sort & Minimum Spanning Tree	78
Part A — Topological sort (DAGs)	78
Part B — Minimum Spanning Tree (MST)	79
Common mistakes	80
Practice questions	80
22 — Dynamic Programming (DP)	82
What it is	82

Why it's useful	82
The two styles	82
Step-by-step — Fibonacci (the template problem)	82
The 5-step DP recipe	82
Pattern 1 — 1D DP: climbing stairs / house robber	83
Pattern 2 — 2D DP: grid paths / edit distance / LCS	83
Pattern 3 — Knapsack (the archetype)	83
Pattern 4 — Coin change (min coins / count ways)	84
Pattern 5 — LIS: longest increasing subsequence	84
When to reach for DP — recognition checklist	84
Common mistakes	84
Practice questions	85
23 — Tries (Prefix Trees)	86
What it is	86
Why it's useful	86
Implementation (array-of-pointers version)	86
Implementation (hash-map children) — for large alphabets / memory	87
Pattern 1 — Autocomplete / collect all words with a prefix	87
Pattern 2 — Count words with a prefix (store counts in nodes)	87
Pattern 3 — Longest common prefix of a set	87
Pattern 4 — XOR maximization (trie of bits)	87
Common mistakes	88
Practice questions	88
24 — Bit Manipulation	90
What it is	90
Operators	90
Pattern 1 — XOR tricks	90
Pattern 2 — Bit-counting / DP	91
Pattern 3 — Subset enumeration by masks	91
Pattern 4 — Power of two & related checks	91
Pattern 5 — Bitmasks for state (flags, visited sets)	91
Common mistakes	91
Practice questions	92
25 — Disjoint Set Union (DSU / Union-Find)	93
What it is	93
Why it's useful	93
How it works — two optimizations	93
Step-by-step — building components	93
Pattern 1 — Kruskal's MST (see topic 21 for full context)	93
Pattern 2 — Cycle detection in an undirected graph	94
Pattern 3 — Connected components count / sizes	94
Pattern 4 — "Merge with a twist" (the interview favorite)	94
Common mistakes	94

Practice questions	94
26 — Fenwick Tree (Binary Indexed Tree, BIT)	96
What it is	96
Why it's useful	96
How it works	96
Implementation	96
Step-by-step — inversion count (the classic use)	97
Pattern 1 — Range update, point query (difference trick)	97
Pattern 2 — Range update, range sum (two BITs)	97
Pattern 3 — k-th smallest / order statistics	97
Fenwick vs Segment Tree — when to use which	97
Common mistakes	98
Practice questions	98
27 — Segment Trees	99
What it is	99
Why it's useful	99
Implementation — point update + range query (array-based)	99
Step-by-step — query [1, 2] on the tree above	99
Lazy propagation — range updates in $O(\log n)$	100
Pattern — "merge" beyond sum	100
Pattern — segment tree of segment trees / persistence (mention)	101
Common mistakes	101
Practice questions	101
28 — Advanced Graph Algorithms	103
What's in this file	103
1. SCC — strongly connected components	103
2. Bridges & articulation points (undirected graphs)	104
3. Euler path & circuit	104
4. Bipartite matching — Kuhn's algorithm	105
5. Max flow / min cut (concept + Edmonds-Karp)	105
Choosing the right advanced tool	105
Common mistakes	106
Practice questions	106
29 — Problem-Solving Patterns (Recognize → Template)	107
Pattern 1 — Prefix sums (precompute once, answer many)	107
Pattern 2 — Binary search on answer	107
Pattern 3 — Two pointers (opposite & same direction)	107
Pattern 4 — Sliding window	107
Pattern 5 — Monotonic stack	108
Pattern 6 — Greedy	108
Pattern 7 — Backtracking (enumerate everything, pruned)	108
Pattern 8 — BFS / DFS (graphs and grids)	108
Pattern 9 — Dynamic programming	108

Pattern 10 — "Design a data structure" (the mix of everything)	109
Cross-pattern decision map	109
How to attack any problem (the meta-framework)	109
Mixed practice (do these AFTER the per-topic problems)	110
30 — Hints & Solutions for All Practice Questions	111
01 — STL	111
02 — Complexity	112
03 — Arrays	113
04 — Strings	114
05 — Searching	115
06 — Sorting	117
07 — Hashing	118
08 — Two Pointers	119
09 — Sliding Window	120
10 — Linked Lists	121
11 — Stacks	122
12 — Queues & Deques	124
13 — Recursion	125
14 — Backtracking	126
15 — Trees	128
16 — BST	129
17 — Heaps	130
18 — Greedy	131
19 — Graphs	132
20 — Shortest Paths	133
21 — MST & Topo	134
22 — DP	135
23 — Tries	137
24 — Bit Manipulation	138
25 — DSU	139
26 — Fenwick	140
27 — Segment Trees	141
28 — Advanced Graphs	141
Closing notes	142
31 — DSA Checklist & Complexity Cheat Sheet	143
Part A — Mastery checklist (tick off honestly)	143
Part B — Complexity cheat sheet	144
Part C — Final exam survival card (one screen)	145

DSA Notes in C++ — Beginner to Advanced

Comprehensive, self-contained study notes for Data Structures & Algorithms in C++. Assumes you know basic C++ (functions, loops, classes, pointers) but are **new to DSA**.

How to use this course

- Follow the files **in numbered order** — each builds on the previous ones.
- For every topic you get: *what it is, why it's useful, how it works, step-by-step examples, clean C++ code, complexity, when to use it, common mistakes, and practice questions.
- **Do not read the solutions file until you've genuinely tried the problems.** Struggle first — that's where learning happens. Hints are graduated, so read hint 1 → try → hint 2 → try → solution.
- Compile and run every snippet yourself. `g++ -std=c++17 file.cpp -o prog && ./prog`
- Keep a running list of the problems you solved and re-visit them after 1-2 weeks.

File index (study order)

#	File	Topics
0	README.md	This guide
1	01-stl.md	C++ STL containers, iterators, algorithms
2	02-complexity.md	Big-O, time & space complexity, analysis
3	03-arrays.md	Arrays, subarrays, prefix sums, Kadane's
4	04-strings.md	<code>std::string</code> , palindromes, anagrams, matching
5	05-searching.md	Linear & binary search, binary search on answer
6	06-sorting.md	All classic sorts, <code>std::sort</code> , stability, inversions
7	07-hashing.md	<code>unordered_map/set</code> , hash tables, frequency tricks
8	08-two-pointers.md	Opposite & same-direction pointer patterns
9	09-sliding-window.md	Fixed & variable windows, monotonic deque
10	10-linked-lists.md	Singly/doubly lists, reversal, Floyd's cycle
11	11-stacks.md	Stack, monotonic stack, parentheses, postfix
12	12-queues-deques.md	Queue, deque, circular arrays, BFS queues
13	13-recursion.md	Recursion, call stack, divide & conquer
14	14-backtracking.md	Choose-explore-unchoose, permutations, N-Queens
15	15-trees.md	Binary trees, traversals, height, diameter, LCA
16	16-bst.md	Binary search trees, insert/delete, validation
17	17-heaps.md	Heaps, <code>priority_queue</code> , heap sort, top-K
18	18-greedy.md	Greedy strategy, intervals, Huffman, when it works
19	19-graphs.md	Graph representations, BFS, DFS, components, cycles
20	20-shortest-paths.md	Dijkstra, Bellman-Ford, Floyd-Warshall, 0-1 BFS
21	21-mst-topo.md	Topological sort, Kruskal & Prim MST
22	22-dp.md	Dynamic programming, top-down & bottom-up, classic problems
23	23-tries.md	Tries, autocomplete, prefix queries

#	File	Topics
24	24-bit-manipulation.md	Bitwise ops & tricks
25	25-dsu.md	Disjoint Set Union (union-find)
26	26-fenwick.md	Fenwick / Binary Indexed Trees
27	27-segment-trees.md	Segment trees & lazy propagation
28	28-advanced-graphs.md	SCC, bridges, articulation points, matching, flow
29	29-patterns.md	Problem-solving patterns (recognize → template)
30	30-solutions.md	Hints & solutions for all practice questions
31	31-cheatsheet.md	DSA checklist + complexity cheat sheet

Golden rules for learning DSA

1. **Understand over memorize.** Every algorithm is a story: what problem it solves, what invariant it maintains, why it terminates. If you can explain it, you can re-derive it.

2. **Code it, don't just read it.** Reading code feels like learning; typing it is learning.

3. **Draw.** Trees, recursion trees, heap arrays, DP tables — draw them by hand first.

4. **Analyze every solution.** After solving a problem, write its time & space complexity.

If you can't, you don't understand it yet.

5. **Spaced repetition.** Re-solve problems you struggled with after a few days.

6. **Practice order:** Easy → Medium → Challenging, per topic, then mixed (the pattern file).

Final two files: 31-cheatsheet.md is your revision companion; 30-solutions.md is only for after you've tried hard. Good luck — consistency beats intensity.

01 — C++ STL (Standard Template Library)

What it is

The STL is the C++ standard library of **containers** (data structures that hold data), **iterators** (a uniform way to walk through containers), and **algorithms** (functions that operate on containers). It's the single most important thing to learn first, because almost every DSA solution in C++ is built from STL pieces.

Why it's useful

- You rarely need to implement a linked list, hash table, or heap yourself — the STL has battle-tested versions that are fast, correct, and memory-safe.
- It gives you a shared vocabulary: `vector`, `map`, `priority_queue` etc. are the same everywhere.
- Knowing the complexity of each container prevents accidental $O(n^2)$ code.

Containers at a glance

Container	What it is	Access	Insert/Erase	Iteration order
<code>vector<T></code>	Dynamic array	$O(1)$ random (<code>v[i]</code>)	$O(1)$ push/pop back; $O(n)$ middle	Insertion order
<code>array<T, N></code>	Fixed-size array	$O(1)$	—	Insertion order
<code>deque<T></code>	Double-ended queue	$O(1)$ random	$O(1)$ push/pop both ends	Insertion order
<code>list<T></code>	Doubly linked list	$O(n)$	$O(1)$ if you have the iterator	Insertion order
<code>set<T></code>	Sorted unique values (red-black tree)	$O(\log n)$	$O(\log n)$	Sorted
<code>multiset<T></code>	Sorted, duplicates allowed	$O(\log n)$	$O(\log n)$	Sorted
<code>unordered_set<T></code>	Hash set, unique	$O(1)$ avg	$O(1)$ avg	Unspecified
<code>map<K, V></code>	Sorted key→value (tree)	$O(\log n)$	$O(\log n)$	Sorted by key
<code>unordered_map<K, V></code>	Hash key→value	$O(1)$ avg	$O(1)$ avg	Unspecified
<code>stack<T></code>	LIFO	top only	$O(1)$	LIFO
<code>queue<T></code>	FIFO	front/back only	$O(1)$	FIFO
<code>priority_queue<T></code>	Max-heap by default	top only	$O(\log n)$	Sorted (heap order)
<code>pair<A, B> / tuple<...></code>	Fixed bundles of values	<code>.first/.second</code>	—	—

Rule of thumb: `vector` is the default. Only reach for a `map/set` (sorted), an `unordered_*` (hash), a `deque` (push/pop at both ends), or a `priority_queue` (repeatedly need max/min) when the problem demands it.

Key operations (with complexity)

```
#include <bits/stdc++.h>    // quick way to include everything (competitive use)
using namespace std;
```

```
vector<int> v = {3, 1, 4, 1, 5};
v.push_back(9);           // 0(1) amortized
v.pop_back();             // 0(1)
v.size(); v.empty();      // 0(1)
v.back(); v.front();      // 0(1)
sort(v.begin(), v.end()); // 0(n log n)
reverse(v.begin(), v.end());
// vector as stack / common shortcuts:
v.insert(v.begin(), 0);   // 0(n) – avoid when possible
v.erase(v.begin() + 2);  // 0(n)
```

Maps — the two most important facts

```
map<string, int> m;           // sorted, 0(log n) ops
m["apple"]++;               // creates key if missing (operator[] default-inserts)
m.count("apple")            // 1 if present, 0 if not (use count, not operator[] for
checks!)
```

```
unordered_map<string, int> um; // hash, 0(1) average
um["apple"]++;
for (auto& [k, val] : um) { } // structured binding iteration (C++17)
```

operator[] **inserts if the key is missing** — that's why a *check* for presence must use .count(k) or .find(k), not m[k] (which would add the key with value 0).

Sets

```
set<int> s = {3, 1, 2};
s.insert(5);           // 0(log n)
s.erase(1);            // 0(log n)
auto it = s.lower_bound(2); // first element >= 2 → 0(log n)
auto it = s.upper_bound(2); // first element > 2
// lower_bound/upper_bound on sorted containers = binary search
```

Priority queue (heap)

```
priority_queue<int> pq;           // MAX-heap: pq.top() is largest
pq.push(5); pq.pop();            // 0(log n); top() 0(1)

priority_queue<int, vector<int>, greater<int>> minpq; // MIN-heap
// or for custom ordering:
struct Cmp { bool operator()(pair<int,int> a, pair<int,int> b) const {
    return a.second > b.second; } }; // note: > makes it a min-heap on .second
```

Iterators — the universal interface

```
for (auto it = v.begin(); it != v.end(); ++it) { *it; }
for (int& x : v) { ... }           // range-for; note the & to modify
for (int x : v) { ... }           // copies each element (expensive for big objects)
```

Must-know algorithms (<algorithm>)

Call	Effect	Complexity
sort(begin, end)	Sort ascending	O(n log n)
sort(begin, end, cmp)	Sort with custom comparator	O(n log n)
stable_sort	Sort, keeps equal elements' order	O(n log n)
reverse(begin, end)	Reverse range	O(n)
binary_search(b, e, x)	True if x present (sorted input)	O(log n)
lower_bound(b, e, x) / upper_bound	Iterator to first $\geq x$ / $> x$	O(log n)

Call	Effect	Complexity
min/max(a, b); min_element/max_element	Extrema	O(1) / O(n)
accumulate(b, e, init)	Sum	O(n)
count(b, e, x)	Count occurrences	O(n)
find(b, e, x)	First position or end()	O(n)
fill(b, e, x) / fill_n	Set range to value	O(n)
unique(b, e)	Remove *adjacent* duplicates (sort first!)	O(n)
next_permutation(b, e)	Next lexicographic permutation (returns bool)	O(n)
swap(a, b)	Swap values	O(1)
__builtin_popcount(x)	# of set bits (int)	O(1)

```
// Custom comparator: returns true if a should come BEFORE b
sort(v.begin(), v.end(), [](int a, int b) { return a > b; }); // descending
sort(pairs.begin(), pairs.end(), [](auto& a, auto& b) {
    if (a.first != b.first) return a.first < b.first; // by first asc
    return a.second > b.second; // then second desc
});
```

Comparator contract: it must be a *strict weak ordering* — `cmp(a,a)` is false, and `cmp(a,b) && cmp(b,c)` implies `cmp(a,c)`. A comparator that violates this (e.g. returns true for equal elements) is undefined behavior.

Step-by-step example — counting word frequencies

Task: read words, print the most frequent word (ties → lexicographically smallest).

Input: apple banana apple cherry banana apple
Counts: apple:3 banana:2 cherry:1 → answer: apple

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<string> words = {"apple", "banana", "apple", "cherry", "banana", "apple"};
    unordered_map<string, int> freq; // 1. count with hash map
    for (const string& w : words) freq[w]++;

    string best; int bestCount = 0;
    for (auto& [w, c] : freq) { // 2. scan for max
        if (c > bestCount) { best = w; bestCount = c; }
        else if (c == bestCount && w < best) best = w; // tie-break
    }
    cout << best << ": " << bestCount << "\n"; // apple: 3
}
```

Step 1 is O(n) with `unordered_map`. Step 2 is O(k) over distinct words. Total **O(n)**.

When to use what — decision guide

- Need sorted data with fast membership/insert/delete? → `set/map`
- Need fast lookup by key, order irrelevant? → `unordered_set/unordered_map`
- Need only max or min repeatedly? → `priority_queue`
- Need FIFO processing? → `queue`; LIFO? → `stack`; both ends? → `deque`

- Need random access + flexible size? → `vector`
- Building an adjacency list for graphs? → `vector<vector<int>>`
- Need sorted *and* duplicates? → `multiset/multimap`

Common mistakes

1. **Modifying a container while iterating it.** Erasing inside a range-for is UB. Erase via iterator and capture the returned iterator, or collect indices and erase after.
2. **Using `m[k]` to check existence** — silently inserts key `k`.
3. erase **on a set inside a loop** invalidates the iterator: use `it = s.erase(it);` (C++11 and later return the next iterator).
4. **Range-for by value on big objects** copies every element; use `const auto&`.
5. **Assuming `unordered_map` iteration order** — it's arbitrary. Sort keys if order matters.
6. `vector<bool>` **is weird** — it's bit-packed; `&v[0]` doesn't work and references are proxies. Use `vector<char>` or `vector<int>` when you need pointer-like behavior.
7. **Using `std::list` when you want `vector`** — cache-unfriendly, almost always slower.
8. **Comparator returning true for equal elements** — UB in `sort`.

Practice questions

- **Easy 1:** Given an array, print the element that appears most often (ties → smallest value). Justify your complexity.
- **Easy 2:** Given two arrays, print the intersection (distinct common values), then the union.
- **Medium 1:** Given an array of integers, find the `k` most frequent elements.
- **Medium 2:** Given a stream of integers, maintain a structure that supports `add(x)` and `getMedian()` efficiently. (Hint: two `priority_queue`s.)
- **Challenging 1:** Implement a **LRU cache** (get/set in $O(1)$) using `list` + `unordered_map`.
- **Challenging 2:** Given an array, find the length of the longest subarray with **at most `k`** distinct elements (hard without hashing + windowing; revisit after topic 09).

Hints & solutions: file 30-solutions.md.

02 — Time & Space Complexity (Big-O)

What it is

Time complexity describes how the running time of an algorithm grows as the input size n grows.

Space complexity describes how much extra memory it uses. We express both with **Big-O notation**, which describes the *asymptotic* (large- n) growth rate, ignoring constants and lower-order terms.

- $3n^2 + 7n + 1 \rightarrow O(n^2)$
- $5n + 1000 \rightarrow O(n)$
- $2^n + n^{10} \rightarrow O(2^n)$

Why it's useful

- It tells you **whether your solution will pass** before you run it, and what algorithm you need to design.
- It gives an order-independent language to compare algorithms: "sorting is $O(n \log n)$ " is true everywhere.
- It guides *architecture*: $O(n^2)$ on $n = 10^5$ is 10^{10} operations — too slow; you need a better algorithm, not a faster machine.

The growth ladder (memorize the shape, not the formula)

$O(1)$	constant	– array index, hash lookup	instant
$O(\log n)$	logarithmic	– binary search, balanced tree ops	20 steps for $n = 1M$
$O(\sqrt{n})$	sqrt	– factor search up to $\sqrt{n}$	
$O(n)$	linear	– single pass over input	
$O(n \log n)$	linearithmic	– sort, merge, heap	~20M steps for $n = 1M$
$O(n^2)$	quadratic	– nested loops over input	1e12 for $n = 1M \rightarrow$ too slow
$O(n^3)$	cubic	– triple loop / Floyd-Warshall	
$O(2^n)$	exponential	– subsets, brute force	dies at $n \approx 25$
$O(n!)$	factorial	– permutations	dies at $n \approx 11$

Practical budget: a modern computer does roughly **10^8 simple operations/second**. With 1-2 second time limits:

n	Allowed complexity roughly
≤ 10	$O(n!)$
≤ 20	$O(2^n)$
≤ 500	$O(n^3)$
$\leq 5,000$	$O(n^2)$
$\leq 10^6$	$O(n \log n)$
$\leq 10^7$	$O(n)$
10^8+	$O(\log n)$ or $O(1)$, or clever math

These are rough guidelines — constant factors matter too — but they steer your design.

How to analyze code

Rule 1 — Loops multiply

```
for (int i = 0; i < n; i++) { x += a[i]; } // O(n)
for (int i = 0; i < n; i++)
```

```

    for (int j = 0; j < n; j++)                // 0(n²)
        sum += a[i]*b[j];
for (int i = 0; i < n; i += 2) ...              // n/2 iterations → still 0(n)
for (int i = 1; i < n; i *= 2) ...              // log2(n) iterations → 0(log n)

```

Rule 2 — Sequential blocks add

```

for (int i = 0; i < n; i++) doA();    // 0(n)
for (int i = 0; i < n; i++) doB();    // 0(n) → total 0(n) + 0(n) = 0(n)

```

Rule 3 — Function calls add up

Every call to an $O(m)$ function inside an $O(n)$ loop gives $O(n \cdot m)$. **Watch for hidden costs:**
`v.erase(v.begin())` is $O(n)$; `string s = s + c` inside a loop is $O(n)$ per append → $O(n^2)$ total (strings are immutable-ish: each + copies).

Rule 4 — Recursion → recurrence (Master Theorem light)

If $T(n) = a \cdot T(n/b) + f(n)$:

Case	Example	Result
Split in half, one recursive call, $O(1)$ extra	binary search	$T(n) = T(n/2) + O(1) \rightarrow \mathbf{O(\log n)}$
Split in half, two calls, $O(n)$ merge	merge sort	$T(n) = 2T(n/2) + O(n) \rightarrow \mathbf{O(n \log n)}$
Split in half, two calls, $O(1)$ extra	naïve Fibonacci	$T(n) = 2T(n-1) + O(1) \rightarrow \mathbf{O(2^n)}$

Step-by-step example — analyzing three solutions to "sum of array"

Problem: given array `a[0..n-1]`, compute sum of all elements.

Solution A — simple loop:

```

long long sum = 0;
for (int x : a) sum += x;          // n iterations, O(1) each → 0(n) time, O(1) space

```

Solution B — recursive:

```

long long sumRec(int* a, int n) {          // T(n) = T(n-1) + O(1) → 0(n)
    if (n == 0) return 0;
    return a[n-1] + sumRec(a, n-1);        // recursion stack: n frames → 0(n) space
}

```

Time $O(n)$, space $O(n)$ **because of the call stack** — an important distinction from A.

Solution C — prefix sums (precompute once, answer many queries):

```

vector<long long> pref(n + 1, 0);
for (int i = 0; i < n; i++) pref[i+1] = pref[i] + a[i];    // build: 0(n)
// range sum [l, r] = pref[r+1] - pref[l] → 0(1) per query afterwards

```

Build $O(n)$ time + $O(n)$ space; each query $O(1)$. Trading memory for speed — a core DSA idea.

Space complexity notes

- Count **extra** memory beyond the input (though some problems count input too — read the statement). Example: in-place sort = $O(1)$ extra; merge sort = $O(n)$ extra.

- Call stack counts.** Deep recursion (e.g. 10^6 depth) overflows the stack (~8 MB default). Convert to iterative or use an explicit stack when depth is huge.

- Constants still matter in practice: an $O(n)$ algorithm with huge constant can lose to $O(n \log n)$.

Common mistakes

1. $O(2 \cdot n) = O(n)$ — you can't "beat" another $O(n)$ by a constant factor in Big-O terms; but a *smaller constant* can still win in real contests.
2. **Forgetting hidden costs:** string concatenation in a loop ($O(n^2)$), `vector::insert` at front ($O(n)$), `unordered_map` worst case ($O(n)$ per op — adversarial inputs).
3. $\log$ **base doesn't matter** for Big-O: $\log_2 n$ and $\log_{10} n$ differ by a constant.
4. **Confusing best-case with worst-case.** Always state worst case unless specified.
5. n **vs** m : when two inputs exist (e.g. n nodes, m edges), report $O(n + m)$, not $O(n)$.
6. **Underestimating recursion depth** → stack overflow at runtime even though the algorithm is theoretically fine.
7. **Amortized $\neq$ worst case.** `vector::push_back` is $O(1)$ *amortized* — occasional $O(n)$ reallocations spread out. For strict guarantees use `deque` or `pre-allocate`.

Practice questions

- **Easy 1:** Give the time & space complexity of: (a) binary search, (b) printing all pairs (i, j) of an array, (c) printing all subarrays, (d) `next_permutation` loop over all permutations of length n .
- **Easy 2:** Find the largest element in an array — prove it can't be done in less than $O(n)$ comparisons in general.
- **Medium 1:** An algorithm takes $f(n) = n^2/2 + 100n$ microseconds. At what n does it exceed 1 second? Now assume you optimize to $g(n) = 10n \cdot \log_2(n)$ — at what n does it exceed 1 second? What does this teach you about constants vs growth rate?
- **Medium 2:** Analyze the complexity of this loop and explain the result:

```
for (i=1; i<=n; i++) for (j=i; j<=n; j+=i) work();
```

(sum of n/i over i — harmonic series).
- **Challenging 1:** Prove that comparison-based sorting requires $\Omega(n \log n)$ comparisons in the worst case (hint: decision tree with $n!$ leaves).
- **Challenging 2:** Design an algorithm that finds the k -th smallest element of an unsorted array in $O(n)$ expected time (hint: quickselect — revisit after topic 05/06) and analyze it.

Hints & solutions: file 30-solutions.md.

03 — Arrays

What it is

An array is a **contiguous block of memory** holding elements of one type, addressed by an integer index. `int a[5]` and `std::vector<int>` are both arrays; `vector` adds dynamic resizing and safety. Because elements sit side-by-side in memory, reading `a[i]` is **$O(1)$** — the CPU computes `base_address + i * sizeof(element)`.

Why it's useful

- $O(1)$ random access is the reason almost every data structure (heaps, segment trees, hash tables) is *implemented* on top of arrays.
- Arrays model sequences: subarrays, prefixes, rotations — the raw material of thousands of interview problems.
- Cache-friendly: sequential access is fast, which matters in practice.

How it works / key vocabulary

```
a = [ 3 | -1 | 7 | 4 | 9 ]
idx 0   1   2   3   4
```

- **Subarray**: contiguous slice, e.g. `[7,4]` — there are $O(n^2)$ subarrays.
- **Subsequence**: not necessarily contiguous, order preserved, e.g. `[3,7,9]`.
- **Prefix sum** `pref[i] = sum of a[0..i-1]`. Range sum `a[l..r] = pref[r+1] - pref[l]`.
- **Difference array** `diff`: `diff[i] = a[i] - a[i-1]` — supports "add `x` to range `[l,r]`" in $O(1)$ by doing `diff[l]+=x; diff[r+1]-=x`; then one prefix pass to reconstruct.

Pattern 1 — Prefix sums (the single most reused trick)

Problem: given `a`, answer many queries "sum of `a[l..r]`" fast.

Precompute: `pref[i] = pref[i-1] + a[i-1]`

Answer: `sum(l..r) = pref[r+1] - pref[l]`

```
vector<long long> pref(n + 1, 0);
for (int i = 0; i < n; ++i) pref[i+1] = pref[i] + a[i];      // build O(n)

auto rangeSum = [&](int l, int r) { return pref[r+1] - pref[l]; }; // O(1) query

// Example: a = [3,-1,7,4] → pref = [0,3,2,9,13]
// rangeSum(1,2) = pref[3]-pref[1] = 9-3 = 6 = -1+7 ✓
```

Prefix sums generalize: prefix products, prefix XOR, prefix min/max, prefix counts.

Pattern 2 — Kadane's algorithm (maximum subarray sum)

Problem: find the maximum sum of any contiguous subarray. (`a` can contain negatives)

Idea: as we scan, keep best ending at `i`. Either extend the previous subarray (`bestEnd + a[i]`) or start fresh at `a[i]`.

Invariant: `bestEnd[i] = max(a[i], bestEnd[i-1] + a[i])`

Answer = max over all `bestEnd[i]`

```
int kadane(const vector<int>& a) {
    int bestEnd = 0, best = INT_MIN;      // bestEnd = best sum of subarray ending here
    for (int x : a) {
        bestEnd = max(x, bestEnd + x);    // extend or restart
        best = max(best, bestEnd);
    }
}
```

```

    }
    return best;
    [4,-1,2,1])
}
// a = [-2,1,-3,4,-1,2,1,-5,4] → 6 (subarray

```

Time **O(n)**, space **O(1)**. If the problem forbids choosing an empty subarray, start `bestEnd` from `INT_MIN` or use `max(x, bestEnd + x)` as above (handles all-negative input correctly).

Step-by-step example — "two-sum with sorted array"

Problem: array sorted ascending; find indices $i < j$ with $a[i] + a[j] == \text{target}$, or report none.
`a = [2, 7, 11, 15], target = 9`

Approach: two pointers from both ends (detailed in topic 08).

```

left=0 (2), right=3 (15): sum=17 > 9 → move right left
left=0 (2), right=2 (11): sum=13 > 9 → move right left
left=0 (2), right=1 (7): sum=9 == target → answer (0,1)

```

Step-by-step example — rotate array in-place by k

`a = [1,2,3,4,5,6,7]`, `k = 3` → want `[5,6,7,1,2,3,4]`
`k %= n` (`7%3` → wait: 3 stays 3). Trick: reverse parts.
 reverse whole: `[7,6,5,4,3,2,1]`
 reverse `[0,k)`: `[5,6,7,4,3,2,1]`
 reverse `[k,n)`: `[5,6,7,1,2,3,4]` ✓

```

void rotate(vector<int>& a, int k) {
    int n = a.size(); k %= n; if (k == 0) return;
    reverse(a.begin(), a.end());           // O(n)
    reverse(a.begin(), a.begin() + k);     // O(k)
    reverse(a.begin() + k, a.end());       // O(n-k)
}
// Total O(n) time, O(1) extra space – beats the naive O(n·k) shift.

```

Common array mistakes

1. **Off-by-one / out-of-bounds.** `for (int i=1; i<=n; i++) a[i]` on a 0-indexed array. `Vector's at()` throws; `[]` silently corrupts memory.
2. **Integer overflow** in sums: `int sum` overflows around $2 \cdot 10^9$; use `long long` when sums can exceed that. (Classic hidden bug in Kadane / prefix sums.)
3. **Negative indices / size_t underflow:** `a.size()-1` when size is 0 → huge number.
4. **Modifying while iterating** (removing elements shifts indices).
5. **Confusing subarray (contiguous) with subsequence** — different problems, different algorithms.
6. **Ignoring the empty subarray / empty prefix case** in range queries.

Practice questions

- **Easy 1:** Given an array, move all zeros to the end while keeping the relative order of non-zeros (in place, $O(n)$). *Example: `[0,1,0,3,12] → [1,3,12,0,0]`.
- **Easy 2:** Find the **second largest** distinct element of an array in $O(n)$ and $O(1)$ space.
- **Easy 3:** Given a binary array, find the maximum number of consecutive 1s.
- **Medium 1:** Maximum subarray sum (Kadane) — then also print the subarray itself.
- **Medium 2:** Given an array, find **two elements whose sum is closest to a target** (hint: sort + two pointers).

- **Medium 3: Subarray sum equals K:** count subarrays with sum exactly K (hint: prefix sums + hash map — revisit after topic 07).
- **Challenging 1: Maximum product subarray** (hint: negatives flip max/min — keep both).
- **Challenging 2:** Trapping rain water — compute water trapped after rain given heights (hint: two pointers tracking max-left/max-right, or prefix max arrays).
- **Challenging 3:** Given an array, find the longest subarray with sum divisible by k (hint: prefix-sum remainders mod k — revisit after hashing).

Hints & solutions: file 30-solutions.md.

04 — Strings

What it is

A string is a sequence of characters. In C++ you have two flavors:

- **C-strings:** `char s[] / const char*` — null-terminated arrays. Error-prone; avoid.
- `std::string` — a dynamic, safe, $O(1)$ -access string class with rich methods. Use this.

Why it's useful

Strings appear in *everything*: *parsing input, validation, DNA sequences, search engines, natural language, serialization. String problems teach character-level thinking (anagrams, palindromes, frequency counting) and string-matching algorithms that underpin text search.*

Key `std::string` operations

```
string s = "hello";
s.size() / s.length()    // 0(1)
s[i]                     // 0(1) access (0-indexed)
s += 'x'; s.push_back('x'); // 0(1) amortized
s.substr(pos, len)       // 0(len) — copies!
s.find("ll")             // first index or string::npos (0(n·m) worst case)
s.compare(t); s == t      // lexicographic compare
s.insert(pos, t); s.erase(pos, len);
reverse(s.begin(), s.end()); // in place, 0(n)
sort(s.begin(), s.end());    // useful for anagram checks, 0(n log n)
stoi(s), stoll(s), to_string(x) // conversions
isalpha(c), isdigit(c), tolower(c), toupper(c)
```

Performance traps: `s + c` in a loop is $O(n)$ each time $\rightarrow O(n^2)$ total. Prefer `s += c` (appends in $O(1)$ amortized) or build a `vector<char>/ostringstream`. `substr` copies — use `string_view` (C++17) when you only need to read a slice.

Pattern 1 — Frequency counting (anagrams, first-unique-char)

An **anagram** is a rearrangement: "listen" $\leftrightarrow$ "silent". Two strings are anagrams iff their **character frequency maps** are equal.

```
s = "listen"  $\rightarrow$  {l:1,i:1,s:1,t:1,e:1,n:1}
t = "silent"  $\rightarrow$  same  $\rightarrow$  anagram  $\checkmark$ 

bool isAnagram(const string& a, const string& b) {
    if (a.size() != b.size()) return false;
    vector<int> cnt(26, 0); // lowercase letters only
    for (char c : a) cnt[c - 'a']++;
    for (char c : b) cnt[c - 'a']--;
    return all_of(cnt.begin(), cnt.end(), [](int x){ return x == 0; });
}
// 0(n) time, 0(1) space (fixed 26 buckets)
```

Pattern 2 — Palindrome checks & longest palindromic substring

A palindrome reads the same forwards and backwards: "racecar".

Check: two pointers, $i=0$, $j=n-1$, compare `s[i]==s[j]`, move inward. $O(n)$.

Longest palindromic substring — expand around each center (there are $2n-1$ centers: each character and each gap between characters):

```
string longestPalindrome(const string& s) {
```

```

int n = s.size(), start = 0, maxLen = 1;
auto expand = [&](int l, int r) { // returns length of palindrome centered here
    while (l >= 0 && r < n && s[l] == s[r]) { l--; r++; }
    return r - l - 1;
};
for (int i = 0; i < n; ++i) {
    int len1 = expand(i, i); // odd-length center
    int len2 = expand(i, i + 1); // even-length center
    int len = max(len1, len2);
    if (len > maxLen) { maxLen = len; start = i - (len - 1) / 2; }
}
return s.substr(start, maxLen);
}
// O(n2) time, O(1) space. (O(n) exists – Manacher's algorithm – but expand-around-center
// is the version to master first.)

```

Pattern 3 — String matching (naive → hashing/KMP)

Problem: find all occurrences of pattern P (length m) in text T (length n).

Naive: for each i in 0..n-m, compare T[i..i+m-1] with P → O(n·m).

Rabin-Karp idea (hash-based): compute a rolling hash of P and of each window of T, and only compare windows whose hash matches.

```

// Rolling hash with a fixed modulus is contest-standard; use long long to avoid overflow.
bool containsSubstring(const string& t, const string& p) {
    int n = t.size(), m = p.size();
    if (m > n) return false;
    const long long B = 911382323, M = 1e9 + 7; // base and modulus
    vector<long long> powB(n + 1, 1), pref(n + 1, 0);
    for (int i = 0; i < n; ++i) {
        powB[i+1] = powB[i] * B % M;
        pref[i+1] = (pref[i] * B + t[i]) % M; // hash of t[0..i]
    }
    long long hp = 0;
    for (char c : p) hp = (hp * B + c) % M; // hash of pattern

    for (int i = 0; i + m <= n; ++i) {
        long long hw = (pref[i+m] - pref[i] * powB[m] % M + M) % M; // hash of t[i..i+m-1]
        if (hw == hp && t.substr(i, m) == p) return true; // verify to avoid
        collisions
    }
    return false;
}
// O(n + m) average. The extra == check guards against hash collisions.

```

You'll later see **KMP** (topic 28 or patterns) which is O(n+m) worst case with no hashing. For interviews, rolling hash + verification is usually enough.

Step-by-step example — group anagrams

Input: ["eat","tea","tan","ate","nat","bat"]

Sort each word: eat→aet, tea→aet, tan→ant, ate→aet, nat→ant, bat→abt

Group by sorted form: {aet: [eat,tea,ate]}, {ant: [tan,nat]}, {abt: [bat]}

Output: [[eat,tea,ate],[tan,nat],[bat]]

```

vector<vector<string>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> groups;
    for (string& s : strs) {
        string key = s;
        sort(key.begin(), key.end()); // canonical form
        groups[key].push_back(s);
    }
    vector<vector<string>> res;
    for (auto& [_, v] : groups) res.push_back(v);
}

```

```

    return res;
}
// O(n · L log L) time where L = max word length, O(n·L) space.

```

Common mistakes

1. **substr in hot loops** — copies; $O(\text{len})$ each call. Prefer indices/string_view.
2. **Building strings with + in loops** — quadratic. Use += / push_back / ostreamstream.
3. **find returning npos** — always compare `s.find(c) != string::npos`.
4. **Comparing char vs int:** `s[i] - 'a'` for indices (not `s[i]` directly).
5. **Case sensitivity assumptions** — lower-case first, `tolower` when unsure.
6. **Modifying a string while iterating over it** invalidates positions.
7. **Empty-string edge cases** in matching/palindrome code.

Practice questions

- **Easy 1:** Valid palindrome — ignore non-alphanumerics and case ("A man, a plan, a canal: Panama" → true).
- **Easy 2:** Given a string, find the first non-repeating character and return its index (else -1).
- **Easy 3:** Check if two strings are **anagrams** (without sorting — $O(n)$ with counts).
- **Medium 1:** Longest substring without repeating characters (hint: sliding window — topic 09).
- **Medium 2:** Longest palindromic substring (expand around centers).
- **Medium 3:** Given a string of digits, decode count — number of ways to decode "123" (hint: DP — topic 22).
- **Challenging 1:** Given two strings `s` and `t`, return the **minimum window** in `s` containing all characters of `t` (sliding window + counts).
- **Challenging 2:** Given a pattern and a string, check if the string matches the pattern where '*' matches any sequence and '?' matches one char (wildcard matching — DP).

Hints & solutions: file 30-solutions.md.

05 — Searching

What it is

Searching finds a target among data. Two fundamental algorithms:

- **Linear search:** scan every element — $O(n)$, works on anything.
- **Binary search:** repeatedly halve a **sorted** (or monotonic) range — $O(\log n)$.

Why it's useful

Binary search is the workhorse behind $O(\log n)$ solutions and, more importantly, the "**binary search on answer**" pattern: when you can test "is X feasible?" but can't compute the optimum directly, binary search the **value** of the answer. This pattern solves problems that look nothing like "find a number in a list" (see below, and patterns file).

Binary search — the classic

Invariant: we maintain $[lo, hi]$ such that the answer is inside.

At each step: $mid = lo + (hi - lo)/2$ (avoids overflow of $lo+hi$!)

Compare $a[mid]$ with target and cut the search space in half.

```
// Find any index where a contains target (a sorted ascending). Return -1 if absent.
int binarySearch(const vector<int>& a, int target) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {                // note: <=, not <
        int mid = lo + (hi - lo) / 2;    // safe mid: (lo+hi)/2 overflows for huge values
        if (a[mid] == target) return mid;
        else if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
//  $O(\log n)$ ,  $O(1)$  space
```

Step-by-step trace: $a = [1, 3, 5, 7, 9, 11]$, target = 9

$lo=0$ $hi=5$ $mid=2 \rightarrow a[2]=5 < 9 \rightarrow lo=3$

$lo=3$ $hi=5$ $mid=4 \rightarrow a[4]=9 == 9 \rightarrow$ return 4 ✓ (2 comparisons, vs 5 for linear)

Lower bound / upper bound (finding boundaries with duplicates)

For **sorted arrays with duplicates** we often need the **first position $\geq$ target**:

```
// first index i with a[i] >= target (like std::lower_bound); returns a.size() if none
int lowerBound(const vector<int>& a, int target) {
    int lo = 0, hi = (int)a.size();    // hi is "one past the end" – search space [lo, hi)
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < target) lo = mid + 1;    // mid can't be the answer
        else hi = mid;                        // mid might be the answer – keep it in range
    }
    return lo;
}
```

The "**keep or discard mid**" distinction is the #1 source of bugs:

Variant	Condition	Move	Loop
Classic exact match	$a[mid] == t \rightarrow$ return	$lo=mid+1$ / $hi=mid-1$	$lo \leq hi$
<code>lower_bound</code> (first $\geq t$)	$a[mid] < t \rightarrow lo=mid+1$, else $hi=mid$	keep mid	$lo < hi$

Variant	Condition	Move	Loop
upper_bound (first > t)	$a[mid] \leq t \rightarrow lo = mid + 1$, else $hi = mid$	keep mid	$lo < hi$
last index $\leq t$	$a[mid] \leq t \rightarrow lo = mid$, else $hi = mid - 1$	keep mid on success	$lo < hi$ with $mid = (lo + hi + 1) / 2$

Rule: whenever the *answer itself can equal mid, keep mid in the range ($hi = mid$ or $lo = mid$) and use $lo < hi$; whenever mid is provably not the answer, discard it with ± 1 and use $lo \leq hi$. For $lo = mid$ loops, use $mid = (lo + hi + 1) / 2$ to guarantee progress.

`std::lower_bound / std::upper_bound` give you these for free on sorted containers:

```
auto it = lower_bound(a.begin(), a.end(), target); // first >= target
int idx = it - a.begin();
```

Binary search on answer (the pattern that wins interviews)

When: you must minimize/maximize some value X, and "is X achievable?" is easy to test.
Then: binary search X over the feasible range; the predicate is monotonic (false,false,...,true,true or vice versa).

Example — split array largest sum (minimize the max subarray sum):

$a = [7, 2, 5, 10, 8]$, $k = 2 \rightarrow$ split into 2 contiguous parts minimizing the larger part's sum.
Test(cap): can we split into $\leq k$ parts each with sum $\leq cap$?
cap=10 $\rightarrow [7, 2]$? sum $9 \leq 10$ ok, $[5]$? next $10 > 10 \rightarrow$ greedy: parts=[7,2],[5],[10,8]? wait
Greedy: accumulate until adding $a[i]$ exceeds cap, start new part.
cap=10: $[7, 2](9)$, then 5: $9+5=14 > 10 \rightarrow$ new part $[5]$, then 10 $\rightarrow$ new part $[10]$, then 8 $\rightarrow$ new part $[8] = 4$ parts $> k \rightarrow$ infeasible
cap=14: $[7, 2, 5](14)$, $[10, 8](18)$? $10+8=18 > 14 \rightarrow [10], [8] = 3$ parts $> 2 \rightarrow$ infeasible
cap=15: $[7, 2, 5]=14$, $[10]=10$ then 8: $18 > 15 \rightarrow [10], [8] \rightarrow 3$ parts $\rightarrow$ infeasible
cap=16: $[7, 2, 5]=14$, then 10: $24 > 16 \rightarrow [10]=10$, then 8: $18 > 16 \rightarrow [8] \rightarrow 3$ parts $\rightarrow$ infeasible
cap=17: $[7, 2, 5]=14$, $[10]=10$, $10+8=18 > 17 \rightarrow [8] \rightarrow 3 \rightarrow$ infeasible
cap=18: $[7, 2, 5]=14$, $[10, 8]=18 \rightarrow 2$ parts $\leq 2 \rightarrow$ feasible!
cap=19.. : feasible $\rightarrow$ answer 18.

```
int splitArray(vector<int>& a, int k) {
    long long lo = 0, hi = 0;
    for (int x : a) { lo = max(lo, (long long)x); hi += x; } // answer  $\in$  [max, sum]

    auto feasible = [&](long long cap) {
        int parts = 1; long long cur = 0;
        for (int x : a) {
            if (cur + x > cap) { parts++; cur = x; }
            else cur += x;
        }
        return parts <= k;
    };

    while (lo < hi) {
        long long mid = lo + (hi - lo) / 2;
        if (feasible(mid)) hi = mid; // try smaller
        else lo = mid + 1; // too small  $\rightarrow$  need bigger cap
    }
    return (int)lo;
}
//  $O(n \log(\text{sum}))$  – each test is  $O(n)$ ,  $\log(\text{sum})$  iterations.
```

Signature signs: *"minimize the maximum", "maximize the minimum", "smallest X such that...", "k-th something" $\rightarrow$ think binary search on answer. The predicate must be monotonic (once feasible, everything larger stays feasible).

Step-by-step example — sqrt via binary search

Problem: compute `floor(sqrt(x))` for $x = 27$ without library `sqrt`.
 Monotone predicate: $m*m \leq x$ is true then false as m grows.
`lo=0, hi=27`
`mid=13: 169>27 → hi=12 ; mid=6: 36>27 → hi=5 ; mid=2: 4≤27 → lo=3`
`mid=4: 16≤27 → lo=5 ; mid=5: 25≤27 → lo=6 → lo==hi → answer 5 ✓ ( $5^2=25 \leq 27 < 36$ )`

Use long long for `mid*mid`.

Common mistakes

1. **Overflow:** $(lo+hi)/2$ overflows with huge bounds — use $lo + (hi-lo)/2$.
2. **Wrong loop bound** — $lo < hi$ vs $lo \leq hi$; pick by the keep-vs-discard rule above.
3. **Infinite loop** with $lo = mid$ — always use $mid = (lo+hi+1)/2$ there.
4. **Unsorted input** — binary search assumes sorted (or monotonic). Sort first or use linear.
5. **Predicate not monotonic** — binary search on answer gives wrong answers silently.
6. **Off-by-one on the answer range** (e.g. $hi = sum$ vs $sum+1$, $lo = 0$ vs max element).
7. **Integer multiplication overflow** inside the predicate ($mid*mid, mid*x$) — use long long.

Practice questions

- **Easy 1:** Given a **sorted** array with duplicates, find the first and last position of a target (use lower/upper bound ideas). *Example: `[5,7,7,8,8,10]`, `8 → [3,4]`.
- **Easy 2:** Search in a **rotated sorted array** (e.g. `[4,5,6,7,0,1,2]`, find 0) — $O(\log n)$.
- **Medium 1:** Find the **peak element** in an array where $a[i-1] < a[i] > a[i+1]$ for some i ($O(\log n)$; hint: compare $a[mid]$ with $a[mid+1]$ to decide which side to go).
- **Medium 2: Koko eating bananas:** min integer eating speed to finish all piles within h hours (classic binary search on answer).
- **Medium 3:** Find the **smallest missing positive** integer in an unsorted array (hint: cyclic sort or marking — not binary search; think $O(n)$ time, $O(1)$ space).
- **Challenging 1:** Median of two sorted arrays in $O(\log(\min(n,m)))$ (hard — binary search the partition point).
- **Challenging 2:** Given a sorted array of *unknown length* (access via `get(i)` that returns a sentinel beyond the end), find the target in $O(\log n)$.

Hints & solutions: file 30-solutions.md.

06 — Sorting

What it is

Sorting arranges elements in a defined order (usually ascending). It's rarely the **goal** — it's the **enabler: sorted data makes binary search, two pointers, and greedy algorithms possible, and turns "find closest pair" or "merge intervals" into easy problems.*

Why it's useful

- **Order enables $O(\log n)$ search** and linear scans that exploit order.
- Many "unsorted" problems become trivial after one sort ($O(n \log n)$).
- Sorting is the standard example for **divide & conquer** and **inversion counting**.

Comparison table

Sort	Best	Average	Worst	Space	Stable	Notes
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	☐	educational only
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	☐	fewest swaps; finds k-th min cheaply
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	☐	great on nearly-sorted / small n
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	☐	stable; the basis of inversion count
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	☐	fastest in practice (introsort)
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	☐	non-comparison; small integer range
Radix	$O(d \cdot n)$	$O(d \cdot n)$	$O(d \cdot n)$	$O(n)$	☐	digits of fixed width

In practice: use `std::sort` (introsort — hybrid quicksort+heapsort+insertion) almost always. Implement the classic sorts yourself to **understand** them (and for interviews), not to use them.

The two you must deeply understand

Merge sort (divide & conquer, stable)

Idea: split in half, sort each half recursively, merge the two sorted halves.
Invariant: `merge()` takes two sorted halves and produces one sorted array in $O(n)$.

```
void merge(vector<int>& a, int l, int m, int r) {
    vector<int> L(a.begin()+l, a.begin()+m+1), R(a.begin()+m+1, a.begin()+r+1);
    int i = 0, j = 0, k = l;
    while (i < L.size() && j < R.size())
        a[k++] = (L[i] <= R[j]) ? L[i++] : R[j++]; // <= keeps stability
    while (i < L.size()) a[k++] = L[i++];
    while (j < R.size()) a[k++] = R[j++];
}
```

```
void mergeSort(vector<int>& a, int l, int r) {
    if (l >= r) return; // base case: 0 or 1 element
    int m = l + (r - l) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}
// Recurrence  $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$  time,  $O(n)$  extra space
```

Step-by-step: [38, 27, 43, 3]

```
split → [38,27] [43,3]
[38,27] → [38] [27] → merge → [27,38]
[43,3] → [43] [3] → merge → [3,43]
merge [27,38] + [3,43]:
    27 vs 3 → take 3; 27 vs 43 → take 27; 38 vs 43 → take 38; take 43 → [3,27,38,43] ✓
```

Quick sort (partition-based, in-place)

Idea: pick a pivot, partition so everything $\leq$ pivot is left of it, recurse on both sides. The pivot lands in its final sorted position – nothing crosses it later.

```
int partition(vector<int>& a, int lo, int hi) {
    int pivot = a[hi]; // Lomuto: pivot = last element
    int i = lo; // i = boundary of "smaller" region
    for (int j = lo; j < hi; ++j)
        if (a[j] < pivot) swap(a[i++], a[j]); // move smaller elements to the left
    swap(a[i], a[hi]); // pivot to its final position
    return i;
}

void quickSort(vector<int>& a, int lo, int hi) {
    if (lo >= hi) return;
    int p = partition(a, lo, hi);
    quickSort(a, lo, p - 1);
    quickSort(a, p + 1, hi);
}
// Average  $O(n \log n)$ ; worst  $O(n^2)$  when pivot is always min/max (e.g. already sorted input)
// → fix: choose pivot randomly or as median-of-three.
```

Step-by-step: [10, 80, 30, 90, 40, 50, 70] (pivot = 70)

```
j scans: 10<70 swap(i=j=0,10)... after scan: [10,30,40,50,80,90,70]
swap pivot to position 4 → [10,30,40,50,70,90,80] – 70 is final.
Recurse left [10,30,40,50] and right [90,80] → fully sorted. ✓
```

Counting sort (non-comparison, $O(n + k)$)

Use when values are small non-negative integers (e.g. ages 0-120):

```
vector<int> countingSort(const vector<int>& a, int k) { // values in [0, k]
    vector<int> cnt(k + 1, 0), out(a.size());
    for (int x : a) cnt[x]++;
    for (int i = 1; i <= k; i++) cnt[i] += cnt[i-1]; // cumulative = positions
    for (int i = (int)a.size() - 1; i >= 0; i--) { // backward pass → stable
        out[--cnt[a[i]]] = a[i];
    }
    return out;
}
//  $O(n + k)$  time,  $O(k)$  space. Stable. Radix sort applies it per digit for bigger ranges.
```

Custom comparators and stability

```
vector<pair<int,int>> v = {{3,1},{3,2},{1,5}};
sort(v.begin(), v.end()); // sorts by first, then second
sort(v.begin(), v.end(), [](auto& x, auto& y) {
    return x.second > y.second; // sort by second descending
});
```

```
});
// `stable_sort` keeps original order for equal keys — matters for "sort by time, keep
// insertion order for ties" problems.
```

Step-by-step example — merge intervals (sorting unlocks the solution)

Problem: given intervals `[[1,3],[2,6],[8,10],[15,18]]`, merge overlapping ones.

Sort by start → `[[1,3],[2,6],[8,10],[15,18]]`

Take `[1,3]`; next `[2,6]`: $2 \leq 3$ (overlap) → merge → `[1,6]`; next `[8,10]`: $8 > 6$ → push `[1,6]`, start `[8,10]`...

Result: `[[1,6],[8,10],[15,18]]` ✓

```
vector<vector<int>> merge(vector<vector<int>>& iv) {
    sort(iv.begin(), iv.end());           // by start (then end)
    vector<vector<int>> res;
    for (auto& cur : iv) {
        if (res.empty() || cur[0] > res.back()[1])
            res.push_back(cur);           // no overlap → new interval
        else
            res.back()[1] = max(res.back()[1], cur[1]); // overlap → extend end
    }
    return res;
}
// O(n log n) due to sort; O(n) for the scan.
```

Common mistakes

1. **Using an $O(n^2)$ sort when the problem needs $O(n \log n)$** — $n = 10^5$ is your usual signal.
2. **Comparator violating strict weak ordering** (e.g. `return a <= b`) — UB in `std::sort`.
3. **Assuming `std::sort` is stable** — it isn't; use `stable_sort` when order of equals matters.
4. **Forgetting that sorting destroys original order** — copy first if indices matter.
5. **Off-by-one in partition / merge indices.**
6. **int overflow in counting sort's cumulative sums or in comparisons of large products.**
7. **Negative values in counting sort** — shift by min value first.

Practice questions

- **Easy 1:** Sort an array of 0s, 1s, and 2s in $O(n)$ (Dutch national flag — hint: 3 pointers).
- **Easy 2:** Given two sorted arrays, merge them **in place** into the first array (which has extra space at the end) — $O(m + n)$.
- **Medium 1:** Count **inversions** in an array (hint: merge sort counts during merge — when an element from the right half is taken, all remaining left elements invert with it).
- **Medium 2:** Given an unsorted array, find the k -th smallest element in **$O(n)$ average** (quickselect — quicksort's partition, recurse into one side only).
- **Medium 3: Largest number** formed by concatenating an integer array: `[3,30,34,5,9]` → "9534330" (hint: comparator `x+y > y+x`).
- **Challenging 1: Maximum gap** between successive elements in sorted order in $O(n)$ (hint: pigeonhole + buckets).
- **Challenging 2:** Given an array, find the length of the **longest increasing subsequence** in $O(n \log n)$ (hint: patience sorting with a tail array — revisit after topic 22 if stuck).

Hints & solutions: file 30-solutions.md.

07 — Hashing (Hash Tables)

What it is

A hash table stores key→value pairs in an array using a **hash function** $h(\text{key})$ that maps a key to an index. To look up a key, compute $h(\text{key})$ and go straight to that slot — no scanning. In C++: `unordered_map` (key→value) and `unordered_set` (keys only).

Why it's useful

- **$O(1)$ average** insert / lookup / delete. This single fact upgrades countless algorithms: "check if we've seen this before" goes from $O(n)$ (linear scan) to $O(1)$ (hash).
- The canonical tool for **frequency counting**, **pairing/finding complements**, and **deduplication**.

How it works

Keys: "apple" "banana" "cherry"
hash: $h(\text{"apple"})=2$ $h(\text{"banana"})=0$ $h(\text{"cherry"})=2$ ← collision!

Collisions happen (two keys → same index). Two standard resolutions:

- **Chaining:** each slot holds a linked list of entries; worst case $O(n)$ per op.
- **Open addressing:** probe forward to the next free slot.

C++ `unordered_*` uses chaining and **rehashes** (grows the bucket array) when the **load factor** ($n / \text{buckets}$) exceeds a threshold (~ 1.0), keeping ops $O(1)$ amortized. **Worst case is $O(n)$** — with an adversarial hash or a broken hash function, everything lands in one bucket. (C++ uses per-process random seeds for string/integer hashing, mitigating this in practice.)

Core patterns

1. Frequency counting — $O(n)$

```
unordered_map<int, int> freq;
for (int x : a) freq[x]++;
// find most frequent, count distinct, check duplicates, etc. — all  $O(n)$  with hashing.
```

2. Complement lookup — "two sum" for unsorted arrays

Problem: find $i \neq j$ with $a[i] + a[j] == \text{target}$.

Idea: as we scan, store seen values → for each $a[i]$, look up $\text{target} - a[i]$ in $O(1)$.

```
vector<int> twoSum(vector<int>& a, int target) {
    unordered_map<int, int> seen; // value → index
    for (int i = 0; i < (int)a.size(); ++i) {
        int need = target - a[i];
        if (seen.count(need)) return {seen[need], i};
        seen[a[i]] = i;
    }
    return {};
}
//  $O(n)$  time,  $O(n)$  space. (vs  $O(n \log n)$  sort + pointers — hash wins when indices matter.)
```

3. Prefix-sum + hash — "subarray sum equals k"

Problem: count subarrays whose sum equals k .

Key insight: subarray sum $(i..j) = \text{pref}[j+1] - \text{pref}[i]$. We need $\text{pref}[j+1] - \text{pref}[i] = k$
↪ $\text{pref}[i] = \text{pref}[j+1] - k$. So while scanning, count how many previous prefix sums

equal (current_prefix - k). Also prefix sums equal to k itself count (pref[i] = 0 case).

```
int subarraySum(vector<int>& a, int k) {
    unordered_map<long long, int> cnt;           // prefix sum → how many times seen
    cnt[0] = 1;                                // empty prefix
    long long pref = 0; int ans = 0;
    for (int x : a) {
        pref += x;
        ans += cnt[pref - k];                  // previous prefix sums completing subarrays
        cnt[pref]++;
    }
    return ans;
}
// O(n) time, O(n) space. Example: a=[1,2,3], k=3 → subarrays [1,2] and [3] → 2.
```

This **same** trick handles "longest subarray with sum k", "subarray with sum divisible by k" (store remainders), and "number of subarrays with XOR k" (store prefix XORs).

4. Deduplication & set ops

```
unordered_set<int> s(a.begin(), a.end());      // distinct values
// intersection: iterate the smaller set, check membership in the larger → O(min(n,m))
```

Hashable vs not

`unordered_map<K,V>` needs K to have a `std::hash<K>` — built in for ints, strings, pointers, etc. **Not** for `pair<int,int>` or custom structs by default. Options:

```
// 1. Encode into a single hashable key:
unordered_map<long long, int> m;               // key = x * 1'000'003LL + y (be careful)
// 2. Provide a custom hash:
struct PairHash {
    size_t operator()(const pair<int,int>& p) const {
        return ((size_t)p.first << 32) ^ (size_t)p.second;
    }
};
unordered_map<pair<int,int>, int, PairHash> m;
```

Step-by-step example — first repeating element

a = [2, 5, 1, 2, 3, 5] → first element that repeats (2 appears again earliest): answer 2
Scan left to right, inserting into a set:
2 → new; 5 → new; 1 → new; 2 → ALREADY SEEN → answer 2 ✓

```
int firstRepeated(const vector<int>& a) {
    unordered_set<int> seen;
    for (int x : a) {
        if (seen.count(x)) return x;          // first one found scanning left→right
        seen.insert(x);
    }
    return -1;
}
```

Common mistakes

1. **Using `m[k]` to check existence** — inserts key k with default value. Use `.count()`.
2. **Iterating `unordered_map` and expecting sorted/insertion order** — it's arbitrary.
3. **Mutating keys after insertion** — `unordered_set` position is stale → broken lookups.
4. **Assuming worst case $O(1)$** — adversarial input can make it $O(n)$ per op; when a strict bound matters and keys are comparable, `map/set` ($O(\log n)$, guaranteed) may be safer.
5. **Custom struct keys without a hash** — compile error; add `std::hash` specialization.

6. **Overflow** in hash values or encoded keys (use `long long`).
7. **Forgetting the** `cnt[0] = 1` **seed** in prefix-sum problems — empty prefix is real.

Practice questions

- **Easy 1:** Find if an array contains duplicates (any value appearing twice).
- **Easy 2:** Given two strings, check if they're anagrams using one hash table pass.
- **Easy 3:** Find the element with the **highest frequency** (ties → smaller value).
- **Medium 1:** Subarray sum equals K (pattern above). Then the **longest** subarray with sum K variant.
- **Medium 2:** Group anagrams (topic 04) — redo using hashing only.
- **Medium 3:** Given two arrays, find their intersection with counts (multiset semantics), e.g. `[1,2,2,1]` & `[2,2]` → `[2,2]`.
- **Challenging 1:** Longest consecutive sequence in $O(n)$: `[100,4,200,1,3,2]` → 4
(hint: put all in a set; only start counting from numbers with no left neighbor).
- **Challenging 2: Top K frequent elements** in $O(n \log k)$ (hint: freq map + heap — topic 17).

Hints & solutions: file 30-solutions.md.

08 — Two Pointers

What it is

Two pointers is a technique where **two indices** traverse a sequence, cooperating to solve a problem in a single pass. Two flavors:

- **Opposite direction:** l starts at the left, r at the right; they move toward each other (classic on sorted arrays).
- **Same direction:** both start at the left; one runs ahead (fast/slow, or "window" pointers — the basis of sliding window in topic 09).

Why it's useful

It converts many $O(n^2)$ brute forces (nested loops over pairs or subarrays) into **$O(n)$** single-pass solutions, using only **$O(1)$ extra space**. It's the go-to for sorted-array pair/triplet problems, and its same-direction variant powers in-place array rewriting.

Pattern 1 — Opposite direction (sorted array pair problems)

Problem: given sorted array, find pair ($i < j$) with $a[i] + a[j] == \text{target}$, or report none.
Why it works: if $\text{sum} > \text{target}$, the only way to shrink the sum is to move right pointer left (moving left pointer right only increases it). If $\text{sum} < \text{target}$, move left right. Each step eliminates one candidate index permanently $\rightarrow$ at most n steps.

```
pair<int,int> twoSumSorted(const vector<int>& a, int target) {
    int l = 0, r = (int)a.size() - 1;
    while (l < r) {
        int s = a[l] + a[r];
        if (s == target) return {l, r};
        else if (s < target) l++;      // need bigger sum
        else r--;                     // need smaller sum
    }
    return {-1, -1};
}
//  $O(n)$  time,  $O(1)$  space
```

Step-by-step: $a = [2, 7, 11, 15]$, $\text{target} = 9$

```
l=0(2) r=3(15)  $\rightarrow 17 > 9 \rightarrow r--$ 
l=0(2) r=2(11)  $\rightarrow 13 > 9 \rightarrow r--$ 
l=0(2) r=1(7)  $\rightarrow 9 == 9 \rightarrow (0,1) \checkmark$ 
```

This pattern extends naturally:

- **3Sum:** fix one element, two-pointer the rest $\rightarrow O(n^2)$ instead of $O(n^3)$.
- **Container with most water / trapping rain water:** move the pointer at the *shorter side, because the shorter side caps the result.
- **Two-sum closest to target:** track best as you go.

Pattern 2 — Same direction (fast/slow, in-place rewriting)

Problem: remove duplicates from sorted array in place, return new length.
Idea: slow pointer marks where the next kept element goes; fast scans for new values.
Invariant: $a[0..\text{slow}-1]$ is the cleaned prefix.

```
int removeDuplicates(vector<int>& a) {
    if (a.empty()) return 0;
    int slow = 1;                                // first kept position is index 0
    for (int fast = 1; fast < a.size(); fast++)
        if (a[fast] > a[slow-1]) a[slow++] = a[fast];
    return slow;
}
```

```

    for (int fast = 1; fast < (int)a.size(); ++fast)
        if (a[fast] != a[slow - 1]) // new distinct value
            a[slow++] = a[fast];    // write it at the slow position
    return slow;                  // new length; a[0..slow-1] valid
}
// O(n) time, O(1) space

```

Step-by-step: a = [0,0,1,1,1,2,2,3]

```

fast=1: a[1]=0 == a[0] → skip
fast=2: a[2]=1 != a[0] → a[1]=1, slow=2
fast=3: a[3]=1 == a[1] → skip ; fast=4: == a[1] → skip
fast=5: a[5]=2 != a[1] → a[2]=2, slow=3
fast=6: == a[2] → skip ; fast=7: a[7]=3 != a[2] → a[3]=3, slow=4
→ a = [0,1,2,3,...], length 4 ✓

```

Other same-direction uses: move zeros to end, partition arrays, linked-list cycle detection (Floyd's — topic 10), finding the middle of a list.

Pattern 3 — Merging / comparing two sequences

Problem: merge two sorted arrays (like merge sort's merge step).

Idea: pointer i in A, j in B; at each step take the smaller front element.

```

vector<int> mergeSorted(const vector<int>& a, const vector<int>& b) {
    vector<int> res; res.reserve(a.size() + b.size());
    int i = 0, j = 0;
    while (i < a.size() && j < b.size())
        res.push_back(a[i] <= b[j] ? a[i++] : b[j++]);
    while (i < a.size()) res.push_back(a[i++]);
    while (j < b.size()) res.push_back(b[j++]);
    return res;
}
// O(n + m)

```

When to reach for two pointers

- Sorted input + "find pair/triplet with property" → opposite direction.
- "Do it in place with O(1) space" → same-direction rewriting.
- Two sorted sequences to combine/compare → merge pattern.
- Linked list problems (cycle, middle, k-th from end) → fast/slow.

Common mistakes

1. **Loop condition** $l \leq r$ **where** $l < r$ **is required** (a single element is not a pair).
2. **Moving the wrong pointer** — the rule is: move the side whose move *can* improve the result (in pair-sum: sum too big → move right; too small → move left).
3. **Forgetting the input must be sorted** for the opposite-direction pattern.
4. **Not skipping duplicates** in 3Sum-style problems → duplicate answers.
5. **Index drift / off-by-one** when writing with `slow` then reading with `slow-1`.
6. **Assuming O(n) but accidentally rescanning** from `l` each iteration — that's $O(n^2)$.
7. **Overflow:** $a[l] + a[r]$ may overflow int — use `long long` in sums.

Practice questions

- **Easy 1:** Two-sum on a **sorted** array (must be O(n), two pointers).
- **Easy 2:** Check if a string is a palindrome (pointers from both ends; skip non-alphanumerics).

- **Easy 3:** Given two sorted arrays, find their intersection (distinct values) — $O(n+m)$.
- **Medium 1: 3Sum** — all unique triplets summing to zero.
- **Medium 2: Container with most water** — heights array, maximize area between two lines.
- **Medium 3: Trapping rain water** — total water trapped (two-pointer or prefix-max).
- **Challenging 1: 4Sum** — all unique quadruplets summing to target ($O(n^3)$).
- **Challenging 2:** Smallest window in an array of 0/1 that, when flipped, maximizes the longest run of 1s (hint: this is really a sliding window problem — topic 09).

Hints & solutions: file 30-solutions.md.

09 — Sliding Window

What it is

A sliding window is a **contiguous subarray** that moves over the sequence one step at a time. Instead of recomputing the window's aggregate from scratch each time ($O(n^2)$ total), you **add the entering element and remove the leaving element** — $O(1)$ per step, **$O(n)$ total**.

Why it's useful

Any problem asking for the *best subarray satisfying a property* (longest/shortest subarray with sum $\leq k$, max of windows of size k , substrings with at most k distinct chars) is $O(n^2)$ brute force but $O(n)$ with a sliding window. It's a top-5 interview pattern.

Two kinds

Kind	Window size	Mechanics	Example
Fixed	exactly k	add right, remove left every step	max sum of k consecutive
Variable	grows/shrinks	expand right; shrink left while property violated	longest subarray, sum $\leq k$

Pattern 1 — Fixed window: maximum sum of subarrays of size k

$a = [1, 4, 2, 10, 23, 3, 1, 0, 20]$, $k = 4$
Window sums: 17, 39, 38, 37, 27, 24 → answer 39
Brute force recomputes each window sum: $O(n \cdot k)$.
Sliding: sum of next window = previous sum - leaving + entering → $O(n)$.

```
int maxSumFixed(const vector<int>& a, int k) {
    int n = a.size();
    long long sum = 0;
    for (int i = 0; i < k; i++) sum += a[i];    // first window
    long long best = sum;
    for (int i = k; i < n; i++) {
        sum += a[i] - a[i - k];                // slide: add right, remove left
        best = max(best, sum);
    }
    return best;
}
// O(n) time, O(1) space
```

Pattern 2 — Variable window: longest subarray with sum $\leq k$ (all positive)

$a = [3, 1, 2, 7, 4, 2, 1, 1, 5]$, $k = 8$
Idea: right pointer extends the window; while sum $> k$, advance left to shrink.
The window $[l..r]$ is always the longest ending at r with sum $\leq k$.
Answer = max over r of $(r - l + 1)$.

```
int longestAtMostK(const vector<int>& a, int k) {    // assumes non-negative elements
    int l = 0, n = a.size();
    long long sum = 0; int best = 0;
    for (int r = 0; r < n; ++r) {
        sum += a[r];                                // expand
        while (sum > k) {                            // shrink until valid
            sum -= a[l]; ++l;
        }
        best = max(best, r - l + 1);
    }
}
```

```

    return best;
}
// O(n): each element enters once (r) and leaves once (l).

```

Step-by-step (a = [3,1,2,7,4], k = 8):

```

r=0: sum=3 → [3] len1; r=1: sum=4 → [3,1] len2; r=2: sum=6 → [3,1,2] len3
r=3: sum=13 >8 → remove 3 (l=1, sum=10) → remove 1 (l=2, sum=9) → remove 2 (l=3, sum=7) → [7] len1
r=4: sum=11 >8 → remove 7 (l=4, sum=4) → [4] len1
best = 3 ✓

```

Negative elements? The while(sum > k) shrink trick breaks with negatives (removing an element could be bad but a later one worse). With negatives, use a different tool: prefix sums + binary search on answer, or a deque-based approach. Know when the standard window applies: **non-negative (or all-same-sign) arrays**.

Pattern 3 — Variable window with a *condition counter: *longest substring with k distinct chars*

Generalize: instead of tracking a sum, track the number of distinct chars in the window and shrink while it exceeds k.

```

int longestKDistinct(const string& s, int k) {
    int l = 0, best = 0;
    vector<int> cnt(26, 0); int distinct = 0;
    for (int r = 0; r < (int)s.size(); ++r) {
        if (cnt[s[r]-'a']++ == 0) distinct++; // new distinct char enters
        while (distinct > k) {
            if (--cnt[s[l]-'a'] == 0) distinct--; // a char fully leaves
            ++l;
        }
        best = max(best, r - l + 1);
    }
    return best;
}

```

This "shrink while condition violated" skeleton covers: longest substring without repeating chars, minimum window substring (shrink while *satisfied, track min), longest subarray with at most k zeros, etc. **The only differences are what you count and when you shrink.**

Pattern 4 — Monotonic deque: max in every window of size k (O(n))

Problem: for each window of size k, output its maximum. Naive: $O(n \cdot k)$. Heap: $O(n \log k)$.

Monotonic deque keeps candidate maxima in decreasing order → $O(n)$ total.

Idea: maintain a deque of indices; values strictly decreasing. When a new element arrives, pop from back while it's $\leq$ new value (they can never be the max of a future window).

Pop from front while the index falls outside the window. Front is always the window max.

```

vector<int> maxSlidingWindow(const vector<int>& a, int k) {
    deque<int> dq; // indices, values strictly decreasing
    vector<int> res;
    for (int r = 0; r < (int)a.size(); ++r) {
        while (!dq.empty() && a[dq.back()] <= a[r]) dq.pop_back(); // remove dominated
        dq.push_back(r);
        if (dq.front() <= r - k) dq.pop_front(); // expired index
        if (r >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}
// O(n): each index pushed and popped at most once. a=[1,3,-1,-3,5,3,6,7], k=3 → [3,3,5,5,6,7]

```

Step-by-step (a = [1,3,-1,-3], k=3):

```

r=0: dq=[0] (1)
r=1: 3>1 → pop 0, dq=[1] (3)
r=2: -1 ≤ 3 → push, dq=[1,2] (3,-1); front 1 ≥ 0 → window [1,3,-1] max 3 ✓
r=3: -3 ≤ -1 → push, dq=[1,2,3]; front 1 ≤ 0? No (r-k=0, 1>0 keep) → max 3 ✓

```

Common mistakes

1. **Shrinking when you should be expanding** — always expand r first, then shrink l .
2. **Missing the shrink loop** (while not if) — one removal may not restore validity.
3. **Off-by-one in window size** ($r - l + 1$ vs $r - l$).
4. **Using the fixed-window pattern on variable-window problems** and vice versa.
5. **Applying sum-window with negative elements** — the monotonicity assumption breaks.
6. **Deque pitfalls:** forgetting to pop expired indices from the front, or using values

where indices are needed (duplicates!).

7. **Forgetting that hash-based distinct counting is $O(1)$ amortized but has worst-case cost**

— with big alphabets use arrays/`unordered_map` and shrink correctly.

8. **Integer overflow** on window sums — `long long`.

Practice questions

- **Easy 1:** Maximum sum of any contiguous subarray of size exactly k .
- **Easy 2:** Given a binary array, find the **maximum number of consecutive 1s** if you can flip at most one 0 to 1 (window with at most one zero).
- **Medium 1:** Longest substring **without repeating characters**.
- **Medium 2:** Longest subarray with **at most k distinct elements**.
- **Medium 3: Minimum window substring** (all chars of t in s — hard-ish but a canonical window problem).
- **Challenging 1:** Max of every window of size k **with the monotonic deque** in $O(n)$.
- **Challenging 2:** Given an array, count the number of subarrays whose **maximum is exactly X** (**hint: count subarrays with $\max \leq X$, subtract for $< X$ — monotonic stack may also help; revisit after topic 11**).

Hints & solutions: file 30-solutions.md.

10 — Linked Lists

What it is

A linked list is a sequence of **nodes**, each holding a value and a pointer to the next node (singly linked), plus optionally a pointer to the previous node (doubly linked).

Singly: [3]→[7]→[1]→null Doubly: null↔[3]↔[7]↔[1]↔null

Why it's useful

- **O(1) insertion/deletion *at a known position*** (given the node/iterator) — unlike a vector, which must shift elements.
 - The natural model for: undo history, browser back/forward, adjacency lists (as lists), hash-table chaining, LRU caches.
 - **Interview gold:** pointer manipulation is where null-handling and edge cases live.
- Reality check: vectors are usually faster in practice (cache locality), so the STL's list is niche. But you must know linked-list *algorithms* cold for interviews and for building other structures.*

Core operations

```
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int v) : val(v), next(nullptr) {}
};
```

Insert at head (O(1)) / delete at head (O(1))

```
ListNode* insertFront(ListNode* head, int v) {
    ListNode* node = new ListNode(v);
    node->next = head;
    return node;           // NEW head – return it! Callers must reassign.
}
ListNode* deleteFront(ListNode* head) {
    if (!head) return nullptr;
    ListNode* nxt = head->next;
    delete head;           // (in interviews memory cleanup is often skipped)
    return nxt;
}
```

The dummy node — your best friend

Problem: "delete the k-th node from the end", "reverse a sublist", "merge two lists" — all need to handle the head itself changing. A dummy node before the head makes the head a normal middle node → no special-casing.

```
ListNode* deleteKthFromEnd(ListNode* head, int k) {
    ListNode dummy(0); dummy.next = head;
    ListNode* fast = &dummy;
    for (int i = 0; i < k; ++i) fast = fast->next; // advance fast by k
    ListNode* slow = &dummy;
    while (fast->next) { slow = slow->next; fast = fast->next; } // then together
    ListNode* toDelete = slow->next; // k-th from end
    slow->next = toDelete->next; // unlink
    delete toDelete;
    return dummy.next; // new head (safe even if head removed)
}
// O(n) time, O(1) space
```

Pattern 1 — Reversal (in place, iterative)

Idea: walk with three pointers prev, cur, nxt. At each step, point cur->next = prev, then advance all three. Invariant: everything before cur is reversed.

```
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* cur = head;
    while (cur) {
        ListNode* nxt = cur->next;    // save next BEFORE rewiring
        cur->next = prev;             // rewire
        prev = cur; cur = nxt;        // advance
    }
    return prev;                      // new head (old tail)
}
// O(n) time, O(1) space. Recursive version exists – see topic 13.
```

Step-by-step: 1→2→3→null

```
cur=1: nxt=2, 1→null; prev=1, cur=2
cur=2: nxt=3, 2→1;   prev=2, cur=3
cur=3: nxt=null, 3→2; prev=3, cur=null → return 3 → 3→2→1→null ✓
```

Pattern 2 — Floyd's cycle detection (tortoise & hare)

Idea: slow moves 1 step, fast moves 2 steps. If there's a cycle, they MUST meet (in $\leq n$ steps). If fast hits null → no cycle.

```
bool hasCycle(ListNode* head) {
    ListNode* slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;    // met → cycle
    }
    return false;
}
// O(n) time, O(1) space (vs O(n) space for a visited set)
```

Find the cycle's start: after they meet, move slow to head and advance both one step — they meet again at the cycle entry (proven by pointer-arithmetic on the cycle length).

Pattern 3 — Fast/slow for the middle

```
ListNode* middleNode(ListNode* head) {    // for even length, returns 2nd middle
    ListNode* slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    return slow;                          // when fast reaches end, slow is at middle
}
```

Pattern 4 — Merge two sorted lists

```
ListNode* mergeTwo(ListNode* l1, ListNode* l2) {
    ListNode dummy(0), *tail = &dummy;
    while (l1 && l2) {
        if (l1->val <= l2->val) { tail->next = l1; l1 = l1->next; }
        else { tail->next = l2; l2 = l2->next; }
        tail = tail->next;
    }
    tail->next = l1 ? l1 : l2;              // attach the remainder
    return dummy.next;
}
// O(n + m)
```

Common mistakes

1. **Losing the head:** after `head = head->next` inside a loop, you can't get back. Keep a copy or use a dummy.
2. **Dereferencing null:** check `if (!node || !node->next)` before `node->next->val`.
3. **Rewiring before saving** `next` in reversal — classic corrupted list.
4. **Forgetting that insertion at head changes the head** — return the new head.
5. **Cycles from careless rewiring** — infinite loops; test on small lists by hand.
6. **Infinite loops in while conditions** when the list has a cycle and you don't use tortoise-hare or a visited set.
7. **Off-by-one in k-from-end** ($k=1$ is the last node, not null).
8. **Doubly-linked list:** forgetting to fix the previous pointer of the `*next*` node.

Practice questions

- **Easy 1:** Reverse a singly linked list (iterative, then recursive).
- **Easy 2:** Detect a cycle; then find the node where the cycle begins.
- **Easy 3:** Merge two sorted linked lists.
- **Medium 1:** Remove the k -th node from the end in one pass.
- **Medium 2:** Given the head of a sorted list, remove **all** duplicates that appear more than once (not keeping one copy) — $1 \rightarrow 2 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1 \rightarrow 4$.
- **Medium 3:** Reorder list $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n \rightarrow L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{(n-1)} \rightarrow \dots$ (find middle, reverse second half, interleave).
- **Challenging 1:** Merge k sorted linked lists in $O(n \log k)$ (heap — topic 17).
- **Challenging 2:** Copy a list with **random pointers** (each node has `next` and a random pointer to any node) in $O(n)$ — hint: interleave copies, or hash map `node → clone`.
- **Challenging 3:** Implement an **LRU cache** with $O(1)$ get/put using a doubly linked list + hash map.

Hints & solutions: file 30-solutions.md.

11 — Stacks

What it is

A stack is a **LIFO** (Last-In, First-Out) structure: the most recently pushed element is the first popped. Think of a stack of plates — you take from the top.

push(1) push(2) push(3) → top=3 → pop → 3, top=2

std::stack<T> wraps a deque by default. Core ops, all **O(1)**:

```
stack<int> st;
st.push(x);    st.pop();           // pop returns void — read top() first!
int t = st.top();           // top element without removing
st.empty(); st.size();
```

Why it's useful

A stack is the natural model for anything with a **"must undo the most recent thing first"** structure: function calls (the call stack!), expression evaluation, bracket matching, and — most important for DSA — the **monotonic stack** pattern (next greater element, and everything derived from it).

Pattern 1 — Balanced parentheses

Problem: is "[{}]" balanced? Every closing bracket must match the most recent unmatched opening bracket — exactly LIFO.

```
bool isValid(const string& s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else {
            if (st.empty()) return false;           // closing with nothing open
            char open = st.top();
            if ((c == ')' && open != '(') ||
                (c == ']' && open != '[') ||
                (c == '}' && open != '{')) return false;
            st.pop();
        }
    }
    return st.empty();                             // leftover opens → unbalanced
}
// O(n) time, O(n) space
```

Step-by-step: s = "[{}]"

push {, push [, push (→ top (; see) → matches (→ pop ; see] → matches [→ pop
see } → matches { → pop → stack empty → balanced ✓

Pattern 2 — Monotonic stack: next greater element

Problem: for each element, find the index of the next element to the RIGHT that is strictly greater (or -1). Brute force $O(n^2)$.

Idea: maintain a stack of indices whose "next greater" isn't known yet. When we see a[i] greater than the element at the stack top, we've found the answer for it — pop and record.
Invariant: values in the stack are strictly decreasing (from bottom to top) — a "monotonic decreasing stack".

```
vector<int> nextGreater(const vector<int>& a) {
    int n = a.size();
    vector<int> res(n, -1);
    stack<int> st;           // indices, values strictly decreasing
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && a[st.top()] < a[i]) {
```

```

        res[st.top()] = a[i];          // a[i] is the next greater for st.top()
        st.pop();
    }
    st.push(i);
}
return res;
}
// O(n): each index pushed/popped once.

```

Step-by-step: a = [4, 5, 2, 25]

```

i=0: st=[0] (4)
i=1: 5>4 → res[0]=5, pop; st=[1] (5)
i=2: 2<5 → push → st=[1,2] (5,2)
i=3: 25>2 → res[2]=25, pop; 25>5 → res[1]=25, pop; st=[3]
→ res = [5, 25, 25, -1] ✓

```

Variants (all the same code, tweaked):

- *Previous* greater: scan right→left. *Next/previous smaller:* flip the comparison.*
- **Largest rectangle in a histogram:** for each bar, find the nearest smaller bar on each side — the bar is the minimum of that span. Monotonic stack in O(n).
- **Daily temperatures** (days until warmer), **sliding window max** (alternative to deque).

Pattern 3 — Expression evaluation

Infix "3 + 4 * 2" → operators have precedence. Two stack algorithms:

1. Two-stack (operands + operators) – shunting-yard style, handles precedence.
2. Convert to postfix "3 4 2 * +" then evaluate with ONE stack:
push numbers; on operator, pop two, compute, push result.

```

int evalPostfix(const vector<string>& tokens) {    // e.g. ["3","4","2","*","+"]
    stack<int> st;
    for (const string& t : tokens) {
        if (t == "+" || t == "-" || t == "*" || t == "/") {
            int b = st.top(); st.pop();           // b = SECOND operand (order matters for -
and /)
            int a = st.top(); st.pop();
            if (t == "+") st.push(a + b);
            if (t == "-") st.push(a - b);
            if (t == "*") st.push(a * b);
            if (t == "/") st.push(a / b);
        } else st.push(stoi(t));
    }
    return st.top();
}
// O(n)

```

Step-by-step example — "simplify path" style (undo semantics)

Input: "/a/./b/../../c/" → tokens: a, ., b, .., .., c

Process with a stack of directory names:

```

a → push; . → ignore; b → push → [a,b]; .. → pop b → [a]; .. → pop a → [];
c → push → [c] → answer "/c" ✓

```

This "push on progress, pop on undo" is the **same shape** as parentheses and matching problems — recognize the pattern.

Common mistakes

1. **pop() on an empty stack** — always empty()-check before top()/pop().
2. **Forgetting pop() returns void** — read top() first, then pop().

3. **Monotonic stack comparison direction** — decide: decreasing (for next greater) vs increasing (for next smaller), and whether to use $<$ or $\leq$ for strictness.

4. **Not realizing stack values may need to be indices** (to compute distances).

5. **Using a stack where a queue (FIFO) is the right model** — BFS, level-order, etc.

6. **Recursion overflows** where an explicit stack loop wouldn't (deep nesting) — convert when depth can be huge.

7. **Operator operand order** in $-$ and $/$ (b = top is the *right* operand).

Practice questions

- **Easy 1:** Valid parentheses.

- **Easy 2:** Implement a **min stack** — push/pop/top/getMin() all $O(1)$ (hint: store pairs, or a second stack of minima).

- **Easy 3:** Evaluate a postfix expression.

- **Medium 1:** Next greater element (then the "daily temperatures" variant).

- **Medium 2: Largest rectangle in a histogram** (monotonic stack, $O(n)$).

- **Medium 3: Asteroid collision** — $[5, 10, -5] \rightarrow [5, 10]$ (stack of survivors; negative asteroids moving left, positive moving right).

- **Challenging 1: Trapping rain water** using a monotonic stack ($O(n)$ — the "water above each bar" interpretation).

- **Challenging 2:** Given a string with '(', ')', '*' where * can be (,), or empty, check if it can form valid parentheses (greedy with two counters — a famous trick).

Hints & solutions: file 30-solutions.md.

12 — Queues & Deques

What they are

- **Queue: FIFO** (First-In, First-Out). Like a ticket line: first in line is served first.
- **Deque** (double-ended queue): insert/remove at **both ends** — a queue that's also a stack. `std::deque` also gives $O(1)$ random access.

Queue: front $\rightarrow$ [a, b, c] $\leftarrow$ back `push_back(c)` `pop_front()` $\rightarrow$ a
Deque: `push_front` / `push_back` / `pop_front` / `pop_back` — all $O(1)$

```
queue<int> q;
q.push(1); q.push(2);
int f = q.front();    // 1 (peek, doesn't remove)
q.pop();              // removes front — returns void!
q.back(); q.empty(); q.size();
```

```
deque<int> dq;
dq.push_back(x); dq.push_front(x); dq.pop_back(); dq.pop_front();
dq[i];           //  $O(1)$  random access
```

Why they're useful

- **BFS** (topic 19) processes nodes in the order discovered — exactly FIFO. Level-order tree traversal is BFS with a queue.
- **Deque** is the engine of the $O(n)$ sliding-window maximum (topic 09), and models "recent items, evict oldest" (cache) use cases.
- Queues model *producers/consumers, task scheduling, request buffering, printing jobs* — real-world FIFO everywhere.

Implementing a queue with an array (circular buffer)

A naive array queue shifts elements on pop ($O(n)$). A **circular buffer** reuses freed slots:

size = 5, front = 0, rear = 0 (empty). Insert: `rear = (rear+1) % size`.
Remove: `front = (front+1) % size`. Full when `(rear+1) % size == front`.

```
struct MyQueue {
    vector<int> a; int head = 0, tail = 0, n;
    MyQueue(int cap) : a(cap), n(cap) {}
    bool empty() const { return head == tail; }
    bool full() const { return (tail + 1) % n == head; }    // sacrifice one slot
    void push(int x) {
        if (full()) return;           // or resize — production code resizes
        a[tail] = x;
        tail = (tail + 1) % n;
    }
    int pop() {
        int v = a[head];
        head = (head + 1) % n;
        return v;
    }
};
// All ops  $O(1)$ . This is how std::queue is built (on top of deque) in spirit.
```

Pattern 1 — Level-order traversal of a binary tree (BFS)

```

    / \
   2  3    → visit 1, then 2,3, then 4,5,6,7 (level by level)
  / \ / \
 4  5 6  7

```

```

vector<vector<int>> levelOrder(TreeNode* root) {
    if (!root) return {};
    queue<TreeNode*> q; q.push(root);
    vector<vector<int>> res;
    while (!q.empty()) {
        int levelSize = q.size();           // snapshot: all nodes currently in queue = one level
        vector<int> level;
        for (int i = 0; i < levelSize; ++i) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back(level);
    }
    return res;
}
// O(n) time, O(n) space (queue holds at most one level)

```

Step-by-step trace:

```

q = [1]           → level [1]; push 2,3 → q=[2,3]
snapshot size=2   → level [2,3]; push 4,5,6,7 → q=[4,5,6,7]
snapshot size=4   → level [4,5,6,7] → done. Result: [[1],[2,3],[4,5,6,7]] ✓

```

Pattern 2 — Deque as a monotonic structure (sliding window max)

Full treatment in topic 09; the essence: keep a deque whose **values are strictly decreasing**. Front = current window's max; new elements pop the back while $\leq$ them; expired indices pop the front. **O(n)** for all window maxima.

```

vector<int> maxInWindows(const vector<int>& a, int k) {
    deque<int> dq;           // indices, values decreasing
    vector<int> res;
    for (int i = 0; i < (int)a.size(); ++i) {
        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
        dq.push_back(i);
        if (dq.front() <= i - k) dq.pop_front();
        if (i >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}

```

Pattern 3 — Queue problems that aren't BFS

- **"Rot oranges / walls and gates / shortest path in grid"**: multi-source BFS — push all starting sources into the queue first, then BFS outward (all get correct distances in level order).
- **"First unique character in a stream"**: queue of candidates + frequency map; pop front while it's no longer unique.
- **Implement stack using queues / queue using stacks** — classic interview drills that force you to understand both structures' orderings.
- **Task scheduler / deque task processing** — model time as a queue of pending work.

Common mistakes

1. `pop()` **returns void** (both queue and stack) — read `front()/top()` first.
2. **Using `q.size()` inside a loop while pushing** — new pushes change `size()`; snapshot the level size first (see level-order above).
3. **Circular-buffer off-by-one** — the $(tail+1) \% n == head$ full-check is why one slot is wasted; get the modulo right.
4. **Modulo of negative numbers** — in C++, $-1 \% 5 == -1$; normalize with $(x \% n + n) \% n$.
5. **Choosing queue vs stack wrongly** — BFS=queue, DFS=stack (or recursion).
6. deque **iterator invalidation**: unlike vector, deque invalidates iterators on push at the front (and often on `push_back`); don't hold iterators across pushes.
7. **Priority queue $\neq$ queue** — `priority_queue` is a heap (topic 17); don't use it for FIFO.

Practice questions

- **Easy 1:** Implement a queue using **two stacks** (amortized $O(1)$ per op).
- **Easy 2:** Given a binary tree, print it level by level (then **zigzag**: reverse odd levels).
- **Easy 3:** First unique character in a **stream** (process chars one at a time; after each char, answer the first non-repeating char so far).
- **Medium 1:** Sliding window maximum with a deque ($O(n)$).
- **Medium 2: Number of recent calls** — given timestamps, count calls in the last 3000ms (queue; pop while too old).
- **Medium 3: Rotting oranges** — grid with fresh/rotten oranges, minutes until all rot (multi-source BFS), or -1 if impossible.
- **Challenging 1: Design a circular deque** supporting all four ops in $O(1)$.
- **Challenging 2:** Given a grid, **shortest path in a binary matrix** (8-directional BFS, distance = levels).
- **Challenging 3: Zero-one matrix** — distance to the nearest 0 for every cell (multi-source BFS from all zeros).

Hints & solutions: file 30-solutions.md.

13 — Recursion

What it is

Recursion is when a **function calls itself** to solve a smaller version of the same problem. Every recursive function needs:

1. **Base case(s)**: the smallest input, answered directly (no recursion).
2. **Recursive case**: reduce the problem to a smaller one, and **trust** the recursive call

to solve it — then combine.

```
int factorial(int n) {  
    if (n <= 1) return 1;           // base case  
    return n * factorial(n - 1);    // recursive case – trust it!  
}
```

Why it's useful

- The **cleanest** expression of many algorithms: tree traversals, divide & conquer (merge/quick sort), backtracking, and **dynamic programming** (top-down) are all naturally recursive.
- It teaches you to **think in terms of smaller subproblems** — the core DSA skill.
- Every recursive call runs on the **call stack**: the runtime remembers where to resume. That stack is a real cost (space) and a real limit (stack overflow on deep recursion).

How to write recursion — the "leap of faith"

1. **Define the signature** — what does $f(x)$ return, given input x ?
2. **Find the smallest cases** — write their answers directly (base case).
3. **Assume f works for all smaller inputs** — write the larger case in terms of them.
4. **Verify termination** — each call must move strictly toward a base case.

If steps 1-3 are consistent, the function is correct. You don't "trace" it; you prove it.

Pattern 1 — Linear recursion: Fibonacci

$F(0)=0$, $F(1)=1$, $F(n)=F(n-1)+F(n-2)$.

Naive recursion recomputes $F(2)$ many times — the recursion tree has 2^n nodes.

```
int fib(int n) {  
    if (n <= 1) return n;           // base  
    return fib(n - 1) + fib(n - 2); // TWO recursive calls → exponential  
}  
// T(n) = T(n-1) + T(n-2) + O(1) → O(2^n) time, O(n) stack space
```

The tree for $\text{fib}(5)$:

```
          fib(5)  
        fib(4)  fib(3)  
      fib(3) fib(2) fib(2) fib(1)  
    fib(2) fib(1) ...
```

Notice $\text{fib}(3)$ appears twice, $\text{fib}(2)$ three times — **overlapping subproblems**. That's the signal for **memoization / dynamic programming** (topic 22): store answers, reuse them, $O(n)$. Recursion + a cache = the simplest correct DP.

Pattern 2 — Divide & conquer: merge sort

Divide the problem in half, solve both halves recursively, combine the results.
 $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$. See topic 06 for full code.

```
void mergeSort(vector<int>& a, int l, int r) {
    if (l >= r) return;                // base: 0/1 element is sorted
    int m = l + (r - l) / 2;           // divide
    mergeSort(a, l, m);                // conquer left
    mergeSort(a, m + 1, r);            // conquer right
    merge(a, l, m, r);                 // combine (O(n))
}
```

Pattern 3 — Recursion on trees

Trees are *defined* recursively (a node = value + left subtree + right subtree), so recursion is the natural fit:

```
int maxDepth(TreeNode* root) {
    if (!root) return 0;                // base
    return 1 + max(maxDepth(root->left), maxDepth(root->right)); // trust!
}
```

Pattern 4 — Recursive linked-list reversal

```
ListNode* reverseList(ListNode* head) {
    if (!head || !head->next) return head; // base: 0/1 nodes
    ListNode* newHead = reverseList(head->next); // reverse the rest — trust it
    head->next->next = head; // the rest's tail now points at head
    head->next = nullptr; // head becomes the new tail
    return newHead; // return the OLD tail (new head)
}
// O(n) time, O(n) stack space — the iterative version (topic 10) is O(1) space.
```

Step-by-step for 1→2→3: the call for 2 returns 3→2; then we make 2→1 and 1→null: result 3→2→1→null. ✓

Tail recursion & iteration

If the recursive call is the *last* thing a function does (nothing to combine after), you can often rewrite as a loop — avoiding stack growth:

```
// tail-recursive sum → iterative:
long long sumIter(int n) { long long s = 0; for (int i = 1; i <= n; ++i) s += i; return s; }
```

Common mistakes

1. **Missing / unreachable base case** → infinite recursion → stack overflow (crash).
2. **No progress toward the base case** (e.g. $f(n)$ calling $f(n)$) — same crash.
3. **Tracing instead of trusting** — you can't hold 2^n frames in your head; write the signature, prove the base, trust the smaller call.
4. **Deep recursion overflow:** each frame costs ~dozens of bytes; 10^6 depth = crash. Convert to iteration/loop when depth is unbounded (or raise the stack size deliberately).
5. **Exponential time from overlapping subproblems** — memoize (topic 22).
6. **Side effects in recursion** (modifying shared state at the wrong time) — especially in backtracking (topic 14) where you must *undo* changes.
7. **Forgetting the combine step** (divide & conquer needs a merge/combine).
8. **Stack overflow in debug builds with big inputs** — recursion is $O(n)$ space; say so in your complexity analysis.

Practice questions

- **Easy 1:** Recursively compute the sum of digits of a number.
- **Easy 2:** Recursively reverse a string (in place with indices, or return new string).
- **Easy 3:** Count the number of ways to climb n stairs taking 1 or 2 steps at a time — write it recursively, then note the pattern (it's Fibonacci).
- **Medium 1:** Merge two sorted lists recursively (no dummy node).
- **Medium 2: Powerset** — all subsets of a set, using the recursion "include $a[i]$ or not" (also see topic 14 / 24).
- **Medium 3:** Recursively check if a binary tree is a mirror of itself.
- **Challenging 1: Tower of Hanoi** — move n disks in $2^n - 1$ moves; implement and count moves.
- **Challenging 2:** Given a string, print all **palindromic partitions** (split into palindrome substrings) — recursion + a palindrome check.
- **Challenging 3:** Recursive **exponentiation by squaring**: $\text{pow}(x, n)$ in $O(\log n)$ (n can be negative — handle carefully).

Hints & solutions: file 30-solutions.md.

14 — Backtracking

What it is

Backtracking is recursion with a "**choose - explore - unchoose**" loop: you build a solution incrementally, and when a partial solution can't lead to a valid one, you **undo the last choice** and try the next option.

Template:

```
void backtrack(state) {
    if (state is a complete valid solution) { record it; return; }
    for (each candidate choice) {
        if (choice is valid/pruned) {
            apply(choice);           // choose
            backtrack(newState);     // explore
            undo(choice);            // unchoose – CRITICAL
        }
    }
}
```

Why it's useful

It's the general tool for **enumerating all combinations/permutations/paths** when there's no shortcut: subsets, permutations, N-Queens, Sudoku, word search, regex matching, graph paths. Time is usually exponential — that's expected — but **pruning** (rejecting partial solutions early) is what makes many instances practical.

Pattern 1 — All subsets (powerset)

Problem: given distinct elements, generate all subsets.

Idea: at each index, decide include / exclude. 2^n subsets.

```
void genSubsets(vector<int>& a, int i, vector<int>& cur, vector<vector<int>>& res) {
    if (i == a.size()) { res.push_back(cur); return; } // complete – record
    // exclude a[i]
    genSubsets(a, i + 1, cur, res);
    // include a[i]
    cur.push_back(a[i]);
    genSubsets(a, i + 1, cur, res);
    cur.pop_back(); // unchoose!
}

vector<vector<int>> subsets(vector<int>& a) {
    vector<vector<int>> res; vector<int> cur;
    genSubsets(a, 0, cur, res);
    return res;
}
// 0(2^n) subsets × 0(n) copy each = 0(n·2^n) total
```

Pattern 2 — Permutations

Problem: all orderings of the elements. $n!$ permutations.

Idea: at each position, try every unused element. Use a used[] array (or swap-based).

```
void genPerm(vector<int>& a, int pos, vector<vector<int>>& res) {
    if (pos == a.size()) { res.push_back(a); return; } // a itself is the current perm
    for (int i = pos; i < (int)a.size(); ++i) {
        swap(a[pos], a[i]); // choose a[i] for position pos
        genPerm(a, pos + 1, res); // explore the rest
        swap(a[pos], a[i]); // unchoose
    }
}
```

```
vector<vector<int>> permute(vector<int>& a) {
    vector<vector<int>> res;
    genPerm(a, 0, res);
    return res;
}
// 0(n!) permutations
```

Step-by-step for [1,2,3] at pos=0: swap(0,0) → recurse [1,...]; swap(0,1) → recurse [2,...]; swap(0,2) → recurse [3,...]. Each level fixes one position. The swap back restores the array for the next iteration — the essence of unchoose.

Pattern 3 — N-Queens (the canonical pruning example)

Problem: place n queens on an $n \times n$ board so none attack each other (no same row/col/diagonal). Key pruning: rows are handled by recursion depth; track used columns, and the two diagonals ($r-c$ is constant on the "\ " diagonal, $r+c$ on the "/" diagonal). $0(n!)$ worst, tiny in practice.

```
void solve(int r, int n, vector<int>& col, vector<bool>& usedCol,
           vector<bool>& diag1, vector<bool>& diag2, vector<vector<string>>& res) {
    if (r == n) { // all queens placed
        vector<string> board(n, string(n, '.'));
        for (int c = 0; c < n; ++c) board[col[c]][c] = 'Q'; // col[c] = row of queen in col c
        res.push_back(board);
        return;
    }
    for (int c = 0; c < n; ++c) {
        if (usedCol[c] || diag1[r - c + n - 1] || diag2[r + c]) continue; // prune
        usedCol[c] = diag1[r - c + n - 1] = diag2[r + c] = true;
        col[c] = r;
        solve(r + 1, n, col, usedCol, diag1, diag2, res);
        usedCol[c] = diag1[r - c + n - 1] = diag2[r + c] = false; // unchoose
    }
}
```

Pattern 4 — Word search (grid DFS + backtracking)

Problem: does the word exist in a grid by moving up/down/left/right through matching chars? Choose = mark cell visited; explore = recurse on 4 neighbors; unchoose = unmark.

```
bool dfs(vector<vector<char>>& g, const string& w, int i, int r, int c) {
    if (i == w.size()) return true;
    if (r < 0 || r >= g.size() || c < 0 || c >= g[0].size()) return false;
    if (g[r][c] != w[i]) return false; // prune: char mismatch
    char save = g[r][c]; g[r][c] = '#'; // choose: mark visited
    bool ok = dfs(g, w, i + 1, r + 1, c) || dfs(g, w, i + 1, r - 1, c) ||
              dfs(g, w, i + 1, r, c + 1) || dfs(g, w, i + 1, r, c - 1);
    g[r][c] = save; // unchoose
    return ok;
}

bool exist(vector<vector<char>>& g, string w) {
    for (int r = 0; r < g.size(); ++r)
        for (int c = 0; c < g[0].size(); ++c)
            if (dfs(g, w, 0, r, c)) return true;
    return false;
}
// 0(n·m·4^L) worst case (L = word length), pruned hard in practice
```

Pruning — the difference between $O(2^n)$ that dies and $O(2^n)$ that runs

Always prune **as early as possible**:

- In combination-sum problems: sort candidates; **stop the loop** once the current candidate already exceeds the target (monotonic order) instead of recursing deeper.

- In N-Queens: check attacks via $O(1)$ arrays instead of scanning the board.
- In Sudoku: only try digits valid for the cell's row/col/box.
- In "partition equal subset" (backtracking on sorted input): skip duplicate candidates at the same recursion level (`if (i > start && a[i] == a[i-1]) continue;`).

Common mistakes

1. **Forgetting to unchoose** — state leaks across branches, producing garbage.
2. **Mutating shared state (global arrays) without restore** — the #1 backtracking bug.
3. **Not copying results** — pushing a reference to a `cur` vector that later mutates.
4. **Pruning too late** — exploring dead branches to full depth.
5. **Wrong loop start** for combinations (start at i , not 0) vs permutations (always try all) — using `start=i` for permutations skips valid ones.
6. **Duplicates from repeated elements** — sort + skip same-value at the same level.
7. **Infinite recursion** — a choice must strictly progress toward the base case; in grids, always mark visited before recursing (else you bounce between two cells).
8. **Stack depth** — backtracking depth equals the solution length (e.g. word length, board size); usually fine, but be aware.

Practice questions

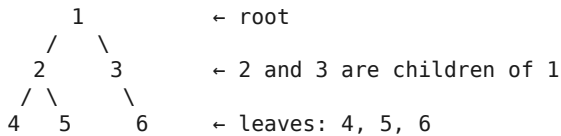
- **Easy 1:** Generate all subsets of $[1,2,3]$ (then all subsets with duplicates $[1,2,2]$ — handle duplicates).
- **Easy 2:** Generate all permutations of $[1,2,3]$ (then with duplicates — skip swaps that repeat a value at the same position).
- **Medium 1: Combination sum** — unique combos of candidates (can reuse each) that sum to target.
- **Medium 2: Letter combinations of a phone number** — "23" $\rightarrow$ ["ad","ae",...].
- **Medium 3: N-Queens** ($n \leq 9$) — return all distinct solutions.
- **Medium 4:** Word search in a grid.
- **Challenging 1: Sudoku solver** — fill the board (backtracking + box/row/col checks).
- **Challenging 2: Palindrome partitioning** — all ways to split a string into palindrome substrings.
- **Challenging 3: Partition to k equal sum subsets** (prune hard: sort descending, skip duplicates, early exit on impossible) — this one punishes bad pruning.

Hints & solutions: file 30-solutions.md.

15 — Trees (Binary Trees)

What it is

A tree is a **hierarchical** structure: nodes connected by edges, with a **root** at the top. Each node has children; nodes with no children are **leaves**. A **binary tree** has at most 2 children per node (left, right). No cycles, and every node except the root has exactly one parent.



Vocabulary: **height** = longest path from node to a leaf (leaf = 0). **depth** = distance from root (root = 0). **Level** = set of nodes at the same depth.

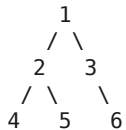
Why it's useful

Trees organize data by hierarchy and enable **O(log n)-ish** operations when balanced (BSTs, heaps, segment trees are all trees). They model file systems, HTML DOM, org charts, game states (minimax), and more. Tree traversals are the gateway to graph algorithms.

Node definition & building

```
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
};
```

Traversals — the core skill



Traversal	Order	Output above
Preorder (node, left, right)	root first	1, 2, 4, 5, 3, 6
Inorder (left, node, right)	sorted for BSTs	4, 2, 5, 1, 3, 6
Postorder (left, right, node)	children first	4, 5, 2, 6, 3, 1
Level-order (BFS)	top to bottom, left to right	1, 2, 3, 4, 5, 6

```
void inorder(TreeNode* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->val << " ";
    inorder(root->right);
}
// preorder: visit before recursing; postorder: visit after both recursions.
// All three are O(n) time, O(height) space (recursion stack).
```

Level-order uses a queue (topic 12):

```
vector<vector<int>> levelOrder(TreeNode* root) {
    if (!root) return {};
```

```

queue<TreeNode*> q; q.push(root);
vector<vector<int>>> res;
while (!q.empty()) {
    int sz = q.size(); // nodes currently in queue = one full level
    vector<int> level;
    for (int i = 0; i < sz; ++i) {
        TreeNode* node = q.front(); q.pop();
        level.push_back(node->val);
        if (node->left) q.push(node->left);
        if (node->right) q.push(node->right);
    }
    res.push_back(level);
}
return res;
}

```

Iterative DFS (explicit stack — handy when recursion depth is a concern):

```

vector<int> preorderIter(TreeNode* root) {
    vector<int> res;
    stack<TreeNode*> st; if (root) st.push(root);
    while (!st.empty()) {
        TreeNode* n = st.top(); st.pop();
        res.push_back(n->val);
        if (n->right) st.push(n->right); // push right first so LEFT is processed first
        if (n->left) st.push(n->left);
    }
    return res;
}

```

Pattern 1 — Height / depth

```

int maxDepth(TreeNode* root) {
    if (!root) return 0;
    return 1 + max(maxDepth(root->left), maxDepth(root->right));
}
// O(n) time, O(height) space

```

Pattern 2 — Diameter (longest path between any two nodes)

The longest path passes "through" some node and uses its left height + right height.

```

int diameterOfBinaryTree(TreeNode* root) {
    int best = 0;
    function<int(TreeNode*)> dfs = [&](TreeNode* n) -> int {
        if (!n) return 0;
        int L = dfs(n->left), R = dfs(n->right);
        best = max(best, L + R); // path through this node
        return 1 + max(L, R); // return this subtree's height
    };
    dfs(root);
    return best;
}
// O(n)

```

Pattern 3 — LCA (lowest common ancestor)

LCA of nodes p and q = the deepest node that has both p and q in its subtree.
 Recursive rule: if root is p or q → root is the answer for this subtree.
 Otherwise search left and right; if both return non-null → root is the LCA;
 if only one returns non-null → that one is the LCA.

```

TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (!root || root == p || root == q) return root;
    TreeNode* L = lowestCommonAncestor(root->left, p, q);
    TreeNode* R = lowestCommonAncestor(root->right, p, q);
}

```

```

    if (L && R) return root;           // p and q on different sides → root is LCA
    return L ? L : R;                 // both in one subtree (or neither)
}
// O(n) time, O(height) space. (For many queries, precompute depths + binary lifting –
// see topic 28.)

```

Pattern 4 — Same tree / subtree / symmetry (mirror-recursion)

```

bool isSameTree(TreeNode* p, TreeNode* q) {
    if (!p || !q) return p == q;      // both null → true; one null → false
    return p->val == q->val &&
           isSameTree(p->left, q->left) &&
           isSameTree(p->right, q->right);
}
// Symmetric tree: isSameTree(root->left, root->right) with LEFT-vs-RIGHT mirrored.

```

Pattern 5 — Serialize / reconstruct

- **Serialize:** any traversal encoding (e.g. preorder with nulls marked) can rebuild the tree.

- **Build from preorder + inorder:** the preorder's first element is the root; find it in inorder to split left/right subtrees (use a hash map for O(1) index lookup → O(n) total).

Common mistakes

1. **Forgetting the if (!root) base case** — null dereference.
2. **Confusing height (leaf = 0) with depth (root = 0)** — off-by-ones in balance checks.
3. **Unbalanced recursion on skewed trees** — a "tree" that's really a list → O(n) depth → stack overflow. Know your input constraints.
4. **root->left used without checking root** — order the null check first.
5. **Passing by value where a reference is needed** for accumulating state.
6. **Level-order without snapshotting size** — see topic 12.
7. **Treating BST problems as general-tree problems** (topic 16 shows the shortcuts).

Practice questions

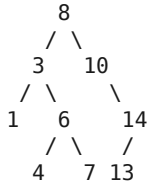
- **Easy 1:** Maximum depth of a binary tree.
- **Easy 2:** Given two trees, check if they're the **same tree**; then if one is a **subtree** of the other.
- **Easy 3:** Binary tree **inorder traversal** (recursive AND iterative).
- **Medium 1:** Diameter of a binary tree.
- **Medium 2:** Binary tree **level-order traversal** (then zigzag / reverse levels).
- **Medium 3:** Lowest common ancestor of two nodes.
- **Medium 4:** Check if a tree is **balanced** (height of left/right differ ≤ 1 at every node) in O(n).
- **Challenging 1:** Build a binary tree from **preorder + inorder** traversals.
- **Challenging 2:** **Serialize and deserialize** a binary tree.
- **Challenging 3:** **Maximum path sum** — any path (not necessarily through root) that maximizes node-sum (like diameter but with values and negatives).

Hints & solutions: file 30-solutions.md.

16 — Binary Search Trees (BST)

What it is

A BST is a binary tree with the **BST property**: for every node, everything in the **left subtree is smaller**, everything in the **right subtree is larger** (no duplicates, usually).



Why it's useful

The property makes **search/insert/delete $O(\text{height})$** — $O(\log n)$ on a balanced tree. That's the engine inside `std::map/std::set` (red-black trees). And **inorder traversal of a BST visits values in sorted order** — a free sort and the key to many problems.

Search

```
bool searchBST(TreeNode* root, int key) {
    if (!root) return false;
    if (root->val == key) return true;
    return key < root->val ? searchBST(root->left, key)
        : searchBST(root->right, key);
}
// O(height): each step discards an entire subtree.
```

Insert

```
TreeNode* insertBST(TreeNode* root, int key) {
    if (!root) return new TreeNode(key); // found the spot
    if (key < root->val) root->left = insertBST(root->left, key);
    else root->right = insertBST(root->right, key);
    return root; // return root so callers keep the tree
}
// O(height). NOTE: must assign the result – root can change.
```

Delete (the tricky one — 3 cases)

Case 1: leaf → just remove it (return null).
Case 2: one child → replace node with that child.
Case 3: two children → replace value with its **INORDER SUCCESSOR** (smallest in right subtree), then delete that successor from the right subtree (it has ≤ 1 child).

```
TreeNode* minNode(TreeNode* root) { // leftmost node = minimum
    while (root->left) root = root->left;
    return root;
}

TreeNode* deleteBST(TreeNode* root, int key) {
    if (!root) return nullptr;
    if (key < root->val) root->left = deleteBST(root->left, key);
    else if (key > root->val) root->right = deleteBST(root->right, key);
    else { // found the node to delete
        if (!root->left) return root->right; // cases 1 & 2
        if (!root->right) return root->left; // cases 1 & 2
        TreeNode* succ = minNode(root->right); // case 3
        root->val = succ->val; // copy successor's value
        root->right = deleteBST(root->right, succ->val); // delete the successor
    }
}
```

```

    }
    return root;
}
// O(height). Always reassign the returned root at every level.

```

Validate a BST (the classic gotcha)

WRONG: check only `root->val` vs left/right child values. Consider 5 with right child 6, whose left child is 3 — every node violates locally but the tree is NOT a valid BST.
 RIGHT: pass down a (min, max) range; each node must lie within it.

```

bool isValidBST(TreeNode* root, long long lo = LLONG_MIN, long long hi = LLONG_MAX) {
    if (!root) return true;
    if (root->val <= lo || root->val >= hi) return false;
    return isValidBST(root->left, lo, root->val) &&
           isValidBST(root->right, root->val, hi);
}
// O(n)

```

Step-by-step on the WRONG example: root 5: left null, right 6 → range (5, ∞) for the right subtree.
 Node 6: its left child 3 must satisfy (5, 6) — 3 fails → false. ✓ caught.

Pattern — k-th smallest / k-th largest (inorder + early exit)

```

int kthSmallest(TreeNode* root, int k) {
    int count = 0, ans = 0;
    function<void(TreeNode*)> go = [&](TreeNode* n) {
        if (!n || count >= k) return;
        go(n->left);                // inorder: ascending
        if (++count == k) { ans = n->val; return; }
        go(n->right);
    };
    go(root);
    return ans;
}
// O(k + height). Alternative: iterative inorder with a stack — same idea.

```

Pattern — floor / ceil, and "closest value"

```

TreeNode* floorBST(TreeNode* root, int key) {    // largest value ≤ key
    TreeNode* res = nullptr;
    while (root) {
        if (root->val == key) return root;
        if (root->val < key) { res = root; root = root->right; } // candidate, go bigger
        else root = root->left;                                // too big, go smaller
    }
    return res;
}
// O(height), no recursion needed — the tree guides you.

```

Balanced vs unbalanced — and self-balancing BSTs

Tree	Height (worst)	Where used
Naive BST (insertion order)	O(n) (e.g. inserting 1,2,3,...)	educational
AVL / Red-black	O(log n) guaranteed	<code>std::map</code> , <code>std::set</code>
Splay / Treap	O(log n) amortized	advanced

Naive BSTs degrade to linked lists on sorted input — that's why `map/set` self-balance. For interviews, you almost never implement AVL/RB from scratch; you use `std::set/map` or (for range problems) a **segment tree** (topic 27).

Common mistakes

1. **Not reassigning the root** after insert/delete (`root = insertBST(root, key);`).
2. **Local-only validation** instead of the (min, max) range check.
3. **Duplicate handling unclear** — decide $\leq$ vs $<$ and stay consistent.
4. **Deleting a two-child node by removing the node itself** — must replace with successor (or predecessor) first.
5. **Assuming the tree is balanced** — worst-case height $O(n)$ for naive insertion order.
6. **Recursion depth on skewed trees** $\rightarrow$ stack overflow; use iterative where depth is risky.
7. **Overflow in range checks** — use `long long` bounds (values can be `INT_MIN/INT_MAX`).

Practice questions

- **Easy 1:** Search in a BST (iterative — while loop, no recursion).
- **Easy 2:** Insert into a BST.
- **Easy 3:** Find the **minimum** and **maximum** value in a BST.
- **Medium 1:** Validate a BST (range method) — then also solve with inorder (should be strictly increasing).
- **Medium 2:** Delete a node from a BST (all three cases).
- **Medium 3:** K-th smallest element in a BST (then k-th largest).
- **Medium 4:** Convert a **sorted array** into a **height-balanced** BST (middle = root, recurse).
- **Challenging 1: Lowest common ancestor in a BST** — $O(\text{height})$, using value comparisons (both $<$ node $\rightarrow$ left; both $>$ node $\rightarrow$ right; else node is the LCA).
- **Challenging 2:** Given a BST, convert it to a **sorted doubly linked list** in place (inorder + prev pointer wiring).
- **Challenging 3:** Recover a BST whose two nodes were swapped ($O(n)$ — find the two inverted positions in inorder).

Hints & solutions: file 30-solutions.md.

17 – Heaps & Priority Queues

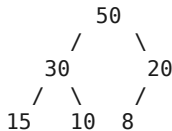
What it is

A **heap** is a complete binary tree (all levels full except possibly the last) with the **heap property**: every parent is $\geq$ its children (**max-heap**) or $\leq$ its children (**min-heap**). The top is always the max/min — that's the whole point. Stored in an **array**:

```

Index:    0    1    2    3    4    5
Heap:    [50, 30, 20, 15, 10, 8]
parent(i) = (i-1)/2
left(i) = 2i+1, right(i) = 2i+2

```



Why it's useful

- **O(1)** peek at max/min, **O(log n)** push and pop. That beats a sorted array (O(n) insert) and an unsorted array (O(n) max).
- Powers the "repeatedly need the extreme" family: top-K, k-way merge, Dijkstra, Huffman, task scheduling, median of a stream.
- `std::priority_queue` is the max-heap; `std::make_heap` exposes the raw heap.

How a heap works (two operations)

Sift-up (on push): append the element; while it's greater (max-heap) than its parent, swap upward.

```
push 40 into [50,30,20,15,10,8]:  
  append → [50,30,20,15,10,8,40]; 40's parent is 20 → 40>20 → swap → [50,30,40,15,10,8,20]  
  40's parent now 50 → stop. ✓
```

Sift-down (on pop): move the last element to the root; while it's smaller than a child, swap with the larger child.

pop from [50,30,40,15,10,8,20]:
 root 20 → [20,30,40,15,10,8]; children 30,40 → 20<40 → swap with 40 → [40,30,20,15,10,8]
 20's children 15,10 → 20>15 → done ✓ root is 40 – the new max.

```

struct MaxHeap {
    vector<int> a;
    void push(int x) {
        a.push_back(x);
        int i = a.size() - 1;
        while (i > 0 && a[(i - 1) / 2] < a[i]) {
            swap(a[(i - 1) / 2], a[i]);
            i = (i - 1) / 2;
        }
    }
    int pop() {
        int top = a[0];
        a[0] = a.back(); a.pop_back();
        int i = 0, n = a.size();
        while (true) {
            int l = 2 * i + 1, r = 2 * i + 2, best = i;
            if (l < n && a[l] > a[best]) best = l;
            if (r < n && a[r] > a[best]) best = r;
            if (best == i) break;
            swap(a[i], a[best]); i = best;
        }
    }
};

```

```

    }
    return top;
}
};
// push/pop  $O(\log n)$ 

```

Heapify — building a heap from an array is $O(n)$, not $O(n \log n)$: sift-down from the last non-leaf up to the root. Most elements sit near the bottom and barely move.

std::priority_queue — the practical interface

```

priority_queue<int> pq; // MAX-heap: top() is the largest
pq.push(5); pq.push(1); pq.push(9); // top() = 9
pq.pop(); // removes 9

priority_queue<int, vector<int>, greater<int>> minpq; // MIN-heap

// Custom comparator — NOTE the reversed semantics (this is the #1 bug):
struct Cmp {
    bool operator()(pair<int,int> a, pair<int,int> b) const {
        return a.second > b.second; // '>' → smallest .second on top (min-heap)
    }
};
priority_queue<pair<int,int>, vector<pair<int,int>>, Cmp> pq;
// Think of it as: "which element is WORSE / should be popped LAST?" Returning true for a
// means a sits BELOW b.

```

Pattern 1 — Top-K elements (min-heap of size k)

Problem: k largest elements of n (n huge). Sort = $O(n \log n)$. Heap = $O(n \log k)$.
 Idea: keep a MIN-heap of size k. Push each element; if size > k, pop the smallest — the heap now holds the k largest seen so far. Top = k-th largest.

```

vector<int> topK(const vector<int>& a, int k) {
    priority_queue<int, vector<int>, greater<int>> pq;
    for (int x : a) {
        pq.push(x);
        if ((int)pq.size() > k) pq.pop();
    }
    vector<int> res;
    while (!pq.empty()) { res.push_back(pq.top()); pq.pop(); }
    return res;
}
//  $O(n \log k)$  time,  $O(k)$  space

```

Pattern 2 — Merge k sorted lists/arrays

Push each list's head into a min-heap (value, listId, index). Pop the min → it's next in the merged output; push that list's next element. Repeat n times.

```

struct Item { int val, listIdx, elemIdx; };
struct Cmp { bool operator()(const Item& x, const Item& y) const { return x.val > y.val; } };

vector<int> mergeKLists(vector<vector<int>>& lists) {
    priority_queue<Item, vector<Item>, Cmp> pq;
    for (int i = 0; i < (int)lists.size(); ++i)
        if (!lists[i].empty()) pq.push({lists[i][0], i, 0});
    vector<int> res;
    while (!pq.empty()) {
        auto cur = pq.top(); pq.pop();
        res.push_back(cur.val);
        if (cur.elemIdx + 1 < (int)lists[cur.listIdx].size())
            pq.push({lists[cur.listIdx][cur.elemIdx + 1], cur.listIdx, cur.elemIdx + 1});
    }
    return res;
}

```

```

}
// O(n log k): n total elements, heap of size k

```

Pattern 3 — Median of a stream (two heaps)

Keep a MAX-heap of the smaller half and a MIN-heap of the larger half, balanced so $|small| == |large|$ or $|small| == |large| + 1$. Median = top of small (odd count) or average of the two tops (even count). Each add: push to small, then rebalance.

```

class MedianFinder {
    priority_queue<int> small;           // max-heap, lower half
    priority_queue<int, vector<int>, greater<int>> large; // min-heap, upper half
public:
    void addNum(int x) {
        small.push(x);
        large.push(small.top()); small.pop(); // rebalance sizes
        if (small.size() < large.size()) { // keep small ≥ large
            small.push(large.top()); large.pop();
        }
    }
    double findMedian() {
        if (small.size() > large.size()) return small.top();
        return (small.top() + large.top()) / 2.0;
    }
};
// addNum O(log n), findMedian O(1), space O(n)

```

Heap sort (bonus)

Build a max-heap ($O(n)$), then repeatedly swap root with the last element and sift-down — $O(n \log n)$ total, $O(1)$ extra space, **not stable**. `std::sort_heap` does this for you.

Common mistakes

1. **Comparator direction inverted** — for `priority_queue`, `>` gives a min-heap. Triple-check when using custom comparators. (This is the single most common heap bug.)
2. **Using a max-heap for top-K** — top-K smallest needs a **max-heap of size k*, top-K largest needs a **min-heap*. Match the heap to the direction.
3. `top()` **on an empty heap** — always check `empty()` first (in `findMedian`, etc.).
4. **Assuming heap = sorted array** — the heap property only guarantees the top; you can't iterate it in sorted order. Pop repeatedly to get sorted order (that's heapsort).
5. **Building a heap by n pushes ($O(n \log n)$)** when `std::make_heap` / constructor with iterators is $O(n)$ — heapify.
6. **Storing stale indices in heap items** (merge-K problem) — re-read the source on pop.
7. **Dijkstra with a heap but re-pushing** — keep a `dist` array and skip stale entries.

Practice questions

- **Easy 1:** k-th largest element in a stream (a class: `add()` returns k-th largest so far).
- **Easy 2:** Given an unsorted array, find the k largest (or k smallest) elements.
- **Medium 1: Top K frequent elements** (freq map + min-heap of size k).
- **Medium 2: Merge k sorted lists** (linked-list version).
- **Medium 3: K closest points to origin** (max-heap of size k).
- **Challenging 1:** Median of a stream (two heaps).

- **Challenging 2: Find k most frequent words** with tie-breaking (heap with a careful comparator).

- **Challenging 3: Task scheduler** — given tasks and cooldown n, minimum intervals to finish (hint: max-heap + queue for cooldown, or pure math with the max frequency).

- **Challenging 4: Sliding window median** — median of every window of size k (two heaps + lazy deletion — a hard classic).

Hints & solutions: file 30-solutions.md.

18 — Greedy Algorithms

What it is

A greedy algorithm makes the **locally optimal choice at each step**, hoping it leads to a globally optimal solution. No backtracking, no reconsidering — commit and move on.

Classic: give change with fewest coins. At each step, take the LARGEST coin that fits.
73¢ with {25,10,5,1}: 25,25,10,10,1,1,1 → 7 coins. Optimal ✓ (for these denominations)

Why it's useful

Greedy solutions are usually **simple and fast** (often $O(n \log n)$ due to a sort, then $O(n)$). When a problem *has* the greedy property, the greedy solution beats every DP. The skill is recognizing when greedy is correct — and when it isn't (a greedy-looking solution that's wrong is a classic trap).

When is greedy correct?

Two common proof techniques:

1. **Exchange argument:** take any optimal solution; if it differs from the greedy choice, swap in the greedy choice — show the solution stays optimal. If every optimal solution can be "exchanged" to include the greedy choice, greedy is optimal.

2. **Matroid / structure argument:** if the problem has the *matroid* structure (roughly: independent sets closed under subsets + an exchange property — e.g. spanning trees, linearly independent vectors), greedy is optimal.

In practice: greedy works when **the choice you make never needs to be undone** because making it doesn't close off any better future. That's exactly why the *0/1 knapsack* is **not** greedy-solvable (taking a "good" item can prevent taking a better combination) but *fractional knapsack* is (you can always top up with the best value-per-weight).

Pattern 1 — Interval scheduling (maximum non-overlapping intervals)

Problem: given start/end times, schedule the maximum number of non-overlapping activities.
Greedy choice: pick the activity that ENDS EARLIEST among the remaining compatible ones.
Why: ending earliest leaves the most room for the rest — exchange argument proves it.

```
int maxActivities(vector<pair<int,int>>& act) {           // (start, end)
    sort(act.begin(), act.end(), [](auto& a, auto& b) { return a.second < b.second; });
    int count = 1, lastEnd = act[0].second;           // pick first (earliest end)
    for (int i = 1; i < (int)act.size(); ++i)
        if (act[i].first >= lastEnd) {                // compatible (no overlap)
            count++;
            lastEnd = act[i].second;
        }
    return count;
}
```

// $O(n \log n)$. THE rule: sort by END time, not start time.

Step-by-step: (1,3) (2,4) (3,6) (5,7) (6,8)

sort by end → (1,3)(2,4)(3,6)(5,7)(6,8)
pick (1,3); (2,4) starts 2 < 3 → skip; (3,6) starts 3 ≥ 3 → pick; (5,7) starts 5 < 6 → skip;
(6,8) starts 6 ≥ 6 → pick → 3 activities ✓ (can't do better than 3)

Pattern 2 — Fractional knapsack / value-density

Problem: items with weight & value, capacity W. Take fractions – maximize total value.
 Greedy: sort by value/weight descending; take as much of the best item as fits.

```
double fractionalKnapsack(vector<array<int,2>>& items, int W) {    // {weight, value}
    sort(items.begin(), items.end(), [](auto& a, auto& b) {
        return (double)a[1]/a[0] > (double)b[1]/b[0];    // by value density, desc
    });
    double total = 0;
    for (auto& [w, v] : items) {
        if (W >= w) { total += v; W -= w; }                // take whole item
        else        { total += v * (double)W / w; break; } // take fraction
    }
    return total;
}
// O(n log n). Contrast: 0/1 knapsack (can't take fractions) needs DP – topic 22.
```

Pattern 3 — Huffman coding (optimal prefix-free codes)

Problem: compress a string; variable-length codes, no code is a prefix of another.
 Greedy: repeatedly merge the TWO least-frequent characters into a tree node whose frequency is the sum. The merge order IS the optimal tree (exchange argument).

```
struct Node { char ch; int freq; Node *l, *r; };
struct Cmp { bool operator()(Node* a, Node* b) const { return a->freq > b->freq; } };

Node* huffmanTree(unordered_map<char,int>& freq) {
    priority_queue<Node*, vector<Node*>, Cmp> pq;    // min-heap by frequency
    for (auto& [ch, f] : freq) pq.push(new Node{ch, f, nullptr, nullptr});
    while (pq.size() > 1) {
        Node* a = pq.top(); pq.pop();
        Node* b = pq.top(); pq.pop();
        pq.push(new Node{'\0', a->freq + b->freq, a, b});    // internal node
    }
    return pq.top();    // root of the code tree
}
// O(n log n). Path to each leaf = that character's code; more frequent chars end up
// shallower (shorter codes) – that's the compression.
```

Step-by-step: frequencies a:5, b:9, c:12, d:13, e:16

merge a,b → 14 ; merge c,d → 25 ; merge 14, e(16) → 30 ; merge 30, 25 → root 55
 Codes (0=left,1=right): a:000, b:001, e:01, c:10, d:11 – "a" is rarest → longest code ✓

Pattern 4 — Jump game / candy / gas station (O(n) greedy scans)

The "can you reach the end" family:

jump game: $\text{maxReach} = \max(\text{maxReach}, i + a[i])$; if $i > \text{maxReach} \rightarrow$ stuck $\rightarrow$ false.
 candy: pass left→right enforcing $a[i] > a[i-1]$, then right→left enforcing the other side, take element-wise max.
 gas station: if total gas < total cost $\rightarrow$ impossible; else the station after the running minimum is the answer.

These have no sort — pure one-pass greedy with a running invariant.

Greedy fails here — know these traps

Problem	Why greedy fails	Correct tool
0/1 knapsack	taking the "best" item blocks better combos	DP
Coin change (non-canonical: {1,3,4}, amount 6)	greedy gives 4+1+1=3 coins; optimal is 3+3=2	DP
Longest path in a graph	local best doesn't compose globally	NP-hard (DAG: DP)

Problem	Why greedy fails	Correct tool
Minimum coin problem in general	see above	DP
Activity selection with *weights	most activities $\neq$ most value	DP on intervals
Traveling salesman	nearest neighbor $\neq$ shortest tour	NP-hard
Partitioning into equal sums	greedy fills buckets badly	DP / backtracking

The #1 skill: before committing to greedy, look for a counterexample. If you can't prove correctness with an exchange argument, seriously consider DP.

Common mistakes

1. **Applying greedy where the problem needs DP** — always ask "could a locally-best choice ever backfire?" (knapsack, coin change with weird denominations).

2. **Sorting by the wrong key** — intervals: end time; value-density: ratio; Huffman: frequency. Sorting by start for scheduling is the classic wrong answer.

3. **Ties handling** in custom comparators (strict weak ordering!).

4. **Forgetting to sort at all** — greedy without order isn't greedy, it's guessing.

5. **Overflow** in comparisons like v/w — compare $v1*w2 > v2*w1$ with long long instead of double division.

6. **Interval problems: $\geq$ vs $>$ for touching intervals** — decide whether $[1,2]$ and $[2,3]$ conflict (usually they don't: $\geq$).

7. **Not proving correctness** — greedy solutions are either beautifully right or subtly wrong. Test with brute force on small random inputs when unsure.

Practice questions

- **Easy 1:** Given an array, jump game — can you reach the last index (greedy maxReach)?
- **Easy 2: Assign cookies** — greediness, s and g arrays (children greed factor vs cookie sizes).
- **Medium 1:** Maximum non-overlapping intervals (or minimum intervals to remove to make the rest non-overlapping).
- **Medium 2: Candy** — distribute candies with ratings (two passes).
- **Medium 3: Gas station** — complete the circuit (running-sum trick).
- **Medium 4: Non-overlapping intervals $\rightarrow$ minimum arrow shots to burst balloons** (same sort-by-end + count overlaps).
- **Challenging 1: Huffman coding** — implement encoding, prove optimality sketch.
- **Challenging 2: Task scheduler** — min intervals with cooldown (greedy by max frequency + math, or heap + queue).
- **Challenging 3: Queue reconstruction by height** — $[[7,0],[4,4],[7,1],\dots]$ — insert tallest first (greedy by height desc, then position).

Hints & solutions: file 30-solutions.md.

19 — Graphs: Representations, BFS, DFS

What it is

A graph is a set of **vertices** (nodes) connected by **edges**. Real-world models: social networks (people→friendships), maps (cities→roads), the web (pages→links), dependencies (tasks→prerequisites).

Graph:

```
  1 -- 2 -- 3
    \  /    |
      4     5
```

Term	Meaning
Undirected vs directed	edge is two-way vs one-way
Weighted vs unweighted	edges carry costs or not
Cycle	a path that returns to its start
Connected component	maximal set of mutually reachable nodes
Tree	connected, acyclic graph (n nodes, n-1 edges)
Degree	number of edges at a vertex

Representations

```
// 1. Adjacency LIST – the default. O(V+E) space; iterate neighbors fast.
vector<vector<int>> adj(n);           // adj[u] = all neighbors of u
adj[u].push_back(v);
if (undirected) adj[v].push_back(u);

// 2. Adjacency MATRIX – O(V^2) space; O(1) edge check; good for tiny/dense graphs.
vector<vector<bool>> mat(n, vector<bool>(n, false));
mat[u][v] = true;

// 3. Edge list – vector of (u, v, w); used by Kruskal's MST.
struct Edge { int u, v, w; };
```

For **weighted** graphs: `vector<vector<pair<int,int>>> adj(n); → adj[u].push_back({v, w});`

DFS (depth-first) — go deep before going wide

Recursive DFS:
visit(u) → for each neighbor v not visited → visit(v)

```
vector<bool> vis(n, false);
```

```
void dfs(int u) {
    vis[u] = true;
    for (int v : adj[u])
        if (!vis[v]) dfs(v);
}
```

// O(V + E) — each vertex visited once, each edge scanned once

Iterative DFS (explicit stack — same order-ish, no recursion-depth risk):

```
stack<int> st; st.push(0); vis[0] = true;
while (!st.empty()) {
    int u = st.top(); st.pop();
    for (int v : adj[u]) if (!vis[v]) { vis[v] = true; st.push(v); }
}
```

BFS (breadth-first) — explore in layers, shortest paths in unweighted graphs

BFS visits all nodes at distance d before distance $d+1$ → the FIRST time you reach a node is along a SHORTEST path (in unweighted graphs).

```
vector<int> bfsDist(int src, int n) {
    vector<int> dist(n, -1);
    queue<int> q;
    dist[src] = 0; q.push(src);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (dist[v] == -1) {          // unvisited
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
    return dist;
}
// O(V + E)
```

Step-by-step BFS on 1-2-3-4-5 from 1:

$q=[1]$ $d1=0$ → pop 1, neighbors 2 → $q=[2]$ $d2=1$ → pop 2, neighbors 1,3 → 3 unvisited → $q=[3]$ $d3=2$ → pop 3, neighbor 4 → $d4=3$ → pop 4, neighbor 5 → $d5=4$ → pop 5 → done.
distances [0,1,2,3,4] — shortest paths ✓

The 6 problems DFS/BFS solve

1. Connected components

```
int components = 0;
for (int u = 0; u < n; ++u)
    if (!vis[u]) { components++; dfs(u); }    // each unvisited start = new component
```

2. Cycle detection

- **Undirected:** during DFS, if we reach an already-visited node that **isn't our parent**, there's a cycle.

- **Directed:** a back-edge to a node **currently on the recursion stack** (three-color: white=unvisited, gray=in-stack, black=done).

3. Bipartite check

A graph is bipartite iff it can be 2-colored with no edge joining same colors — BFS/DFS alternate colors; conflict → not bipartite. (Bipartite ⇔ no odd-length cycle.)

```
bool isBipartite(int n, vector<vector<int>>& adj) {
    vector<int> color(n, -1);
    for (int s = 0; s < n; ++s) if (color[s] == -1) {
        queue<int> q; color[s] = 0; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) {
                if (color[v] == -1) { color[v] = 1 - color[u]; q.push(v); }
                else if (color[v] == color[u]) return false;
            }
        }
    }
    return true;
}
```

4. Topological sort (DAG) — see topic 21

5. Shortest path in unweighted graphs — BFS (above)

6. Grid BFS/DFS (flood fill, islands)

Treat each cell as a vertex, 4 neighbors as edges:

```
int islands(vector<vector<char>>& g) {
    int R = g.size(), C = g[0].size(), ans = 0;
    int dr[] = {1,-1,0,0}, dc[] = {0,0,1,-1};
    function<void(int,int)> dfs = [&](int r, int c) {
        if (r < 0 || r >= R || c < 0 || c >= C || g[r][c] != '1') return;
        g[r][c] = '0'; // mark visited in place
        for (int k = 0; k < 4; ++k) dfs(r + dr[k], c + dc[k]);
    };
    for (int r = 0; r < R; ++r)
        for (int c = 0; c < C; ++c)
            if (g[r][c] == '1') { ans++; dfs(r, c); }
    return ans;
}
// O(R·C)
```

Step-by-step (flood fill): cell (1,1) is '1' → DFS swallows its whole island, marking each cell '0'. The outer loop then skips already-eaten land. Count of DFS starts = islands.

Choosing DFS vs BFS

Need	Choose
Shortest path (unweighted) / level-order / min steps	BFS
Existence / reachability / connectivity	Either (DFS simpler)
Topological order, SCC, bridges, tree DP	DFS
All paths / exhaustive search	DFS (+backtracking)
Deep graphs, stack risk	BFS (or iterative DFS)

Common mistakes

1. **Forgetting to mark visited BEFORE pushing/recursing** — infinite loops in cycles (BFS doubles work even with marking later; DFS may never terminate).
2. **Directed graph, undirected assumptions** — add both edges only for undirected graphs.
3. **Indexing: 0-based vs 1-based** — resize arrays to n+1 when input is 1-based.
4. **Recursion depth** on a long path (10^5+) — use iterative DFS or BFS.
5. **BFS distance = levels only if all edges cost 1** — weighted graphs need Dijkstra.
6. **Forgetting multi-source problems** (rotting oranges, nearest zero) — push ALL sources into the queue first.
7. **In grid DFS, mixing up rows/cols** or missing bounds checks.
8. **Not resetting vis between test cases** in multi-test inputs.

Practice questions

- **Easy 1:** Number of islands (grid DFS/BFS).
- **Easy 2:** Find if there's a path between two nodes (BFS or DFS).

- **Easy 3:** Depth of a graph / **maximum depth of n-ary tree**.
- **Medium 1: Rotting oranges** — minutes until all fresh rot (multi-source BFS).
- **Medium 2: Clone graph** — deep copy (BFS/DFS + map original→clone).
- **Medium 3: Course schedule** — can you finish all courses given prerequisites (directed cycle detection; also see topic 21).
- **Medium 4:** Shortest path in a binary matrix (BFS on grid).
- **Challenging 1: Word ladder** — min transformations from start to end word where each step changes one letter (BFS over word space).
- **Challenging 2: Pacific Atlantic water flow** — from which cells can water reach both oceans (two BFS/DFS from the edges, reversed direction).
- **Challenging 3: Accounts merge / number of provinces** — union-find style connectivity (also topic 25).

Hints & solutions: file 30-solutions.md.

20 — Shortest Paths (Dijkstra, Bellman-Ford, Floyd-Warshall, 0-1 BFS)

What it is

Given a **weighted** graph, find the minimum-cost path between nodes. Four classic algorithms, each with a job:

Algorithm	Graph type	Complexity	Result
BFS	unweighted	$O(V + E)$	distances from one source
0-1 BFS	edges cost 0 or 1	$O(V + E)$	distances from one source
Dijkstra	non-negative weights	$O((V + E) \log V)$	distances from one source
Bellman-Ford	any weights (no negative cycles)	$O(V \cdot E)$	distances from one source, negative OK
Floyd-Warshall	any weights (no negative cycles)	$O(V^3)$	distances between ALL pairs

Dijkstra — the workhorse

Idea (greedy): repeatedly finalize the unvisited node with the SMALLEST known distance. When node u is finalized, its distance is guaranteed optimal – because any other path to u must leave the "settled set" through a node with $\geq$ distance, and edge weights ≥ 0 can only increase it. Relaxation: a path through u may improve neighbors.

```
const long long INF = 4e18;
```

```
vector<long long> dijkstra(int src, int n, vector<vector<pair<int,int>>>& adj) {
    vector<long long> dist(n, INF);
    dist[src] = 0;
    priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;           // stale entry – skip (critical!)
        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});      // push improved distance
            }
        }
    }
    return dist;
}
```

```
//  $O((V + E) \log V)$  with a heap; each relaxation is a heap push.
```

Step-by-step on the classic example ($S \rightarrow A:4$, $S \rightarrow B:2$, $B \rightarrow A:1$, $B \rightarrow D:5$, $A \rightarrow D:1$, $D \rightarrow T:3$):

```
dist: S=0, rest=∞. pq = {(0,S)}
pop S → relax: A=4, B=2 → pq={(2,B),(4,A)}
pop B(2) → relax: A=min(4,3)=3, D=7 → pq={(3,A),(4,A-stale),(7,D)}
pop A(3) → relax: D=min(7,4)=4 → pq={(4,D),(7,D-stale)}
pop D(4) → relax: T=7 → pq={(7,T)}
pop T(7) → done. S→B→A→D→T = 2+1+1+3 = 7 ✓
```

Why no negative weights? The greedy "finalize smallest" step assumes settled distances can't improve. A negative edge can improve an already-settled node → wrong answers. Use Bellman-Ford for

negative weights.

Path reconstruction: keep $\text{prev}[v] = u$ when relaxing; walk back from the target.

Bellman-Ford — handles negative weights, detects negative cycles

Idea: relax ALL edges $V-1$ times. After k passes, every path with $\leq k$ edges is optimal; the shortest simple path has at most $V-1$ edges. One more pass: if anything still improves $\rightarrow$ negative cycle exists.

```
// edge list: struct Edge { int u, v, w; };
vector<long long> bellmanFord(int src, int n, vector<Edge>& edges) {
    vector<long long> dist(n, INF); dist[src] = 0;
    for (int i = 0; i < n - 1; ++i) {
        bool changed = false;
        for (auto& e : edges)
            if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) {
                dist[e.v] = dist[e.u] + e.w;
                changed = true;
            }
        if (!changed) break; // early exit: already optimal
    }
    for (auto& e : edges) // one more pass
        if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v])
            return {}; // negative cycle reachable
    return dist;
}
// O(V·E)
```

Step-by-step: edges $S \rightarrow B:2$, $S \rightarrow A:4$, $B \rightarrow A:1$, $B \rightarrow D:5$, $A \rightarrow D:1$, $D \rightarrow T:3$

Pass 1: $S \rightarrow A=4$, $S \rightarrow B=2$ (order matters per pass, all edges relaxed once)

Pass 2: $B \rightarrow A \rightarrow 2+1=3 < 4 \rightarrow A=3$; $B \rightarrow D=7$; $A \rightarrow D \rightarrow 3+1=4 < 7 \rightarrow D=4$

Pass 3: $D \rightarrow T=7$

Pass 4–6: no changes $\rightarrow$ done. Same result as Dijkstra ✓

Floyd-Warshall — all-pairs in $O(V^3)$

Idea: DP over intermediate nodes. $\text{dist}[k][i][j]$ = shortest path using only nodes $0..k$ as intermediates $\rightarrow \text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$.

```
vector<vector<long long>> floydWarshall(int n, vector<vector<long long>>& dist) {
    // dist[u][v] = w, dist[u][u] = 0, INF otherwise (input setup)
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                if (dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
    return dist;
}
// O(V^3). Works with negative weights (no negative cycles); detects them if
// dist[i][i] < 0 after the loop.
```

Why k on the outside? The invariant is "paths through nodes $\{0..k\}$ ". If k were the innermost loop, you'd reuse paths that already used k — losing the invariant. Get the loop order right: **k , then i , then j** .

0-1 BFS — weights are only 0 or 1

Idea: a deque. Weight-0 edges push to FRONT (same distance level), weight-1 edges push to BACK (next level). Each node is finalized the first time popped — like BFS but with two "levels" at once. $O(V + E)$, better than Dijkstra's log factor.

```
vector<int> bfs01(int src, int n, vector<vector<pair<int,int>>>& adj) {
    vector<int> dist(n, INT_MAX); dist[src] = 0;
```

```

deque<int> dq; dq.push_front(src);
while (!dq.empty()) {
    int u = dq.front(); dq.pop_front();
    for (auto [v, w] : adj[u]) {
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            if (w == 0) dq.push_front(v);
            else       dq.push_back(v);
        }
    }
}
return dist;
}

```

Choosing the right tool

Situation	Algorithm
Unweighted	BFS
Weights 0/1	0-1 BFS
Non-negative weights, one source	Dijkstra
Negative weights, one source, no neg. cycle	Bellman-Ford
All pairs	Floyd-Warshall
Negative cycle reachable	Bellman-Ford (detect), then it's ill-posed

Common mistakes

1. **Dijkstra with negative weights** — silently wrong. Check the constraint before choosing.
2. **No stale entry skip** (if ($d > \text{dist}[u]$) continue;) — without it the heap grows with outdated entries; correctness survives but complexity degrades badly.
3. **INF + w overflow** — use long long and check $\text{dist}[u] \neq \text{INF}$ before relaxing.
4. **Floyd-Warshall loop order** — must be k, i, j.
5. **Bellman-Ford missing the negative-cycle detection pass** — or running V passes instead of V-1.
6. **BFS distance on weighted graphs** — BFS levels $\neq$ weighted shortest paths.
7. **Not initializing** $\text{dist}[\text{src}] = 0$ or forgetting $\text{dist}[u][u] = 0$ for Floyd.
8. **1-based input vs 0-based arrays** — resize to $n+1$.

Practice questions

- **Easy 1:** BFS shortest path (unweighted) in a grid — min steps from start to target.
- **Medium 1: Network delay time** — time for a signal to reach all nodes = max of Dijkstra distances (or -1 if unreachable).
- **Medium 2: Cheapest flights within k stops** — Bellman-Ford restricted to k iterations (each pass = one stop).
- **Medium 3: Path with maximum probability** — Dijkstra variant on log probabilities (or "max-probability" heap version).
- **Challenging 1: Swim in rising water / minimum path with max edge** — Dijkstra where the "distance" is the max edge on the path.

- **Challenging 2: Word ladder II** — all shortest transformation sequences (BFS + backtracking, or bidirectional BFS).
- **Challenging 3: Shortest path in a grid with k obstacle removals** — BFS with state (r, c, remaining removals).
- **Challenging 4: Find the city with the smallest number of neighbors at a distance $\leq$ threshold** — **all-pairs (Floyd)** then count.

Hints & solutions: file 30-solutions.md.

21 — Topological Sort & Minimum Spanning Tree

Part A — Topological sort (DAGs)

What it is

A **topological order** of a **directed acyclic graph (DAG)** lists vertices so that for every edge $u \rightarrow v$, **u comes before v**. Dependencies in order: courses before their prerequisites, tasks before the things that depend on them.

DAG: $A \rightarrow C \rightarrow E$ Valid topological orders: A, B, C, D, E (and A,B,D,C,E etc.)
 $B \rightarrow C \rightarrow D$

Why it's useful

- Scheduling with dependencies (build systems, course prerequisites).
- Finding **longest paths in a DAG** (DP in topological order) — critical path analysis.
- Detecting cycles: a DAG always has a full topological order; a graph with a cycle doesn't.

Algorithm 1 — Kahn's algorithm (BFS on in-degrees)

Idea: a node with in-degree 0 has no remaining prerequisites $\rightarrow$ emit it.
Remove it (decrement neighbors' in-degrees); repeat. If we emit fewer than V nodes
 $\rightarrow$ the graph has a cycle.

```
vector<int> topoSort(int n, vector<vector<int>>& adj) {
    vector<int> indeg(n, 0);
    for (int u = 0; u < n; ++u)
        for (int v : adj[u]) indeg[v]++;

    queue<int> q;
    for (int u = 0; u < n; ++u)
        if (indeg[u] == 0) q.push(u);

    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u])
            if (--indeg[v] == 0) q.push(v);    // v's last prerequisite is done
    }
    return order.size() == n ? order : vector<int>{};    // empty  $\rightarrow$  cycle exists
}
// O(V + E)
```

Step-by-step: edges $A \rightarrow C$, $B \rightarrow C$, $C \rightarrow D$

indeg: $A=0, B=0, C=2, D=1 \rightarrow q=[A, B]$
pop $A \rightarrow C$ indeg $2 \rightarrow 1$; pop $B \rightarrow C$ $1 \rightarrow 0 \rightarrow q=[C]$
pop $C \rightarrow D$ $1 \rightarrow 0 \rightarrow q=[D]$; pop $D \rightarrow$ order= $[A, B, C, D]$ ✓ (A before C, B before C, C before D)

Algorithm 2 — DFS-based topological sort

DFS postorder gives "dependencies first" — reverse it. Cycle detection via a three-color visited array (0=unvisited, 1=in current stack, 2=done).

```
vector<int> topoSortDFS(int n, vector<vector<int>>& adj) {
    vector<int> color(n, 0), order;
    bool cyclic = false;
    function<void(int)> dfs = [&](int u) {
        color[u] = 1;    // gray: on stack
        for (int v : adj[u]) {
            if (color[v] == 1) cyclic = true;    // back edge  $\rightarrow$  cycle
            else if (color[v] == 0) dfs(v);
        }
    };
    for (int u = 0; u < n; ++u)
        if (color[u] == 0) dfs(u);
    reverse(order.begin(), order.end());
    return cyclic ? vector<int>{} : order;
}
```

```

    }
    color[u] = 2; // black: done
    order.push_back(u); // postorder
};
for (int u = 0; u < n; ++u) if (color[u] == 0) dfs(u);
if (cyclic) return {};
reverse(order.begin(), order.end()); // postorder reversed = topo order
return order;
}
// O(V + E)

```

Why reverse postorder works: in DFS, a node finishes only after all its descendants finish → descendants come *before* it in postorder → reversed, every $u \rightarrow v$ has u before v . ✓

Longest path in a DAG (the payoff)

Process nodes in topological order: $\text{longest}[v] = \max(\text{longest}[v], \text{longest}[u] + w(u,v))$. Because predecessors come first, $\text{longest}[u]$ is final when we relax v . **$O(V + E)$** — for general graphs this problem is NP-hard.

Part B — Minimum Spanning Tree (MST)

What it is

A **spanning tree** connects all V vertices with $V-1$ edges (no cycles). The **minimum spanning tree** is the spanning tree with the smallest total edge weight — the cheapest way to connect everything.

Why it's useful

- Network design: laying cable/roads/fiber connecting all cities at minimum cost.
- Clustering, approximation algorithms (TSP), and a warm-up for more advanced topics.

Algorithm 1 — Kruskal (sort edges + DSU)

Idea: sort edges by weight; add an edge if it connects two different components (union-find check). Adding it creates a cycle otherwise — skip.
Why greedy is correct (cut property): the cheapest edge crossing any cut is in some MST.

```

// DSU (topic 25) — find with path compression, union by size.
vector<Edge> kruskal(int n, vector<Edge>& edges) {
    sort(edges.begin(), edges.end(), [](auto& a, auto& b) { return a.w < b.w; });
    DSU dsu(n);
    vector<Edge> mst;
    for (auto& e : edges) {
        if (dsu.find(e.u) != dsu.find(e.v)) {
            dsu.unite(e.u, e.v);
            mst.push_back(e);
            if ((int)mst.size() == n - 1) break; // tree complete
        }
    }
    return mst; // size < n-1 → graph disconnected
}
// O(E log E) — dominated by the sort (DSU ops are near- $O(1)$ )

```

Step-by-step: edges (A,B,1) (B,C,2) (A,C,3) (C,D,4)

sort → (A,B,1) (B,C,2) (A,C,3) (C,D,4)
 (A,B) different comps → take. (B,C) → take. (A,C): same comp (A-B-C) → skip (would cycle).
 (C,D) → take → $\text{mst} = \{AB, BC, CD\}$ weight $1+2+4=7$. ✓ (AC would create a cycle.)

Algorithm 2 — Prim (grow from a seed with a heap)

Idea: start from any node; repeatedly add the CHEAPEST edge connecting the tree to an outside node. Uses a min-heap keyed by edge weight.

```

long long prim(int n, vector<vector<pair<int,int>>>& adj) {
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq; // (weight, node)
    vector<bool> inTree(n, false);
    pq.push({0, 0});
    long long total = 0; int taken = 0;
    while (!pq.empty() && taken < n) {
        auto [w, u] = pq.top(); pq.pop();
        if (inTree[u]) continue; // stale
        inTree[u] = true; total += w; taken++;
        for (auto [v, c] : adj[u])
            if (!inTree[v]) pq.push({c, v});
    }
    return taken == n ? total : -1; // -1 → disconnected
}
// O((V + E) log V) with a heap

```

Step-by-step (same graph): start at A

pq={0,A} → take A, push (1,B),(3,C)
 pop (1,B) → take B, push (2,C)
 pop (2,C) → take C, push (4,D)
 pop (4,D) → take D → total 1+2+4=7 ✓ (the (3,C) entry is skipped as stale)

Kruskal vs Prim

	Kruskal	Prim
Structure	edge list, sort	adjacency list, heap
Best when	sparse graphs (few edges)	dense graphs
Key tool	DSU	priority queue
Complexity	$O(E \log E)$	$O((V+E) \log V)$

Both rely on the **cut property**: the lightest edge across any cut belongs to some MST.

Common mistakes

1. **Topo sort**: running Kahn on a graph with a cycle → produces $< V$ nodes; always check.
2. **Topo sort**: forgetting the DAG requirement — topological order doesn't exist for undirected or cyclic graphs.
3. **DFS topo**: forgetting to reverse postorder (or using preorder — wrong direction).
4. **Kruskal**: forgetting the DSU — checking connectivity by scanning is $O(n)$ per edge → $O(E \cdot V)$. Union-find makes it near $O(1)$.
5. **Kruskal**: adding an edge within the same component (creates a cycle) — check find inequality first.
6. **Prim**: not skipping stale heap entries (`inTree[u]` check) — same bug family as Dijkstra.
7. **MST with negative weights** — fine! Both algorithms handle negatives; the "min" is well-defined. (No negative cycles concept applies — trees have no cycles.)
8. **Assuming MST = shortest paths** — they're different structures; Dijkstra $\neq$ MST.

Practice questions

- **Easy 1**: Given a DAG's edges, output a valid topological order (Kahn's).
- **Easy 2**: Detect if a directed graph has a cycle (Kahn's: emit $< V$ nodes → cycle).
- **Medium 1: Course schedule** — finish all courses given prerequisites (topo + cycle)

check); then **Course schedule II** — return the order itself.

- **Medium 2:** Longest path in a DAG (topological DP).

- **Medium 3: Minimum cost to connect all cities** (Kruskal or Prim; return -1 if impossible).

- **Challenging 1: Critical connections in a network** — edges that, if removed, disconnect the graph (bridges — needs Tarjan; topic 28).

- **Challenging 2: Smallest string with swaps / lexicographically smallest by swapping connected positions** — **DSU + topological-ish ordering (revisit after topic 25)**.

- **Challenging 3: Second-best MST** — the MST excluding exactly one edge of the MST (find the cheapest non-tree edge replacing a tree edge).

Hints & solutions: file 30-solutions.md.

22 — Dynamic Programming (DP)

What it is

DP is **recursion + memoization, systematized**: you solve a problem by combining solutions to *overlapping subproblems, and you **store** subproblem answers so each is computed once.

Two requirements:

1. **Optimal substructure** — the optimal solution contains optimal solutions to subproblems (you can combine optimal sub-solutions).
2. **Overlapping subproblems** — the same subproblems recur (otherwise plain divide & conquer is fine — e.g. merge sort).

Why it's useful

DP is the difference between "exponential brute force" and "polynomial": naive Fibonacci is $O(2^n)$, DP Fibonacci is $O(n)$. It's the pattern behind almost every "count the ways / maximize / minimize / is it possible" problem: knapsack, LCS, LIS, edit distance, coin change, grid paths, and more.

The two styles

TOP-DOWN (memoization): recursive, natural to write. Start from the full problem, recurse down, cache results in a table. Risk: recursion depth.

BOTTOM-UP (tabulation): iterative. Fill a table from the base cases up; the answer is a table cell. No recursion, usually faster, often space-optimizable.

Step-by-step — Fibonacci (the template problem)

$F(0)=0$, $F(1)=1$, $F(n)=F(n-1)+F(n-2)$. Subproblems overlap (topic 13's tree).

Top-down:

```
long long fibTD(int n) {
    static vector<long long> memo(100, -1);    // -1 = not computed
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];         // reuse!
    return memo[n] = fibTD(n - 1) + fibTD(n - 2); // compute once, store
}
// O(n) time, O(n) space
```

Bottom-up:

```
long long fibBU(int n) {
    if (n <= 1) return n;
    vector<long long> dp(n + 1);
    dp[0] = 0; dp[1] = 1;                      // base cases
    for (int i = 2; i <= n; ++i)
        dp[i] = dp[i - 1] + dp[i - 2];         // reuse table
    return dp[n];
}
// O(n) time, O(n) space → and since only dp[i-1], dp[i-2] are needed: O(1) space
// by keeping just two variables. ALWAYS look for the space optimization.
```

The 5-step DP recipe

1. **Define the state** — what does $dp[i]$ (or $dp[i][j]$) mean? One sentence, precise.

Example: " $dp[i]$ = max profit using items 0..i with capacity..." — the state *is* the subproblem.

2. **Write the base case(s)** — the smallest states, filled directly.

3. **Write the transition** — how does $dp[i]$ combine smaller $dp[j]$'s? This is the heart.
4. **Choose the answer cell** — often $dp[n]$, $dp[0][n-1]$, or a max over a row.
5. **Analyze** — time = states $\times$ transition cost; space = table size; then optimize space.

Pattern 1 — 1D DP: climbing stairs / house robber

Stairs: $ways(n) = ways(n-1) + ways(n-2)$ (1 or 2 steps at a time).

House robber: $dp[i] = \max(dp[i-1], dp[i-2] + a[i])$ — skip house i , or rob it.

```
int rob(vector<int>& a) {
    int n = a.size();
    if (n == 0) return 0;
    if (n == 1) return a[0];
    vector<int> dp(n);
    dp[0] = a[0];
    dp[1] = max(a[0], a[1]);
    for (int i = 2; i < n; ++i)
        dp[i] = max(dp[i-1], dp[i-2] + a[i]); // skip i vs rob i
    return dp[n-1];
}
// O(n) time, O(1) space with two variables
```

Pattern 2 — 2D DP: grid paths / edit distance / LCS

Unique paths in an $m \times n$ grid (move right/down only):

$dp[i][j] = dp[i-1][j] + dp[i][j-1]$ — you get here from above or from the left.

```
int uniquePaths(int m, int n) {
    vector<vector<long long>> dp(m, vector<long long>(n, 0));
    for (int i = 0; i < m; ++i) dp[i][0] = 1; // base: first column
    for (int j = 0; j < n; ++j) dp[0][j] = 1; // base: first row
    for (int i = 1; i < m; ++i)
        for (int j = 1; j < n; ++j)
            dp[i][j] = dp[i-1][j] + dp[i][j-1];
    return dp[m-1][n-1];
}
// O(m·n) time, O(m·n) space → only previous row needed → O(n) space optimization
```

LCS (longest common subsequence):

```
dp[i][j] = LCS of a[0..i-1] and b[0..j-1]
if a[i-1]==b[j-1]: dp[i][j] = dp[i-1][j-1] + 1
else:
    dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

Pattern 3 — Knapsack (the archetype)

0/1 knapsack: n items (weight w , value v), capacity W . Max value.

State: $dp[i][c]$ = best value using first i items with capacity c .

```
skip item i: dp[i-1][c]
take item i: dp[i-1][c - w[i]] + v[i] (only if c >= w[i])
dp[i][c] = max(skip, take)
```

```
int knapsack(vector<int>& w, vector<int>& v, int W) {
    int n = w.size();
    vector<int> dp(W + 1, 0); // 1D! iterate capacity backwards
    for (int i = 0; i < n; ++i)
        for (int c = W; c >= w[i]; --c) // backwards — critical for 0/1
            dp[c] = max(dp[c], dp[c - w[i]] + v[i]);
    return dp[W];
}
// O(n·W) time, O(W) space.
// Backwards iteration = each item used at most once (dp[c - w[i]] still from previous i).
// Forward iteration = UNBOUNDED knapsack (items reusable) — a classic trap.
```

Step-by-step: items $(w,v) = (2,3), (3,4), (4,5)$; $W = 5$

```

i=0 (w2,v3): dp = [0,0,3,3,3,3]
i=1 (w3,v4): c=5: max(dp[5]=3, dp[2]+4=7)=7; c=4: max(3, dp[1]+4)=4; c=3: max(3, dp[0]+4)=4
            dp = [0,0,3,4,4,7]
i=2 (w4,v5): c=5: max(7, dp[1]+5)=7; c=4: max(4, dp[0]+5)=5 → dp=[0,0,3,4,5,7]
answer 7 = item0+item1 (3+4) ✓

```

Pattern 4 — Coin change (min coins / count ways)

Min coins for amount A with coin denominations (unlimited supply):
 $dp[a] = 1 + \min \text{ over coins } c \leq a \text{ of } dp[a - c]$; $dp[0] = 0$.
 Unlimited supply → iterate amount FORWARD (unbounded knapsack).

```

int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, INT_MAX / 2);
    dp[0] = 0;
    for (int a = 1; a <= amount; ++a)
        for (int c : coins)
            if (c <= a) dp[a] = min(dp[a], dp[a - c] + 1);
    return dp[amount] >= INT_MAX / 2 ? -1 : dp[amount];
}
// 0(amount * #coins)

```

Pattern 5 — LIS: longest increasing subsequence

$dp[i]$ = length of longest increasing subsequence ENDING at i :
 $dp[i] = 1 + \max(dp[j])$ over $j < i$ with $a[j] < a[i]$; base $dp[i] = 1$.
 $O(n^2)$. There's an $O(n \log n)$ version with a "tails" array (patience sorting) –
 know both; the $O(n^2)$ version is easier to get right first.

When to reach for DP — recognition checklist

- **"Count the number of ways to..."** (paths, decodings, combinations) → DP.
- **"Maximum/minimum value subject to constraints"** (knapsack, edit distance, cutting rod) → DP (or greedy — test greedy with counterexamples first).
- **"Is it possible / can we partition"** (subset sum, partition equal) → DP (bool table).
- **"Subsequence / substring optimal"** (LCS, LIS, palindromic) → DP.
- **"From state A to state B with choices at each step"** → DP on states.
- Constraints like $n \leq 10^3$ with a natural 2D state → DP is the expected solution.

Common mistakes

1. **No base case** or wrong one — the table's first row/column must be hand-filled.
2. **Index translation** — with 1-indexed-style states, remember $a[i-1]$ vs $a[i]$; mixing them is the #1 DP bug.
3. **0/1 vs unbounded** — iterate capacity backwards (each item once) vs forwards (reusable). Getting this backwards silently doubles/triples answers.
4. **State too small** — forgetting a dimension (e.g. "at most k transactions" needs k in the state) → wrong answers on tests you didn't imagine.
5. **Top-down without memo** — that's just exponential recursion.
6. **Table too big** — $n^2 = 10^{10}$ cells won't fit; look for a smaller state or different approach.
7. **Overflow** in sums (ways can exceed int) — long long or modulo.
8. **Greedy applied where DP is needed** (or vice versa) — re-read topic 18's trap table.

- 9. **Space optimization too early** — get the 2D table right first, then collapse rows.
- 10. **Recursion depth in top-down** — deep states overflow; switch to bottom-up.

Practice questions

- **Easy 1:** Climbing stairs (count ways).
- **Easy 2:** House robber (1D max).
- **Easy 3:** Minimum cost climbing stairs — $dp[i] = cost[i] + \min(dp[i-1], dp[i-2])$.
- **Medium 1:** 0/1 knapsack (print the chosen items too, using the full 2D table).
- **Medium 2: Longest increasing subsequence** ($O(n^2)$, then $O(n \log n)$).
- **Medium 3: Coin change** (min coins; then the "number of ways" variant).
- **Medium 4: Longest common subsequence** of two strings.
- **Medium 5: Edit distance** — min ops (insert/delete/replace) to transform $a \rightarrow b$.
- **Challenging 1: Word break** — can s be segmented into dictionary words

(1D bool DP + substring checks).

- **Challenging 2: Partition equal subset sum** — can you split into two equal-sum halves (subset-sum bool DP).

- **Challenging 3: Best time to buy/sell stock with at most k transactions** (2D state with transaction count).

- **Challenging 4: Regular expression matching** ($.$ and $*$) — 2D DP with careful $*$ handling.

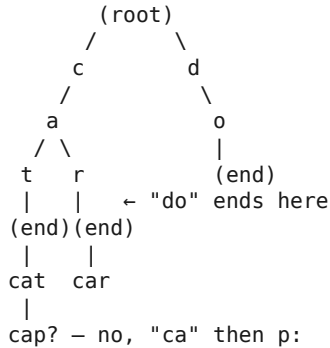
Hints & solutions: file 30-solutions.md.

23 — Tries (Prefix Trees)

What it is

A **trie** is a tree where each **edge is a character** and each node represents a *prefix. *Words sharing a prefix share the tree path. A node marked as "word end" means the prefix leading to it is a complete word.*

Insert: "cat", "car", "cap", "do"



Why it's useful

- **O(L) operations** (L = word length) for insert/search/prefix-check — independent of the number of stored words. A hash map is O(L) average too, but a trie:
 - Supports **prefix queries** (autocomplete, longest common prefix) that hashing can't do efficiently.
 - **Sorted iteration** if you traverse lexicographically.
 - Powers spell check, IP routing, and is a component of **Aho-Corasick** string matching.

Implementation (array-of-pointers version)

```
struct TrieNode {
    TrieNode* next[26] = {};    // one slot per lowercase letter
    bool isWord = false;
};

class Trie {
    TrieNode* root;
public:
    Trie() { root = new TrieNode(); }

    void insert(const string& w) {
        TrieNode* cur = root;
        for (char c : w) {
            if (!cur->next[c - 'a'])
                cur->next[c - 'a'] = new TrieNode();
            cur = cur->next[c - 'a'];
        }
        cur->isWord = true;    // mark the full word
    }

    bool search(const string& w) {    // exact word
        TrieNode* cur = walk(w);
        return cur && cur->isWord;    // must end at a WORD end
    }

    bool startsWith(const string& p) {    // any word with this prefix
```

```

        return walk(p) != nullptr;
    }
private:
    TrieNode* walk(const string& s) {
        TrieNode* cur = root;
        for (char c : s) {
            if (!cur->next[c - 'a']) return nullptr;
            cur = cur->next[c - 'a'];
        }
        return cur;
    }
};
// All three ops O(L) time; space O(total characters over all words).

```

Step-by-step — insert "cat": root → create 'c' → create 'a' → create 't' (isWord=true). Now search("ca") → walks to 'a' but isWord is false → false. startsWith("ca") → true. search("cat") → true. ✓

Implementation (hash-map children) — for large alphabets / memory

```

struct TrieNode {
    unordered_map<char, TrieNode*> next;    // only existing children get nodes
    int count = 0;                        // optional: how many words pass through
    bool isWord = false;
};

```

Pattern 1 — Autocomplete / collect all words with a prefix

Walk to the prefix's node, then DFS from it collecting every word ending below.

```

void collect(TrieNode* n, string& cur, vector<string>& out) {
    if (n->isWord) out.push_back(cur);
    for (auto& [ch, child] : n->next) {
        cur.push_back(ch);
        collect(child, cur, out);
        cur.pop_back();
    }
}

vector<string> autocomplete(Trie& t, const string& prefix) {
    vector<string> out; string cur = prefix;
    TrieNode* p = t.walkPrefix(prefix);    // node after prefix
    if (p) collect(p, cur, out);
    return out;
}
// O(L + K) where K = total characters in matched words

```

Pattern 2 — Count words with a prefix (store counts in nodes)

Add int count to each node; increment on every insertion pass. Then "how many words start with X" is O(L) — read the count at the prefix's node. Also powers the classic "**replace words / prefix match in dictionary**" problems.

Pattern 3 — Longest common prefix of a set

Insert all words; the LCP is the path from the root while a node has **exactly one child** (and no word ends early). Single path until a branch or a word-end → that prefix is common to all.

Pattern 4 — XOR maximization (trie of bits)

Insert numbers bit by bit (MSB first, 31 bits). To maximize $x \oplus y$, walk the trie **preferring the opposite bit** at each level (greedy: the high bit dominates). O(32) per query.

```

int maxXor(vector<int>& a) {
    // simplified — one insert each

```

```

struct Node { Node* nxt[2] = {}; };
Node* root = new Node();
for (int x : a) { // insert
    Node* cur = root;
    for (int b = 30; b >= 0; --b) {
        int bit = (x >> b) & 1;
        if (!cur->nxt[bit]) cur->nxt[bit] = new Node();
        cur = cur->nxt[bit];
    }
}
int best = 0;
for (int x : a) { // query: prefer opposite bits
    Node* cur = root; int val = 0;
    for (int b = 30; b >= 0; --b) {
        int bit = (x >> b) & 1;
        if (cur->nxt[bit ^ 1]) { val |= (1 << b); cur = cur->nxt[bit ^ 1]; }
        else cur = cur->nxt[bit];
    }
    best = max(best, val);
}
return best;
}
// 0(31 · n) — vs 0(n2) brute force.

```

Common mistakes

1. **Search returns true for a *prefix* that isn't a word** — must check `isWord`.
2. **Forgetting to create missing nodes during insert** — null dereference.
3. `startsWith("")` — the empty prefix: the root itself is the answer (return true).
4. **Case/character-set assumptions** — 26 lowercase vs full alphabet vs digits; size the array accordingly (or use a map).
5. **Memory: full 26-slot arrays per node are wasteful** for sparse tries — use maps or sorted vectors of (char, child).
6. **Recursive collection without a word-end stop** — you collect all descendants, which is correct for autocomplete but surprising if you expected exact prefix count.
7. **Deletion** — real tries need reference counts; don't delete nodes with children still pointing to them. (Interviews rarely require deletion — mention the ref-count approach.)
8. **Not resetting cur between operations** — each op must start from root.

Practice questions

- **Easy 1:** Implement a Trie with `insert`, `search`, `startsWith`.
- **Easy 2:** Given a list of words, find the **longest common prefix** (trie, or simpler: sort + compare first/last — do both).
- **Medium 1: Replace words** — a dictionary of roots; replace every word in a sentence by its shortest root that is its prefix (trie walk).
- **Medium 2: Word search II** — find all dictionary words present in a letter grid (trie + grid backtracking; the trie prunes dead ends early).
- **Medium 3:** Count distinct **substrings** of a string using a trie of suffixes — or compute it with the suffix-array alternative.
- **Challenging 1: Maximum XOR of two numbers** in an array (bit trie, $O(n)$).

- **Challenging 2: Design a search autocomplete system** — top-k suggestions by frequency for a growing query (trie with counts + heap).

- **Challenging 3: Shortest unique prefix** for each word in a set (counts in nodes: stop at the first node whose count is 1).

Hints & solutions: file 30-solutions.md.

24 — Bit Manipulation

What it is

Bit manipulation works directly on the **binary representation** of integers. Computers natively think in bits, so bitwise ops are the fastest primitive available — and some problems are *defined* in bits (set/clear a flag, XOR tricks, subset enumeration).

Operators

```
x & y      AND      // 1 only where both have 1
x | y      OR       // 1 where either has 1
x ^ y      XOR      // 1 where they differ (addition without carry!)
~x         NOT      // flip all bits
x << k     left shift → multiply by 2^k
x >> k     right shift → divide by 2^k (floor for positive)
```

Tricks table (n = integer, bits indexed from 0 = least significant):

Trick	Code	Notes
Check bit k	<code>(n >> k) & 1</code>	1 if set
Set bit k	<code>n = (1 << k)</code>	
Clear bit k	<code>n &= ~(1 << k)</code>	
Flip bit k	<code>n ^= (1 << k)</code>	
Power of two	<code>n > 0 && (n & (n-1)) == 0</code>	strips lowest set bit
Lowest set bit	<code>n & (-n)</code>	isolates it
Count set bits	<code>__builtin_popcount(n)</code>	O(1) builtin
Is even/odd	<code>n & 1</code>	0 = even
Add 1 / subtract 1	<code>--n / ++n</code>	rarely needed, fun trivia
Clear lowest set bit	<code>n & (n - 1)</code>	classic loop trick
Get all subsets	loop mask over <code>0..(1<<n)-1</code>	2^n masks
XOR self-cancels	<code>x ^ x == 0, x ^ 0 == x</code>	THE trick for "single number"
Swap (no temp)	<code>a ^= b; b ^= a; a ^= b;</code>	works, but just use <code>std::swap</code>

Pattern 1 — XOR tricks

Problem: every element appears twice except one — find the singleton.

$x \oplus x = 0$ and XOR is commutative → XOR everything; pairs cancel, singleton remains.

```
int singleNumber(vector<int>& a) {
    int x = 0;
    for (int v : a) x ^= v;      // all pairs cancel → x = the singleton
    return x;
}
// O(n) time, O(1) space — beats every hash-map solution.
```

Step-by-step: $a = [4, 1, 2, 1, 2] \rightarrow 4 \oplus 1 \oplus 2 \oplus 1 \oplus 2 = 4 \oplus (1 \oplus 1) \oplus (2 \oplus 2) = 4 \oplus 0 \oplus 0 = 4 \checkmark$

Variants: **every element appears 3 times except one** → count bits mod 3, or track bits with *ones/twos* masks. **Two singletons** → XOR all (get $a \oplus b$); a set bit of the XOR splits the array into two groups, each with one singleton.

Pattern 2 — Bit-counting / DP

Problem: count bits of $0..n$ (i.e., `popcount(i)` for all i).

$O(n)$ with DP: `popcount(i) = popcount(i >> 1) + (i & 1)`.

```
vector<int> countBits(int n) {
    vector<int> dp(n + 1);
    for (int i = 1; i <= n; ++i)
        dp[i] = dp[i >> 1] + (i & 1);    // drop the LSB, reuse its count, add back if odd
    return dp;
}
```

Pattern 3 — Subset enumeration by masks

Problem: all subsets of $\{a,b,c\}$. Enumerate mask $0..7$; bit k set $\leftrightarrow$ include $a[k]$.

```
vector<vector<int>> subsetsByMask(const vector<int>& a) {
    int n = a.size();
    vector<vector<int>> res;
    for (int mask = 0; mask < (1 << n); ++mask) {
        vector<int> sub;
        for (int i = 0; i < n; ++i)
            if (mask & (1 << i)) sub.push_back(a[i]);
        res.push_back(sub);
    }
    return res;
}
//  $O(n \cdot 2^n)$ . The same mask trick powers bitmask DP (TSP over subsets).
```

Pattern 4 — Power of two & related checks

```
bool isPowerOfTwo(int n) { return n > 0 && (n & (n - 1)) == 0; }
// n=16 (10000): 16 & 15 (01111) = 0  $\rightarrow$  true. n=12 (01100): 12 & 11 = 8  $\neq$  0  $\rightarrow$  false.
```

Pattern 5 — Bitmasks for state (flags, visited sets)

A single `int` can hold 32 booleans:

```
int flags = 0;
flags |= (1 << 3);           // set flag 3
bool has = flags & (1 << 3); // check flag 3
flags &= ~(1 << 3);         // clear flag 3
```

Used for: "has visited city set S ending at city i " (bitmask DP / TSP), "which of 26 letters are present" (int mask — anagram equivalence), game states, and small visited sets in graphs.

Common mistakes

1. **Shift overflow** — `1 << 31` on a 32-bit `int` is UB-ish (negative). Use `1LL << k` or unsigned when k is large.
2. **Signed right shift of negatives** — `-1 >> 1` is implementation-defined-ish (arithmetic shift keeps sign). Prefer unsigned or positive values.
3. **Operator precedence** — `a & b == 0` parses as `a & (b == 0)`. **Always parenthesize** bitwise ops in conditions: `(a & b) == 0`.
4. `__builtin_popcount` **on** `long long` — use `__builtin_popcountll`.
5. **Forgetting that XOR of $0..n$ has a formula** — $n\%4$ determines it ($0 \rightarrow n$, $1 \rightarrow 1$, $2 \rightarrow n+1$, $3 \rightarrow 0$). Saves $O(n)$ loops.
6. **Integer width assumptions** — 31-bit vs 64-bit problems: `(x >> 31) & 1` for the sign

bit, or use 64-bit types.

7. **Using ~ to clear bits without masking the width** — ~ 0 is all ones (infinite); combine with & of a mask.

8. **Trying bit tricks where a hash map is clearer** — bits are for speed/memory; don't obfuscate for fun.

Practice questions

- **Easy 1:** Single number (XOR).
- **Easy 2:** Count the number of set bits of a 32-bit integer.
- **Easy 3:** Check if n is a power of two.
- **Medium 1: Reverse bits** of a 32-bit unsigned integer.
- **Medium 2: Missing number** — $0..n$ with one missing, $O(1)$ space (XOR or sum).
- **Medium 3:** Count bits for $0..n$ (DP).
- **Challenging 1: Single number II** — every element appears $3\times$ except one.
- **Challenging 2: Maximum XOR of two numbers** (bit trie — topic 23).
- **Challenging 3: Subsets** with bitmask enumeration (then compare with backtracking — same answer, different tools).
- **Challenging 4: Sum of two integers without + or -** (carry = $a \& b < 1$, sum = $a \oplus b$, loop until carry is 0).

Hints & solutions: file 30-solutions.md.

25 — Disjoint Set Union (DSU / Union-Find)

What it is

DSU maintains a collection of **disjoint sets** (each set = a connected component) and supports two operations:

- **find(x)** — which set does x belong to? (return a representative/root)
- **unite(x, y)** — merge the sets containing x and y.

Initially every element is its own set. `unite` joins them over time.

Why it's useful

- **Dynamic connectivity:** "are a and b connected?" as edges get added — $O(\alpha(n))$ per op, where α is the inverse-Ackermann function: **practically $O(1)$** .
- The engine of Kruskal's MST, cycle detection in undirected graphs, connected components, and a whole family of "merge groups / union by property" problems (accounts, friend circles, image connectivity).

How it works — two optimizations

1. Path compression (find): while walking to the root, point every node on the path directly at the root. Future finds are $O(1)$.

`find(4)` in `1 ← 2 ← 3 ← 4` → compress: `1 ← 2, 1 ← 3, 1 ← 4`

2. Union by size/rank (unite): attach the smaller tree under the larger one — keeps trees shallow (height $\leq \log n$ without compression).

```
struct DSU {
    vector<int> parent, sz; // sz = component size
    DSU(int n) : parent(n), sz(n, 1) {
        iota(parent.begin(), parent.end(), 0); // parent[i] = i
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]); // path compression
        return parent[x];
    }
    void unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return; // already connected
        if (sz[a] < sz[b]) swap(a, b); // union by size
        parent[b] = a;
        sz[a] += sz[b];
    }
    bool same(int a, int b) { return find(a) == find(b); }
};
```

// With both optimizations: $O(\alpha(n))$ amortized per op — effectively constant.

Step-by-step — building components

`unite(1,2) unite(2,3) unite(4,5)` on `{1,2,3,4,5}`
After: sets `{1,2,3}` (root 1), `{4,5}` (root 4)
`find(3) → 1` ; `find(5) → 4` ; `same(2,4) → false`. `unite(3,4)`: sizes 3 vs 2 → root 1 absorbs 4.
Now `same(2,4) → true`. Component count = 5 - 3 unites that merged distinct sets = 2 → 1.

Pattern 1 — Kruskal's MST (see topic 21 for full context)

// sort edges by weight; for each, if `find(u) != find(v)` → unite + add to MST.

```
// The union-find here IS the cycle detection: two nodes already connected = adding this
// edge would create a cycle.
```

Pattern 2 — Cycle detection in an undirected graph

For each edge (u,v): if find(u) == find(v) → cycle exists. Else unite.
Works online (as edges arrive) — no DFS needed. $O(E \cdot \alpha)$.

Pattern 3 — Connected components count / sizes

```
int components = n;
// after each successful unite: components--.
// Size of a component = sz[find(x)].
```

Pattern 4 — "Merge with a twist" (the interview favorite)

Classic hard-ish problems combine DSU with extra bookkeeping:

- **Accounts merge / friend circles:** map emails → person; unite on shared emails; group by root at the end.
- **Lexicographically smallest string after swapping connected positions:** DSU groups swappable positions; sort each group's characters and place them back.
- **Number of islands II (additions online):** each add unions with existing land neighbors; decrement count on merges — answer after every query.
- **Smallest equivalent string (equivalence classes):** union letters, keep the smallest letter as root.
- **Offline queries / reverse processing:** when deletes are hard, process operations backwards as *adds* (classic "number of provinces after removing edges").

Common mistakes

1. **Forgetting find in unite** — you must unite the *roots, *not the raw elements*, or the tree corrupts.
2. **Path compression without union by size (or vice versa)** — both together give $\alpha(n)$; alone, either gives $O(\log n)$ worst case (still fine, but both is standard).
3. **Not returning the new root from recursive find** — `parent[x] = find(parent[x]);` (assignment!) is what compresses.
4. **same(a,b) implemented as `parent[a] == parent[b]`** — WRONG; must call find on both (parent pointers aren't always roots after unions).
5. **Component count bookkeeping wrong** — decrement only when roots differ.
6. **Recursive find on deep trees** — with compression trees are shallow, but for adversarial/uncompressed usage convert to iterative.
7. **0-based vs 1-based** — resize to $n+1$ for 1-based inputs.

Practice questions

- **Easy 1:** Number of **provinces** (friend circles) — count connected components.
- **Easy 2:** Given edges, detect if an undirected graph has a cycle (DSU version).

- **Medium 1: Accounts merge** — group emails by person, output sorted.
- **Medium 2: Number of islands II** — online additions, answer after each add.
- **Medium 3: Smallest string with swaps** — lexicographically smallest string after arbitrary swaps within connected positions.
- **Challenging 1: Redundant connection** — the extra edge that creates a cycle in a near-tree (DSU in edge order → first edge connecting two already-connected nodes).
- **Challenging 2: Longest consecutive sequence** via DSU (or the set-based $O(n)$ trick from topic 07 — try both).
- **Challenging 3: Regions cut by slashes** — a grid of / and \ split into regions (map each cell to 4 sub-cells; union adjacent; count sets).

Hints & solutions: file 30-solutions.md.

26 — Fenwick Tree (Binary Indexed Tree, BIT)

What it is

A Fenwick tree is an array-based structure supporting two operations on a **1-indexed** array:

- **point update:** add x to $a[i]$
- **prefix query:** sum of $a[1..i]$

Both in **$O(\log n)$** — the sweet spot between a raw array (update $O(1)$, query $O(n)$) and a prefix-sum array (query $O(1)$, update $O(n)$).

Why it's useful

- The workhorse for **dynamic prefix sums** (updates interleaved with queries): inversion counting, k -th order statistics, range updates via difference trick, offline problems.
- **Simpler and faster than a segment tree** when you only need prefix queries + point updates. Use a segment tree (topic 27) when you need **range** queries on arbitrary functions (min, gcd) or range updates.

How it works

The trick: index i is responsible for the interval $(i - \text{lowbit}(i), i]$ where $\text{lowbit}(i) = i \& (-i)$ (the lowest set bit). So $\text{tree}[i]$ stores the sum of that slice.

```
Array (1-indexed): a = [_, 3, 1, 2, 7, 4]
tree[1] = a[1]           (1 = lowbit 1)
tree[2] = a[1] + a[2]    (2 = lowbit 2 → interval (0,2])
tree[3] = a[3]           (lowbit 1 → interval (2,3])
tree[4] = a[1..4]        (lowbit 4 → interval (0,4])
tree[5] = a[5]           ...
tree[6] = a[5] + a[6]    (lowbit 2 → interval (4,6])
```

Prefix sum: walk i down by $i -= \text{lowbit}(i)$, adding $\text{tree}[i]$ each step — covers exactly the range with non-overlapping slices. **Update:** walk i up by $i += \text{lowbit}(i)$ — touches exactly the trees containing $a[i]$.

prefix(5) with tree above: $i=5$: $+\text{tree}[5]=7 \rightarrow i=4$: $+\text{tree}[4]=13 \rightarrow i=0$ stop $\rightarrow 7+13=20 = 3+1+2+7+4$ ✓

Implementation

```
struct Fenwick {
    int n;
    vector<long long> tree;
    Fenwick(int n) : n(n), tree(n + 1, 0) {} // 1-indexed; size n+1

    void add(int i, long long delta) { // a[i] += delta
        for (; i <= n; i += i & (-i)) tree[i] += delta;
    }

    long long sum(int i) const { // prefix sum a[1..i]
        long long s = 0;
        for (; i > 0; i -= i & (-i)) s += tree[i];
        return s;
    }

    long long rangeSum(int l, int r) const { return sum(r) - sum(l - 1); }
};
// Both ops  $O(\log n)$ . Build: add each  $a[i] \rightarrow O(n \log n)$ , or  $O(n)$  via the
// "tree[i] = sum of (i-lowbit(i), i]" formula.
```

Step-by-step — inversion count (the classic use)

Problem: count pairs $(i < j)$ with $a[i] > a[j]$. Brute force $O(n^2)$.

Idea: scan left→right; for each $a[j]$, the number of *previous* elements GREATER than it = (elements seen so far) - (previous elements $\leq a[j]$). Keep counts in a Fenwick indexed by VALUE; query prefix $\leq a[j]$ in $O(\log n)$.

```
long long inversionCount(vector<int> a) {
    // coordinate-compress values to 1..m (values can be large/negative)
    vector<int> vals = a;
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());

    Fenwick bit(vals.size());
    long long inv = 0;
    for (int x : a) {
        int idx = lower_bound(vals.begin(), vals.end(), x) - vals.begin() + 1; // 1-indexed
        inv += bit.sum(vals.size()) - bit.sum(idx); // seen so far minus  $\leq x$ 
        bit.add(idx, 1);
    }
    return inv;
}
//  $O(n \log n)$ 
```

Trace $a = [3,1,2]$:

```
x=3 → idx=3; seen 0 → inv 0; add(3)
x=1 → idx=1; seen 1 - prefix(1)=0 → inv += 1 (pair (3,1)); add(1)
x=2 → idx=2; seen 2 - prefix(2)=1 → inv += 1 (pair (3,2)); add(2)
inv = 2 ✓
```

Pattern 1 — Range update, point query (difference trick)

Want: "add x to $a[l..r]$ " (many such updates), then read individual values.

Idea: maintain a BIT over a DIFFERENCE array. $\text{add}(l, x)$; $\text{add}(r+1, -x)$.

Point value of $a[i] = \text{bit.sum}(i)$ - the accumulated deltas at i .

```
// Range add  $[l..r]$  by  $x \rightarrow \text{bit.add}(l, x)$ ;  $\text{bit.add}(r + 1, -x)$ ;
// Value at  $i \rightarrow \text{bit.sum}(i)$ ;
//  $O(\log n)$  per op - vs  $O(n)$  per range update naively.
```

Pattern 2 — Range update, range sum (two BITs)

Maintain two trees: B1 for the $+x$ deltas, B2 for the $-x*(l-1) / +x*r$ corrections.

$\text{prefix}(r) = \text{B1.sum}(r)*r - \text{B2.sum}(r)$. (Derivation: sum over deltas weighted by position.)

Both range updates and range queries become $O(\log n)$ - the standard "2 BIT" trick.

Pattern 3 — k-th smallest / order statistics

Given counts in a BIT, find the smallest index with cumulative count $\geq k$ by binary lifting (descend from the highest power of two): $O(\log n)$. Powers Fenwick-based sorted structures.

```
int kth(int k) { // smallest i with sum(i)  $\geq k$  (1-indexed)
    int idx = 0;
    int bitMask = 1; while (bitMask << 1 <= n) bitMask <<= 1; // highest pow2  $\leq n$ 
    for (int d = bitMask; d >>= 1) {
        int nxt = idx + d;
        if (nxt <= n && tree[nxt] < k) {
            idx = nxt; k -= tree[nxt];
        }
    }
    return idx + 1;
}
```

Fenwick vs Segment Tree — when to use which

Need	Fenwick	Segment tree
Prefix sums + point updates	□ simpler, faster, less memory	□
Arbitrary range queries (min/max/gcd)	□	□
Range updates (add to range)	□ (2-BIT trick)	□ (+ lazy)
Order statistics / k-th	□ (lifting)	□
Code complexity	low	higher

Rule of thumb: prefix + point update → Fenwick. Everything else that needs *ranges* with non-invertible ops or range updates → segment tree.

Common mistakes

1. **0-indexing** — Fenwick is 1-indexed. Shift inputs by +1, allocate n+1, and never use index 0 (infinite loop: $i += i \& (-i)$ from 0 stays 0).
2. **sum loop direction wrong** — prefix goes *down* ($i -= \text{lowbit}$), update goes *up* ($i += \text{lowbit}$). Swap them → wrong answers or infinite loops.
3. **Off-by-one in** `rangeSum(l, r)` — `sum(r) - sum(l-1)`, not `sum(r) - sum(l)`.
4. **int overflow** in sums — use long long.
5. **Not compressing values** when values are large or negative (Fenwick indexes values).
6. **Confusing the range-update trick** — the difference-array BIT answers *point* queries; range-sum queries need the two-BIT variant.
7. **Forgetting** $i \& (-i)$ **with negative** i — keep i positive; $-i$ on unsigned is fine but on signed negatives it's the two's complement — works, but don't overthink; $i \geq 1$ always in the loops above.

Practice questions

- **Easy 1:** Dynamic prefix sums — array + point updates + range sum queries (the "range sum query" classic).
- **Medium 1: Inversion count** (Fenwick, $O(n \log n)$).
- **Medium 2: Range add, point query** (difference-BIT).
- **Medium 3: Range add, range sum** (two BITs).
- **Challenging 1: Count of smaller numbers after self** — for each i , count $j > i$ with $a[j] < a[i]$ (scan right→left with a Fenwick; or merge-sort counting — try both).
- **Challenging 2: Reverse pairs** — count $i < j$ with $a[i] > 2 \cdot a[j]$ (Fenwick + careful value compression, or merge sort).
- **Challenging 3: K-th order statistic with insert/delete** — maintain a multiset via Fenwick counts + binary lifting.

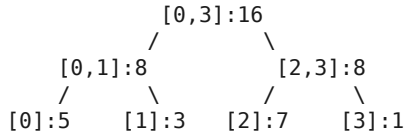
Hints & solutions: file 30-solutions.md.

27 — Segment Trees

What it is

A segment tree is a binary tree where each node stores the **answer for a range** of the array (sum, min, max, gcd, count, ...). The root covers $[0, n-1]$; each node splits its range in half into two children. Point updates and range queries both cost $O(\log n)$.

Array $a = [5, 3, 7, 1]$ (sums in nodes)



Why it's useful

- **Range queries with updates** — any *associative* combine operation (sum, min, max, gcd, OR, AND): query a range in $O(\log n)$, update one element in $O(\log n)$.
- **Lazy propagation** upgrades it to range *updates* in $O(\log n)$ — the Swiss-army knife of "array with range operations" problems.
- Use it when a Fenwick can't: non-invertible operations (min/max/gcd), range updates + range queries of arbitrary functions.

Implementation — point update + range query (array-based)

```
struct SegTree {
    int n;
    vector<long long> tree;
    SegTree(const vector<int>& a) {
        n = a.size();
        tree.resize(4 * n);          // 4n is always enough for the complete tree
        build(a, 1, 0, n - 1);
    }
    void build(const vector<int>& a, int node, int l, int r) {
        if (l == r) { tree[node] = a[l]; return; }
        int m = (l + r) / 2;
        build(a, node * 2, l, m);
        build(a, node * 2 + 1, m + 1, r);
        tree[node] = tree[node * 2] + tree[node * 2 + 1]; // combine
    }
    void update(int idx, long long val, int node, int l, int r) {
        if (l == r) { tree[node] = val; return; } // leaf - set new value
        int m = (l + r) / 2;
        if (idx <= m) update(idx, val, node * 2, l, m);
        else update(idx, val, node * 2 + 1, m + 1, r);
        tree[node] = tree[node * 2] + tree[node * 2 + 1]; // recombine
    }
    long long query(int ql, int qr, int node, int l, int r) {
        if (qr < l || r < ql) return 0; // no overlap
        if (ql <= l && r <= qr) return tree[node]; // fully covered
        int m = (l + r) / 2;
        return query(ql, qr, node * 2, l, m) +
               query(ql, qr, node * 2 + 1, m + 1, r); // partial → split
    }
};
// Build  $O(n)$ , update  $O(\log n)$ , query  $O(\log n)$ , space  $O(n)$ 
```

Step-by-step — query [1, 2] on the tree above

query(1,2) at root [0,3]:
 [0,3] partially covered → go left [0,1] and right [2,3]
 [0,1]: fully inside [1,2]? $ql=1 \leq l=0$? no, but $[0,1] \cap [1,2] \neq \emptyset \rightarrow$ split
 [0]: no overlap → 0 ; [1]: fully covered → 3
 [2,3]: split → [2]: fully covered → 7 ; [3]: no overlap → 0
 result $3 + 7 = 10 = a[1] + a[2] \checkmark$ ($O(\log n)$ nodes visited – the "covers" prune)

Lazy propagation — range updates in $O(\log n)$

Problem: "add x to $a[l..r]$ " for many ranges, then query. Naive: update each leaf $O(n)$.
 Idea: when an update FULLY covers a node, store the delta in a 'lazy' field and stop – don't touch its children. Push lazy down only when we later descend.

```
struct LazySeg {
    int n;
    vector<long long> tree, lazy;
    LazySeg(const vector<int>& a) : n(a.size()), tree(4*n, 0), lazy(4*n, 0) { build(a,1,0,n-1); }

    void build(const vector<int>& a, int node, int l, int r) { /* same as above */ }

    void apply(int node, int l, int r, long long delta) {
        tree[node] += delta * (r - l + 1); // whole segment gets delta
        lazy[node] += delta; // remember for the children
    }

    void push(int node, int l, int r) {
        if (lazy[node] && l != r) {
            int m = (l + r) / 2;
            apply(node * 2, l, m, lazy[node]);
            apply(node * 2 + 1, m + 1, r, lazy[node]);
            lazy[node] = 0;
        }
    }

    void rangeAdd(int ql, int qr, long long delta, int node, int l, int r) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) { apply(node, l, r, delta); return; } // stop here
        push(node, l, r); // hand lazy down
        int m = (l + r) / 2;
        rangeAdd(ql, qr, delta, node * 2, l, m);
        rangeAdd(ql, qr, delta, node * 2 + 1, m + 1, r);
        tree[node] = tree[node * 2] + tree[node * 2 + 1];
    }

    long long rangeSum(int ql, int qr, int node, int l, int r) {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return tree[node];
        push(node, l, r); // MUST push before splitting
        int m = (l + r) / 2;
        return rangeSum(ql, qr, node * 2, l, m) + rangeSum(ql, qr, node * 2 + 1, m + 1, r);
    }
};
// rangeAdd and rangeSum both  $O(\log n)$ 
```

Step-by-step — rangeAdd(0, 2, +5) on $a=[5,3,7,1]$:

root [0,3] partial → push (nothing yet) → recurse
 [0,1] full cover? $ql=0 \leq 0, 1 \leq qr=2 \rightarrow$ yes → apply: $tree=8+10=18$, $lazy=5 \rightarrow$ STOP
 [2,3] partial → push → split → [2] full cover → apply +5 → $tree=12$; [3] no overlap
 Rebuild up: root = $18 + 12 + 1 = 31$. Correct: $(5+5)+(3+5)+(7+5)+1 = 31 \checkmark$
 Later query [1,1]: descends into [0,1] → push $lazy=5$ into children first → $[1] = 3+5 = 8 \checkmark$

Pattern — "merge" beyond sum

The combine step is pluggable; just change the merge line:

Operation	Node value	No-overlap return	Merge
sum	sum	0	$a + b$

Operation	Node value	No-overlap return	Merge
min	min	$+\infty$	$\min(a, b)$
max	max	$-\infty$	$\max(a, b)$
gcd	gcd	0	$\gcd(a, b)$
count of X	count	0	$a + b$

Pattern — segment tree of segment trees / persistence (mention)

For 2D problems (rectangle queries) use a segment tree of Fenwicks/segment trees; for "query the past" use **persistent segment trees** (each update creates new nodes, shares the unchanged ones) — advanced, but the same core idea.

Common mistakes

1. **Array size $4n$** — forgetting the constant-4 factor causes silent out-of-bounds.
2. **Recursion bounds** — every call must check $qr < l \ || \ r < ql$ FIRST; missing the no-overlap case gives wrong merges.
3. **Lazy not pushed before descending** — queries read stale values. The golden rule: *push before splitting, apply on full cover, never update children without pushing.*
4. **Zero-initializing lazy but only applying to $l \neq r$** — pushing on a leaf is a no-op; guard it.
5. **The no-overlap return value** — must be the IDENTITY of the operation (0 for sum, $+\infty$ for min, $-\infty$ for max, 0 for gcd). Wrong identity = wrong answers.
6. long long **overflow** on sums — use long long for $\text{delta} * (r - l + 1)$.
7. **Building with `tree.resize(2*n)`-style shortcuts** — fine for iterative segment trees, but the recursive version needs $4n$.
8. **Forgetting that range updates with assignment (not addition) need different lazy handling** — **assignment lazy must also track "no value assigned yet"**.

Practice questions

- **Easy 1:** Range sum query with point updates (segment-tree version; compare with Fenwick — topic 26).
- **Easy 2:** Range **minimum** query with point updates (change the merge to min).
- **Medium 1:** Range **add, range sum** with lazy propagation.
- **Medium 2:** Count of values **in a range** with point updates — combine Fenwick (order statistics) with range queries.
- **Medium 3:** **GCD range queries** — answer gcd of a subarray with updates.
- **Challenging 1:** Range update with assignment + range sum (lazy with a "hasValue" flag).
- **Challenging 2:** **Skyline / rectangle area union** — sweep line + segment tree of covered lengths (hard classic).
- **Challenging 3:** **K-th smallest in a range with updates** — merge sort tree, or

segment tree over values with Fenwick-like counts (wavelet tree territory).

Hints & solutions: file 30-solutions.md.

28 — Advanced Graph Algorithms

What's in this file

The "hard" graph toolkit, in difficulty order:

1. **Topological sort on DAGs** — recap (topic 21) with DP applications
2. **Strongly Connected Components (SCC)** — Kosaraju & Tarjan
3. **Bridges & Articulation Points** — Tarjan's low-link idea
4. **Euler path / circuit**
5. **Bipartite matching** — Kuhn's algorithm
6. **Max flow / min cut** — Ford-Fulkerson / Edmonds-Karp (concept + Dinic mention)

1. SCC — strongly connected components

What: in a *directed* graph, an SCC is a maximal set of nodes where every node can reach every other. Condensing SCCs into single nodes turns any directed graph into a **DAG** of components — unlocking DAG algorithms on general directed graphs.

A→B→C, B→A (A,B form an SCC), C→D, D alone. Condensed: {A,B} → {C} → {D}

Kosaraju's algorithm (two DFS passes):

Pass 1: DFS the original graph, record nodes in FINISH order.

Pass 2: process the REVERSED graph in reverse-finish order — each DFS finds one SCC.

Why it works: the first component popped from the finish-order stack has no outgoing edges to unvisited nodes (in the original), so a DFS on the reversed graph stays inside it.

```
vector<vector<int>> kosaraju(int n, vector<vector<int>>& adj) {
    vector<vector<int>> radj(n);
    for (int u = 0; u < n; ++u)
        for (int v : adj[u]) radj[v].push_back(u);

    vector<int> order; vector<bool> vis(n, false);
    function<void(int)> dfs1 = [&](int u) {
        vis[u] = true;
        for (int v : adj[u]) if (!vis[v]) dfs1(v);
        order.push_back(u); // finish order
    };
    for (int u = 0; u < n; ++u) if (!vis[u]) dfs1(u);

    vector<vector<int>> sccs;
    function<void(int, vector<int>&)> dfs2 = [&](int u, vector<int>& comp) {
        vis[u] = true; comp.push_back(u);
        for (int v : radj[u]) if (!vis[v]) dfs2(v, comp);
    };
    fill(vis.begin(), vis.end(), false);
    for (int i = n - 1; i >= 0; --i) { // reverse finish order
        int u = order[i];
        if (!vis[u]) { vector<int> comp; dfs2(u, comp); sccs.push_back(comp); }
    }
    return sccs;
}
// O(V + E)
```

Step-by-step: edges A→B, B→C, C→A (one SCC), plus C→D.

Pass 1 (finish order): DFS A → ... finish order e.g. [D, C, B, A]

Pass 2 reversed graph, from A: A reaches B reaches C (via reversed edges), D not reachable from A in reversed graph → SCC1 = {A,B,C}. Then D → SCC2 = {D}. ✓

Tarjan's SCC (single DFS with discovery times + low-link) is the same idea, one pass, no reversed graph — the "low" value of a node = earliest ancestor reachable via a back edge.

Classic SCC problem: "Course schedule with cycles" (which courses are impossible because they're in a cycle), or **2-SAT** (boolean formula satisfiability — condense, check x and $\neg x$ never in the same SCC, then topo-order the DAG for assignment).

2. Bridges & articulation points (undirected graphs)

- **Bridge:** an edge whose removal disconnects the graph ("critical connections").
- **Articulation point:** a vertex whose removal disconnects the graph.

Tarjan's low-link idea: during DFS, $\text{tin}[u]$ = discovery time, $\text{low}[u]$ = smallest tin reachable from u 's subtree via at most one back edge.

- Edge $(u \rightarrow v)$ is a **bridge** iff $\text{low}[v] > \text{tin}[u]$ (v 's subtree can't reach above u).
- u is an **articulation point** iff some child v has $\text{low}[v] \geq \text{tin}[u]$ (root: ≥ 2 children).

```
vector<pair<int,int>> findBridges(int n, vector<vector<int>>& adj) {
    vector<int> tin(n, -1), low(n);
    int timer = 0;
    vector<pair<int,int>> bridges;

    function<void(int,int)> dfs = [&](int u, int parent) {
        tin[u] = low[u] = ++timer;
        for (int v : adj[u]) {
            if (v == parent) continue;           // skip the edge we came from
            if (tin[v] != -1) {
                low[u] = min(low[u], tin[v]);    // back edge → ancestor
            } else {
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] > tin[u])              // bridge!
                    bridges.push_back({u, v});
            }
        }
    };
    for (int u = 0; u < n; ++u) if (tin[u] == -1) dfs(u, -1);
    return bridges;
}
// O(V + E). Articulation points: same, condition low[v] >= tin[u] (plus root rule).
```

Step-by-step: edges 0-1, 1-2, 2-0, 2-3

DFS: $\text{tin}[0]=1$, $\text{tin}[1]=2$, $\text{tin}[2]=3$ (via $0 \rightarrow 1 \rightarrow 2$). At 2: neighbor 0 already visited → $\text{low}[2]=\min(3,1)=1$.
 Edge 2-3: $\text{low}[3]=4 > \text{tin}[2]=3 \rightarrow$ BRIDGE (2,3). Back up: $\text{low}[1]=\min(2,1)=1$, $\text{low}[0]=1$.
 Only (2,3) is a bridge — removing it disconnects {3} from {0,1,2}. ✓

3. Euler path & circuit

Euler circuit: a walk using every edge exactly once, ending where it started. **Euler path:** uses every edge once (start $\neq$ end allowed).

Conditions (connected graph, ignore isolated nodes):

- Undirected: circuit $\Leftrightarrow$ all degrees even; path $\Leftrightarrow$ exactly 0 or 2 odd-degree vertices.
- Directed: circuit $\Leftrightarrow$ in-degree = out-degree everywhere; path $\Leftrightarrow$ one node has out = in+1, one has in = out+1, rest equal.

Hierholzer's algorithm: DFS, and when stuck, splice the current circuit — or the one-liner: keep an edge stack, always take an unused edge, and push nodes in *postorder* (pop order reversed is the

circuit). Used for: reconstructing itineraries, drawing a figure in one stroke, de Bruijn sequences.

4. Bipartite matching — Kuhn's algorithm

Problem: maximum set of edges in a bipartite graph with no shared vertices (assign workers to jobs, seats to students, pairs in dating, domino tiling).

```
bool tryKuhn(int u, vector<vector<int>>& adj, vector<int>& matchR, vector<bool>& seen) {
    for (int v : adj[u]) {
        if (seen[v]) continue;
        seen[v] = true;
        if (matchR[v] == -1 || tryKuhn(matchR[v], adj, matchR, seen)) {
            matchR[v] = u;
            return true;
        }
    }
    return false;
}

int maxMatching(int nL, vector<vector<int>>& adj) {    // adj[0..nL-1] → right side
    vector<int> matchR(nR, -1);
    int result = 0;
    for (int u = 0; u < nL; ++u) {
        vector<bool> seen(nR, false);
        if (tryKuhn(u, adj, matchR, seen)) result++;
    }
    return result;
}
// O(V·E) worst case (fast in practice); Hopcroft-Karp improves to O(E·√V).
```

Idea: repeatedly find an **augmenting path** (alternating unmatched/matched edges, starting and ending on unmatched vertices); flipping it grows the matching by 1. When no augmenting path exists, the matching is maximum (Berge's lemma).

5. Max flow / min cut (concept + Edmonds-Karp)

Problem: max units shipped from source s to sink t through capacity-limited edges.

Ford-Fulkerson idea: while an augmenting path exists (BFS/DFS), push flow along it and add

residual (reverse) edges — reverse edges are how the algorithm *undoes* bad routing.

Edmonds-Karp = Ford-Fulkerson with BFS → $O(V \cdot E^2)$. **Dinic's** (level graph + blocking flows) → $O(V^2 \cdot E)$, the contest standard. **Max-flow min-cut theorem:** the maximum flow equals the minimum capacity cut separating s from t — a stunning duality used to solve cut problems (project selection, image segmentation, matching via flow).

```
// Edmonds-Karp — the BFS-based augmenting path search, in essence:
//   capacity[u][v] (residual), flow[u][v].
//   while BFS finds a path s→t with residual capacity > 0:
//       bottleneck = min residual on the path
//       push bottleneck along the path, subtract from forward edges, add to reverse edges
//   answer = total flow pushed.
// O(V·E²) — the BFS always finds the shortest augmenting path, bounding the iterations.
```

Choosing the right advanced tool

Problem	Tool
"Nodes mutually reachable" / 2-SAT / condensation	SCC (Kosaraju/Tarjan)
"Edge whose removal disconnects" / network reliability	Bridges (Tarjan low-link)
"Vertex whose removal disconnects"	Articulation points
"Use every edge once" / one-stroke drawing / itinerary	Euler path (Hierholzer)

Problem	Tool
Maximum pairing / assignment / covering in bipartite	Kuhn / Hopcroft-Karp
Maximum shipment / min cut / project selection	Max flow (Dinic)
Longest path in DAG / critical path	Topo order + DP
Multiple shortest-path queries on a tree	Binary lifting (LCA) — see note below

Bonus — binary lifting (LCA in $O(\log n)$ per query): precompute $up[v][k]$ = the 2^k -th ancestor of v and $depth[v]$. To get the LCA of u and v : raise the deeper one, then raise both together. This is the "advanced graph" trick hiding in plain sight for tree problems (Kth ancestor, path sums).

Common mistakes

1. **Kosaraju:** running pass 2 on the *original* graph (must be reversed) or not reversing the finish order.
2. **Bridges:** $low[v] > tin[u]$ vs articulation's $low[v] \geq tin[u]$ — get the strictness right; the root articulation case (2+ children) is separate.
3. **Bridges/articulations:** not skipping the parent edge properly (use $v == parent$ on the DFS tree — but careful with multi-edges; track edge ids when duplicates exist).
4. **Euler:** checking degrees but not connectivity of edges (the graph must be connected when ignoring isolated vertices).
5. **Kuhn's:** forgetting to reset `seen` per left vertex (or reusing it across attempts).
6. **Flow:** forgetting residual edges — without them the algorithm can't undo and gives suboptimal (or wrong) answers.
7. **Recursion depth** — Tarjan/Kosaraju on 10^5 -node graphs need iterative DFS or raised limits in some judges.

Practice questions

- **Medium 1: Critical connections in a network** — find all bridges.
- **Medium 2: Number of strongly connected components** (Kosaraju).
- **Medium 3: Course schedule IV** — reachability queries in a DAG (topo + bitset DP).
- **Challenging 1: Reconstruct itinerary** — Euler path (Hierholzer) with lexical tie-break.
- **Challenging 2: Minimum number of vertices to reach all nodes** — nodes with in-degree 0 in the *condensed* DAG.
- **Challenging 3: 2-SAT** — satisfiability via SCC + assignment from condensation order.
- **Challenging 4: Maximum bipartite matching** — "assign tasks / place knights / domino tiling".
- **Challenging 5: Max flow** — "network with capacities", "minimum cut to separate s and t ".

Hints & solutions: file 30-solutions.md.

29 — Problem-Solving Patterns (Recognize → Template)

This is the bridge between "I know the algorithms" and "I can solve unseen problems". Each pattern: **when you see it → what to do**. Practice problems mixed across patterns at the end.

Pattern 1 — Prefix sums (precompute once, answer many)

Recognize: many queries over ranges: "sum/min/count of $a[l..r]$ " for many (l, r) , or "subarrays with property" involving sums. **Do:** $\text{pref}[i+1] = \text{pref}[i] + a[i]$; $\text{range} = \text{pref}[r+1] - \text{pref}[l]$. For *counts of subarrays with sum k , add hashing (topic 07). For XOR, products, min: same shape with the right operator.

Template:

```
vector<long long> pref(n + 1, 0);
for (int i = 0; i < n; ++i) pref[i + 1] = pref[i] + a[i];
// query(l, r) = pref[r+1] - pref[l] → O(1)
```

Pattern 2 — Binary search on answer

Recognize: "minimize the maximum X " / "maximize the minimum X " / "smallest K such that..." / " k -th smallest" — and you can *test* feasibility easily. **Do:** binary search the answer value; $\text{feasible}(\text{mid})$ must be monotonic.

Template:

```
long long lo = <lower bound of answer>, hi = <upper bound>;
while (lo < hi) {
    long long mid = lo + (hi - lo) / 2;
    if (feasible(mid)) hi = mid;      // or lo = mid for "largest feasible"
    else lo = mid + 1;               // or hi = mid - 1
}
```

Checklist before submitting: is the predicate monotonic? Are the bounds right (lo can be the max element, hi the sum — not 0 and 10^{18})? long long everywhere.

Pattern 3 — Two pointers (opposite & same direction)

Recognize (opposite): sorted input, find pair/triplet with sum/difference property; "container with most water"; palindrome checks. **Recognize (same):** "do it in place", remove/keep elements, merge two sorted sequences, linked-list middle/cycle. **Do:** opposite → while $(l < r)$ adjust based on comparison; same → slow/fast with an invariant on the slow prefix.

Pattern 4 — Sliding window

Recognize: "longest/shortest subarray/substring satisfying a condition", "at most k ", "all characters of T ", "any window of size k ". Array values non-negative. **Do:** expand r , shrink l with a while, track the metric. Fixed size → add/remove per step. Window max/min → monotonic deque.

Template (variable):

```
int l = 0, best = 0; <metric = 0>;
```

```

for (int r = 0; r < n; ++r) {
    <add a[r] to metric>;
    while (<metric violates constraint>) { <remove a[l]>; ++l; }
    best = max(best, r - l + 1);
}

```

Trap: negative numbers break the monotonic-shrink assumption.

Pattern 5 — Monotonic stack

Recognize: "next greater/smaller element", "nearest greater on the left/right", "largest rectangle in histogram", "daily temperatures", "sum of subarray minimums". **Do:** stack of indices (not values!) with a monotone order; pop when the current element dominates the top.

Template (next greater):

```

stack<int> st;
for (int i = 0; i < n; ++i) {
    while (!st.empty() && a[st.top()] < a[i]) { res[st.top()] = a[i]; st.pop(); }
    st.push(i);
}

```

Think of it as: "each element looks left/right for its boundary" — then use the boundaries to compute contributions.

Pattern 6 — Greedy

Recognize: "maximum number of intervals", "minimum coins/arrows/meetings", sorted-able inputs with a clear local rule; no combination constraint that could backfire. **Do:** sort by the **right** key (usually end time for intervals, ratio for knapsack), then scan. **Prove or test** with a counterexample before committing.

Pattern 7 — Backtracking (enumerate everything, pruned)

Recognize: "all combinations/permutations/partitions/placements", small n ($\leq 12-15$), grid path existence (word search), puzzles (Sudoku, N-Queens). **Do:** choose $\rightarrow$ explore $\rightarrow$ unchoose; prune early (sorted candidates + break, $O(1)$ validity checks, skip duplicates at the same level).

Pattern 8 — BFS / DFS (graphs and grids)

Recognize: reachability, connected components, shortest path (unweighted $\rightarrow$ BFS), level-order, topological order, cycle detection, flood fill, "islands". **Do:** BFS with a queue when distance matters (and multi-source: push all sources first); DFS when exploring/exhausting paths. Grids: 4-direction neighbors + bounds check + in-place visit marking.

Pattern 9 — Dynamic programming

Recognize: "count the number of ways", "maximize/minimize under constraints", "is it possible", "subsequence/substring optimal", decisions at each step that **interact**. **Do:** *define* `dp[state]`, *write base cases, transitions, answer cell*. *Top-down first if the state is confusing; bottom-up for performance*. Ask: **does an earlier choice constrain later ones?** If yes $\rightarrow$ DP (or backtracking at small n).

Fast DP checklist:

- What is the state? (index? capacity? count? remainder? both arrays' indices?)
 - What are the base cases? Fill them.
 - What's the transition, and in what order must the table be filled?
 - What's the answer cell?
 - Can I optimize space (only the last row/2 values)?
-

Pattern 10 — "Design a data structure" (the mix of everything)

Recognize: get/put in $O(1)$ (LRU $\rightarrow$ list+hash), median of a stream (two heaps), min stack (two stacks), autocomplete (trie), range queries with updates (Fenwick/segment), insert/delete/getRandom (vector + hash of indices). **Do:** decompose the required ops, pick a structure per op, combine.

Cross-pattern decision map

Subarray with a property, non-negative array

- └ property about sums/counts, window can shrink monotonically $\rightarrow$ SLIDING WINDOW
- └ count of subarrays with EXACT sum $\rightarrow$ PREFIX SUM + HASH
- └ max/min of windows $\rightarrow$ MONOTONIC DEQUE

Pair/triplet/quad problem

- └ sorted (or sortable) input $\rightarrow$ TWO POINTERS
- └ unsorted, indices matter $\rightarrow$ HASH (complement lookup)

"First/last position", "k-th something", "minimize the max" $\rightarrow$ BINARY SEARCH / BINARY SEARCH ON ANSWER

"Next/previous greater/smaller", histogram $\rightarrow$ MONOTONIC STACK

Graph/grid

- └ shortest steps, level-order $\rightarrow$ BFS
- └ connectivity, paths, cycles, components $\rightarrow$ DFS
- └ weighted $\rightarrow$ DIJKSTRA; negative $\rightarrow$ BELLMAN-FORD; all-pairs $\rightarrow$ FLOYD
- └ DAG ordering / longest path in DAG $\rightarrow$ TOPOLOGICAL + DP

"Count ways / maximize / minimize / possible?" with interacting choices $\rightarrow$ DP

"All configurations / enumerations", small $n \rightarrow$ BACKTRACKING

Dynamic connectivity, merges $\rightarrow$ DSU

Many range queries + point updates $\rightarrow$ FENWICK (prefix) / SEGMENT TREE (ranges)

String prefix/autocomplete $\rightarrow$ TRIE

Singletons via XOR, subsets via masks, flags $\rightarrow$ BIT MANIPULATION

How to attack any problem (the meta-framework)

1. **Understand:** restate the problem. Input size? Constraints? Edge cases (empty, 1 element, duplicates, negatives, overflow)?

2. **Brute force first:** can you solve it in $O(n^2)/O(2^n)$? This defines the baseline and clarifies the structure. (In interviews, say it out loud.)

3. **Pattern-match:** run the decision map above. Which pattern fits the *constraints? $n \leq 10^5$ means $O(n)$ or $O(n \log n)$; $n \leq 20$ means backtracking/bitmask DP.

4. **Simplify:** solve a special case (all positive, sorted, no duplicates) — then lift the restriction one at a time.

5. **Validate:** dry-run on examples; then on edge cases; compute the complexity; write clean code; if something fails, re-read the pattern assumptions (e.g. sliding window with negatives, binary search on unsorted input).

Mixed practice (do these AFTER the per-topic problems)

- **E1:** Two-sum (hash) — then the sorted two-pointer version.
- **E2:** Best time to buy/sell stock (single pass, track min).
- **M1: Product of array except self** (two prefix passes).
- **M2: Find all anagrams in a string** (sliding window + counts).
- **M3: Maximum subarray** (Kadane) vs **maximum product subarray** (keep min & max).
- **M4: Number of islands** (DFS) vs **largest island area** (same + size).
- **M5: Jump game** (greedy) vs **jump game II** (BFS-like greedy on reach).
- **M6: Longest increasing subsequence** (DP → $O(n \log n)$ tails).
- **M7: Graph valid tree** (connected + no cycles — BFS/DFS or DSU).
- **H1: Median of two sorted arrays** (binary search on partition).
- **H2: Word break II** (memoized recursion + substring checks).
- **H3: Longest valid parentheses** (stack, or DP — try both).

Solutions & hints for the per-topic questions: file 30-solutions.md.

30 — Hints & Solutions for All Practice Questions

*Read this only after genuinely trying each problem. The format per topic: - **Hints** (graduated: try H1 before H2, etc.) - **Easy/Medium**: full C++ solutions. - **Challenging**: hints + solution sketches (full code where compact). Revisit these after you've solved the E/M versions — they're meant to stretch you.*

01 — STL

Hints

- H1 (E1): count with `unordered_map`, then scan for max — handle ties with `w < best`.
- H2 (M1): "k most frequent" = frequency map + a **min-heap of size k** (topic 17).
- H3 (C1): LRU = `list<pair<int,int>>` + `unordered_map<key, list::iterator>`. Get moves a

node to the front; Put evicts from the back when full.

E1 — most frequent element

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector<int> a = {1, 3, 2, 3, 1, 3, 1};
    unordered_map<int, int> freq;
    for (int x : a) freq[x]++;
    int best = INT_MAX, bestCnt = 0;
    for (auto& [x, c] : freq)
        if (c > bestCnt || (c == bestCnt && x < best)) { best = x; bestCnt = c; }
    cout << best << "\n"; // 1 (three times, tie-broken smaller)
}
```

E2 — intersection & union of two arrays

```
vector<int> intersection(vector<int>& a, vector<int>& b) {
    unordered_set<int> s(a.begin(), a.end());
    unordered_set<int> seen; vector<int> res;
    for (int x : b) if (s.count(x) && seen.insert(x).second) res.push_back(x);
    return res;
}
// Union: insert all of a and b into an unordered_set → the set itself.
```

M1 — top K frequent

```
vector<int> topKFrequent(vector<int>& a, int k) {
    unordered_map<int, int> freq;
    for (int x : a) freq[x]++;
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq; // (count, val)
    min-heap
    for (auto& [x, c] : freq) {
        pq.push({c, x});
        if ((int)pq.size() > k) pq.pop();
    }
    vector<int> res;
    while (!pq.empty()) { res.push_back(pq.top().second); pq.pop(); }
    return res;
}
// O(n log k)
```

M2 — median of a stream — see topic 17, Pattern 3 (two heaps). Add $O(\log n)$, get $O(1)$.

C1 — LRU cache

```
class LRUCache {
```

```

int cap;
list<pair<int,int>> items; // (key, value), most-recent at front
unordered_map<int, list<pair<int,int>>::iterator> pos;
public:
    LRUCache(int capacity) : cap(capacity) {}
    int get(int key) {
        auto it = pos.find(key);
        if (it == pos.end()) return -1;
        items.splice(items.begin(), items, it->second); // move to front
        return it->second->second;
    }
    void put(int key, int value) {
        auto it = pos.find(key);
        if (it != pos.end()) { // exists → update + refresh
            it->second->second = value;
            items.splice(items.begin(), items, it->second);
            return;
        }
        items.push_front({key, value});
        pos[key] = items.begin();
        if ((int)items.size() > cap) { // evict least-recent (back)
            pos.erase(items.back().first);
            items.pop_back();
        }
    }
};
// get/put O(1) amortized

```

C2 — longest subarray with at most k distinct — H1: sliding window over *distinct count* (topic 09, Pattern 3); H2: shrink while distinct > k; answer = max window length.

02 — Complexity

Hints

- H1: count the iterations precisely, then drop constants.
- H2 (M2): rewrite the double loop as a sum; recall $1 + 1/2 + 1/3 + \dots + 1/n \approx \ln n$.
- H3 (C1): every comparison-based sort is a decision tree with $n!$ possible outputs.

E1: (a) $O(\log n)$. (b) $O(n^2)$ pairs. (c) $O(n^3)$ — there are $O(n^2)$ subarrays, each printed in $O(n)$ total... precisely, printing all subarrays costs $O(n^3)$ work. (d) $O(n \cdot n!)$: $n!$ permutations $\times O(n)$ per next_permutation.

E2: each comparison can eliminate at most one element from "candidate maximum"; after $n-1$ comparisons exactly one candidate survives. Any algorithm with fewer comparisons has an input where the true max was never compared → provably $O(n)$ minimum.

M1: $n^2/2 + 100n = 10^6 \mu s \rightarrow n \approx 1414$. For $10n \cdot \log_2(n) = 10^6 \rightarrow n \approx 10^4$ (check: $10 \cdot 10^4 \cdot 13.3 \approx 1.33 \cdot 10^6$). Lesson: constants matter at small n , but the growth rate wins at scale — an $O(n \log n)$ algorithm with a $10\times$ constant beats $O(n^2)$ for any real n .

M2: inner loop runs n/i times for outer $i \rightarrow \text{total} = n \cdot (1 + 1/2 + \dots + 1/n) = n \cdot H(n) \approx n \cdot \ln n \rightarrow \mathbf{O(n \log n)}$.

C1: a comparison-based sort must distinguish all $n!$ orderings $\rightarrow$ its decision tree has $\geq n!$ leaves $\rightarrow$ height $\geq \log_2(n!) = \Omega(n \log n)$ (Stirling's approximation).

C2: quickselect — pick a random pivot, partition, recurse into the side containing k . Expected work $T(n) = n + T(n/2) \rightarrow \mathbf{O(n)}$ expected, $O(n^2)$ worst case.

03 — Arrays

Hints

- H1 (E1): two pointers — a slow "write" pointer and a fast "scan" pointer.
- H2 (M2): sort + two pointers from both ends, track the closest sum.
- H3 (M3): prefix sums + a hash map of (prefix → count of occurrences so far).
- H4 (C1): a negative flips max↔min — track **both** the max product and min product ending

at each position.

• H5 (C2): water above position $i = \min(\text{maxLeft}[i], \text{maxRight}[i]) - a[i]$ — do it with two pointers carrying running maxes.

E1 — move zeros

```
void moveZeroes(vector<int>& a) {
    int w = 0;
    for (int x : a) if (x != 0) a[w++] = x;    // write non-zeros
    while (w < (int)a.size()) a[w++] = 0;    // pad the tail
}
```

E2 — second largest

```
int secondLargest(vector<int>& a) {
    int first = INT_MIN, second = INT_MIN;
    for (int x : a) {
        if (x > first) { second = first; first = x; }
        else if (x > second && x != first) second = x;
    }
    return second;    // INT_MIN if none
}
```

E3 — max consecutive ones

```
int findMaxConsecutiveOnes(vector<int>& a) {
    int best = 0, cur = 0;
    for (int x : a) { cur = x ? cur + 1 : 0; best = max(best, cur); }
    return best;
}
```

M1 — Kadane + print subarray

```
pair<int, pair<int, int>> maxSubarray(vector<int>& a) {
    int bestEnd = a[0], best = a[0];
    int start = 0, bestL = 0, bestR = 0;    // track boundaries
    for (int i = 1; i < (int)a.size(); ++i) {
        if (bestEnd + a[i] > a[i]) bestEnd += a[i];
        else { bestEnd = a[i]; start = i; }
        if (bestEnd > best) { best = bestEnd; bestL = start; bestR = i; }
    }
    return {best, {bestL, bestR}};
}
```

M2 — closest pair sum

```
pair<int, int> closestSum(vector<int>& a, int target) {
    sort(a.begin(), a.end());
    int l = 0, r = a.size() - 1, bestDiff = INT_MAX;
    pair<int, int> res;
    while (l < r) {
        int s = a[l] + a[r];
        if (abs(s - target) < bestDiff) { bestDiff = abs(s - target); res = {a[l], a[r]}; }
        s < target ? l++ : r--;
    }
    return res;
}
```

```
}
```

M3 — subarray sum equals K

```
int subarraySum(vector<int>& a, int k) {
    unordered_map<int, int> cnt; cnt[0] = 1;
    int pref = 0, ans = 0;
    for (int x : a) { pref += x; ans += cnt[pref - k]; cnt[pref]++; }
    return ans;
}
```

C1 — max product subarray

```
int maxProduct(vector<int>& a) {
    int maxEnd = a[0], minEnd = a[0], best = a[0];
    for (int i = 1; i < (int)a.size(); ++i) {
        if (a[i] < 0) swap(maxEnd, minEnd); // negative flips roles
        maxEnd = max(a[i], maxEnd * a[i]);
        minEnd = min(a[i], minEnd * a[i]);
        best = max(best, maxEnd);
    }
    return best;
}
```

C2 — trapping rain water (two pointers)

```
int trap(vector<int>& h) {
    int l = 0, r = h.size() - 1, ml = 0, mr = 0, water = 0;
    while (l < r) {
        if (h[l] <= h[r]) { ml = max(ml, h[l]); water += ml - h[l]; l++; }
        else { mr = max(mr, h[r]); water += mr - h[r]; r--; }
    }
    return water;
}
```

C3 — longest subarray with sum divisible by k — H1: use prefix sums **mod k**; H2: store the *first index* each remainder appears; answer = max(i - first[rem]).

04 — Strings

Hints

- H1 (E1): two pointers skipping non-alphanumerics; compare lowercased chars.
- H2 (M1): sliding window over "distinct characters" (topic 09).
- H3 (M3): $dp[i]$ = ways to decode prefix of length i ; $dp[i] = dp[i-1] (+ dp[i-2] \text{ if two-digit valid})$.
- H4 (C2): $dp[i][j]$ = does $s[0..i-1]$ match $p[0..j-1]$; * means "match 0 or more" — the trick is $dp[i][j] = dp[i-1][j] || dp[i][j-1]$ when $p[j-1] == ''$.

E1 — valid palindrome (ignore non-alphanumerics)

```
bool isPalindrome(string s) {
    int l = 0, r = s.size() - 1;
    while (l < r) {
        while (l < r && !isalnum(s[l])) l++;
        while (l < r && !isalnum(s[r])) r--;
        if (tolower(s[l]) != tolower(s[r])) return false;
        l++; r--;
    }
    return true;
}
```

E2 — first non-repeating character

```
int firstUniqChar(string s) {
    vector<int> cnt(26, 0);
    for (char c : s) cnt[c - 'a']++;
    for (int i = 0; i < (int)s.size(); ++i)
        if (cnt[s[i] - 'a'] == 1) return i;
    return -1;
}
```

E3 — anagram without sorting

```
bool isAnagram(string a, string b) {
    if (a.size() != b.size()) return false;
    vector<int> cnt(26, 0);
    for (char c : a) cnt[c - 'a']++;
    for (char c : b) if (--cnt[c - 'a'] < 0) return false;
    return true;
}
```

M1 — longest substring without repeating characters

```
int lengthOfLongestSubstring(string s) {
    vector<int> last(256, -1);
    int l = 0, best = 0;
    for (int r = 0; r < (int)s.size(); ++r) {
        l = max(l, last[s[r]] + 1); // jump l past the previous occurrence
        last[s[r]] = r;
        best = max(best, r - l + 1);
    }
    return best;
}
```

M2 — longest palindromic substring — expand around centers (full code in topic 04).

M3 — decode ways

```
int numDecodings(string s) {
    if (s.empty() || s[0] == '0') return 0;
    vector<int> dp(s.size() + 1, 0); dp[0] = 1; dp[1] = 1;
    for (int i = 2; i <= (int)s.size(); ++i) {
        if (s[i-1] != '0') dp[i] += dp[i-1];
        int two = stoi(s.substr(i - 2, 2));
        if (two >= 10 && two <= 26) dp[i] += dp[i-2];
    }
    return dp[s.size()];
}
```

C1 — minimum window substring — H1: sliding window with a "needed count" (topic 09 Pattern 3); H2: keep have = number of distinct required chars satisfied; shrink while have == need, tracking the min window.

C2 — wildcard matching — H1: 2D DP; H2: $dp[i][j]$; when $p[j-1] == '*'$, the two cases are "use the star for $s[i-1]$ " $\rightarrow dp[i-1][j]$ and "empty star" $\rightarrow dp[i][j-1]$.

05 — Searching

Hints

- H1 (E1): lower_bound for first, upper_bound-1 for last.
- H2 (E2): compare $a[mid]$ with $a[0]$ to detect which half is rotated.
- H3 (M2): binary search the *speed; $feasible(h) = \sum \lceil pile/h \rceil \leq h$.
- H4 (C1): binary search the partition of the smaller array; the median condition is an

inequality on the 4 boundary elements.

E1 — first & last position

```
vector<int> searchRange(vector<int>& a, int t) {
    auto lo = lower_bound(a.begin(), a.end(), t);
    auto hi = upper_bound(a.begin(), a.end(), t);
    if (lo == a.end() || *lo != t) return {-1, -1};
    return {(int)(lo - a.begin()), (int)(hi - a.begin()) - 1};
}
```

E2 — search in rotated sorted array

```
int searchRotated(vector<int>& a, int t) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) return mid;
        if (a[lo] <= a[mid]) { // left half sorted
            if (a[lo] <= t && t < a[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else { // right half sorted
            if (a[mid] < t && t <= a[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

M1 — peak element

```
int findPeakElement(vector<int>& a) {
    int lo = 0, hi = a.size() - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] > a[mid + 1]) hi = mid; // descending here → peak is left
        else lo = mid + 1; // ascending → peak is right
    }
    return lo;
}
```

M2 — Koko eating bananas

```
int minEatingSpeed(vector<int>& piles, int h) {
    int lo = 1, hi = *max_element(piles.begin(), piles.end());
    auto ok = [&](int k) {
        long long hours = 0;
        for (int p : piles) hours += (p + k - 1) / k;
        return hours <= h;
    };
    while (lo < hi) { int mid = lo + (hi - lo) / 2; ok(mid) ? hi = mid : lo = mid + 1; }
    return lo;
}
```

M3 — first missing positive

```
int firstMissingPositive(vector<int>& a) {
    int n = a.size();
    for (int i = 0; i < n; ++i)
        while (a[i] >= 1 && a[i] <= n && a[a[i] - 1] != a[i])
            swap(a[i], a[a[i] - 1]); // place each value at its index
    for (int i = 0; i < n; ++i) if (a[i] != i + 1) return i + 1;
    return n + 1;
}
```

C1 — median of two sorted arrays: binary search the split in the smaller array; for a split (i, j), check $\maxLeftA \leq \minRightB$ && $\maxLeftB \leq \minRightA$. Handle parity: $\text{median} = (\maxLeft + \minRight) / 2$ for

even lengths.

C2 — search unknown-length array: exponential search — check indices 1, 2, 4, 8, ... to find a bound where $\text{get}(i) \geq \text{target}$, then binary search inside $[i/2, i]$.

06 — Sorting

Hints

- H1 (E1): three pointers (low, mid, high) — Dutch national flag.
- H2 (M1): merge sort; when taking from the right half, all *remaining* left elements are inversions with it.
- H3 (M2): partition like quicksort, recurse into one side only (quickselect).
- H4 (C1): after sorting, the max gap $\geq (\text{max} - \text{min}) / (n - 1)$ — bucket by that, only compare bucket boundaries.

E1 — sort colors (Dutch flag)

```
void sortColors(vector<int>& a) {
    int lo = 0, mid = 0, hi = a.size() - 1;
    while (mid <= hi) {
        if (a[mid] == 0) swap(a[lo++], a[mid++]);
        else if (a[mid] == 1) mid++;
        else swap(a[mid], a[hi--]);
    }
}
```

E2 — merge sorted arrays in place

```
void mergeInPlace(vector<int>& a, int m, vector<int>& b, int n) {
    int i = m - 1, j = n - 1, k = m + n - 1; // fill from the back
    while (j >= 0)
        a[k--] = (i >= 0 && a[i] > b[j]) ? a[i--] : b[j--];
}
```

M1 — inversion count (merge sort)

```
long long mergeCount(vector<int>& a, int l, int m, int r) {
    vector<int> L(a.begin()+l, a.begin()+m+1), R(a.begin()+m+1, a.begin()+r+1);
    int i = 0, j = 0, k = l; long long inv = 0;
    while (i < (int)L.size() && j < (int)R.size()) {
        if (L[i] <= R[j]) a[k++] = L[i++];
        else { a[k++] = R[j++]; inv += (long long)L.size() - i; } // all remaining L invert
    }
    while (i < (int)L.size()) a[k++] = L[i++];
    while (j < (int)R.size()) a[k++] = R[j++];
    return inv;
}
// In mergeSort: inv = mergeCount(left) + mergeCount(right) + mergeCount(merge).
```

M2 — k-th smallest (quickselect)

```
int quickselect(vector<int>& a, int lo, int hi, int k) {
    int pivot = a[hi], i = lo;
    for (int j = lo; j < hi; ++j) if (a[j] < pivot) swap(a[i++], a[j]);
    swap(a[i], a[hi]);
    if (i == k) return a[i];
    return i > k ? quickselect(a, lo, i - 1, k) : quickselect(a, i + 1, hi, k);
}
// Expected O(n). Randomized pivot avoids the O(n^2) worst case.
```

M3 — largest number

```

string largestNumber(vector<int>& nums) {
    vector<string> s;
    for (int x : nums) s.push_back(to_string(x));
    sort(s.begin(), s.end(), [](const string& a, const string& b) {
        return a + b > b + a; // "9"+"30" vs "30"+"9"
    });
    if (s[0] == "0") return "0";
    string res;
    for (auto& x : s) res += x;
    return res;
}

```

C1 — maximum gap: sort or bucket. With buckets of size $(\max - \min) / (n - 1)$, the answer is a max over consecutive *bucket* differences — $O(n)$ time, $O(n)$ space.

C2 — LIS in $O(n \log n)$: $\text{tails}[i]$ = smallest tail of an increasing subsequence of length $i+1$; for each x , replace the first tail $\geq x$ (lower_bound). Answer = $\text{tails.size}()$.

07 — Hashing

Hints

- H1 (E1): unordered_set; false if insert returns false.
- H2 (M1): prefix sums + $\text{cnt}[\text{pref} - k]$ (see 03-M3); longest variant stores *first index.
- H3 (C1): longest consecutive sequence — only start counting from numbers with no left neighbor.
- H4 (C2): freq map + min-heap of size k (topic 17).

E1 — contains duplicate

```

bool containsDuplicate(vector<int>& a) {
    unordered_set<int> s;
    for (int x : a) if (!s.insert(x).second) return true;
    return false;
}

```

E2 — anagram with one hash pass — same as 04-E3.

E3 — highest frequency — same as 01-E1.

M1 — subarray sum K (+ longest variant) — subarray count: 03-M3. Longest with sum K :

```

int longestSubarraySumK(vector<int>& a, int k) {
    unordered_map<int, int> first; first[0] = -1;
    int pref = 0, best = 0;
    for (int i = 0; i < (int)a.size(); ++i) {
        pref += a[i];
        if (first.count(pref - k)) best = max(best, i - first[pref - k]);
        if (!first.count(pref)) first[pref] = i; // keep EARLIEST occurrence
    }
    return best;
}

```

M2 — group anagrams — 04's solution (sorted key → groups).

M3 — intersection with counts

```

vector<int> intersect(vector<int>& a, vector<int>& b) {
    unordered_map<int, int> cnt;
    for (int x : a) cnt[x]++;
    vector<int> res;
    for (int x : b) if (cnt[x]-- > 0) res.push_back(x);
    return res;
}

```

```
}
```

C1 — longest consecutive sequence

```
int longestConsecutive(vector<int>& a) {
    unordered_set<int> s(a.begin(), a.end());
    int best = 0;
    for (int x : a) {
        if (s.count(x - 1)) continue;          // not a sequence start
        int len = 1;
        while (s.count(x + len)) len++;
        best = max(best, len);
    }
    return best;
}
// O(n) — every element is touched O(1) times across the whole run.
```

C2 — top K frequent — O1-M1 solution.

08 — Two Pointers

Hints

- H1 (M1): sort; fix i, two-pointer the range (i+1, n-1); skip duplicates.
- H2 (M2): always move the pointer at the *shorter* line — it caps the area.
- H3 (C1): 3Sum extended to 4Sum — fix two, two-pointer the rest; skip duplicates at every level.

E1 — two-sum sorted — O3-style (code in topic 08, Pattern 1).

E2 — palindrome — O4-E1.

E3 — intersection of two sorted arrays

```
vector<int> intersectionSorted(vector<int>& a, vector<int>& b) {
    vector<int> res; int i = 0, j = 0;
    while (i < (int)a.size() && j < (int)b.size()) {
        if (a[i] == b[j]) { if (res.empty() || res.back() != a[i]) res.push_back(a[i]); i++;
        j++; }
        else if (a[i] < b[j]) i++;
        else j++;
    }
    return res;
}
```

M1 — 3Sum

```
vector<vector<int>> threeSum(vector<int>& a) {
    sort(a.begin(), a.end());
    vector<vector<int>> res;
    int n = a.size();
    for (int i = 0; i < n - 2; ++i) {
        if (i > 0 && a[i] == a[i - 1]) continue;          // skip duplicates
        int l = i + 1, r = n - 1;
        while (l < r) {
            int s = a[i] + a[l] + a[r];
            if (s == 0) {
                res.push_back({a[i], a[l], a[r]});
                while (l < r && a[l] == a[l + 1]) l++;    // skip dup
                while (l < r && a[r] == a[r - 1]) r--;
                l++; r--;
            } else if (s < 0) l++;
            else r--;
        }
    }
}
```

```

    }
    return res;
}
// O(n2)

```

M2 — container with most water

```

int maxArea(vector<int>& h) {
    int l = 0, r = h.size() - 1, best = 0;
    while (l < r) {
        best = max(best, (r - l) * min(h[l], h[r]));
        if (h[l] < h[r]) l++; else r--; // move the SHORTER side
    }
    return best;
}

```

M3 — trapping rain water (two-pointer) — 03-C2 solution.

C1 — 4Sum: same skeleton as 3Sum with an extra loop; skip duplicates at i and j; early-break/prune with min/max possible sums (optional but good practice).

C2 — flip at most one 0 for max consecutive ones — sliding window (09-E2).

09 — Sliding Window

Hints

- H1 (M3): window + "needed chars" counter; shrink while ALL requirements are met; track the smallest satisfying window.
- H2 (C1): monotonic deque — pop back while $\leq$ new, pop front when expired.
- H3 (C2): count subarrays with $\max \leq X$ (window shrinkable on $a[i] > X$), then subtract count with $\max \leq X-1$.

E1 — max sum of window k

```

int maxSumWindow(vector<int>& a, int k) {
    long long sum = 0, best = LLONG_MIN;
    for (int i = 0; i < (int)a.size(); ++i) {
        sum += a[i];
        if (i >= k) sum -= a[i - k];
        if (i >= k - 1) best = max(best, sum);
    }
    return best;
}

```

E2 — max consecutive ones with ≤ 1 zero

```

int longestOnes(vector<int>& a) { // binary array, at most one 0 allowed
    int l = 0, zeros = 0, best = 0;
    for (int r = 0; r < (int)a.size(); ++r) {
        if (a[r] == 0) zeros++;
        while (zeros > 1) { if (a[l++] == 0) zeros--; }
        best = max(best, r - l + 1);
    }
    return best;
}

```

M1 — longest substring without repeats — 04-M1.

M2 — at most k distinct — topic 09, Pattern 3 code.

M3 — minimum window substring

```

string minWindow(string s, string t) {
    vector<int> need(256, 0), have(256, 0);
    int needCount = 0;
    for (char c : t) if (need[c]++ == 0) needCount++;

    int l = 0, bestL = 0, bestLen = INT_MAX, satisfied = 0;
    for (int r = 0; r < (int)s.size(); ++r) {
        char c = s[r];
        if (need[c]) {
            have[c]++;
            if (have[c] == need[c]) satisfied++;
        }
        while (satisfied == needCount) { // valid → try to shrink
            if (r - l + 1 < bestLen) { bestLen = r - l + 1; bestL = l; }
            char out = s[l++];
            if (need[out]) {
                if (have[out] == need[out]) satisfied--;
                have[out]--;
            }
        }
    }
    return bestLen == INT_MAX ? "" : s.substr(bestL, bestLen);
}

```

C1 — max sliding window — topic 09, Pattern 4 code (deque).

C2 — count subarrays with max exactly X

```

long long countMaxAtMost(vector<int>& a, int X) { // subarrays with all elements ≤ X
    long long ways = 0, run = 0;
    for (int x : a) { run = (x ≤ X) ? run + 1 : 0; ways += run; }
    return ways;
}
long long countMaxExactly(vector<int>& a, int X) {
    return countMaxAtMost(a, X) - countMaxAtMost(a, X - 1);
}
// O(n)

```

10 — Linked Lists

Hints

- H1 (M2): dummy node + prev pointer; "remove all duplicates" = skip whole runs where `cur->val == cur->next->val`.
- H2 (M3): find middle (slow/fast), reverse the second half, interleave.
- H3 (C2): two passes — or single pass with a hash map node→clone, fixing random after.
- H4 (C3): LRU — 01-C1 solution applies directly.

E1 — reverse list — iterative code in topic 10; recursive in topic 13.

E2 — cycle detection + start

```

ListNode* detectCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next; fast = fast->next->next;
        if (slow == fast) break;
    }
    if (!fast || !fast->next) return nullptr; // no cycle
    slow = head; // move slow to head
    while (slow != fast) { slow = slow->next; fast = fast->next; } // meet at cycle start
    return slow;
}

```

E3 — merge two sorted lists — topic 10, Pattern 4.

M1 — remove k-th from end — topic 10, dummy-node solution.

M2 — remove all duplicates (keep none)

```
ListNode* deleteDuplicatesAll(ListNode* head) {
    ListNode dummy(0); dummy.next = head;
    ListNode* prev = &dummy;
    while (prev->next) {
        ListNode* cur = prev->next;
        if (cur->next && cur->val == cur->next->val) {
            int v = cur->val;
            while (prev->next && prev->next->val == v) // skip the whole run
                prev->next = prev->next->next;
        } else {
            prev = prev->next;
        }
    }
    return dummy.next;
}
```

M3 — reorder list

```
void reorderList(ListNode* head) {
    if (!head || !head->next) return;
    ListNode *slow = head, *fast = head; // 1. middle
    while (fast->next && fast->next->next) { slow = slow->next; fast = fast->next->next; }
    ListNode* mid = slow->next; slow->next = nullptr;
    ListNode* prev = nullptr; // 2. reverse second half
    while (mid) { ListNode* nxt = mid->next; mid->next = prev; prev = mid; mid = nxt; }
    ListNode* a = head, *b = prev; // 3. interleave
    while (b) { ListNode *na = a->next, *nb = b->next; a->next = b; b->next = na; a = na; b = nb; }
}
```

C1 — merge k sorted lists — heap over heads (17-M2 with `ListNode*` instead of indices).

C2 — copy list with random pointers

```
Node* copyRandomList(Node* head) {
    if (!head) return nullptr;
    unordered_map<Node*, Node*> m;
    Node* cur = head;
    while (cur) { m[cur] = new Node(cur->val); cur = cur->next; } // clone all
    cur = head;
    while (cur) { m[cur]->next = m[cur->next]; m[cur]->random = m[cur->random]; cur = cur->next; }
    return m[head];
}
// O(n) time/space. O(1)-space version: interleave clones after originals, fix random,
// then un-weave.
```

C3 — LRU cache — 01-C1.

11 — Stacks

Hints

- H1 (E2): store pairs (value, minSoFar), or a second stack of minima.
- H2 (M2): for each bar, find the nearest *smaller* bar on each side (monotonic stack);

area = height × (right − left − 1).

- H3 (M3): stack of surviving asteroids; a negative asteroid fights the top.

- H4 (C2): greedy — two counters: open parens that can still be matched, and unmatched opens that must be matched.

E1 — valid parentheses — topic 11, Pattern 1.

E2 — min stack

```
class MinStack {
    stack<pair<int,int>> st;           // (value, min so far)
public:
    void push(int x) { st.push({x, st.empty() ? x : min(x, st.top().second)}); }
    void pop() { st.pop(); }
    int top() { return st.top().first; }
    int getMin() { return st.top().second; }
};
```

E3 — postfix evaluation — topic 11, Pattern 3.

M1 — next greater — topic 11, Pattern 2.

M2 — largest rectangle in histogram

```
int largestRectangleArea(vector<int>& h) {
    h.push_back(0);                  // sentinel forces final cleanup
    stack<int> st; int best = 0;
    for (int i = 0; i < (int)h.size(); ++i) {
        while (!st.empty() && h[st.top()] >= h[i]) {
            int height = h[st.top()]; st.pop();
            int left = st.empty() ? -1 : st.top(); // nearest smaller on the left
            best = max(best, height * (i - left - 1));
        }
        st.push(i);
    }
    return best;
}
```

M3 — asteroid collision

```
vector<int> asteroidCollision(vector<int>& a) {
    vector<int> st;
    for (int x : a) {
        bool alive = true;
        while (alive && x < 0 && !st.empty() && st.back() > 0) {
            if (st.back() < -x) { st.pop_back(); continue; } // top explodes, keep fighting
            if (st.back() == -x) st.pop_back();              // both explode
            alive = false;
        }
        if (alive) st.push_back(x);
    }
    return st;
}
```

C1 — trapping rain water (stack): maintain a decreasing stack of indices; when a bar is taller than the top, pop and compute the water above that bar: $\text{min}(\text{left}, \text{right}) - h[\text{popped}]$

- $(\text{right} - \text{left} - 1) \cdot$

C2 — valid parentheses with *:

```
bool checkValidString(string s) {
    int lo = 0, hi = 0;               // range of possible open-paren counts
    for (char c : s) {
        if (c == '(') { lo++; hi++; }
        else if (c == ')') { lo = max(lo - 1, 0); hi--; }
        else { lo = max(lo - 1, 0); hi++; } // '*' can be ( ) or empty
        if (hi < 0) return false;
    }
    return lo == 0;
}
```

```
}
```

12 — Queues & Deques

Hints

- H1 (E1): push into the "in" stack; pop = move everything to the "out" stack (amortized $O(1)$).
- H2 (M3): multi-source BFS — push all rotten oranges first; minutes = BFS levels.
- H3 (C3): multi-source BFS from all 0-cells; distance = levels.

E1 — queue with two stacks

```
class MyQueue {
    stack<int> in, out;
public:
    void push(int x) { in.push(x); }
    int pop() {
        if (out.empty()) while (!in.empty()) { out.push(in.top()); in.pop(); }
        int v = out.top(); out.pop(); return v;
    }
    int peek() {
        if (out.empty()) while (!in.empty()) { out.push(in.top()); in.pop(); }
        return out.top();
    }
    bool empty() { return in.empty() && out.empty(); }
};
// Amortized O(1) per operation.
```

E2 — level order (+ zigzag) — topic 12, Pattern 1. Zigzag: reverse level on odd rows.

E3 — first unique char in a stream

```
// maintain queue<char> q and freq map; on each char: push, freq++.
// answer: while (!q.empty() && freq[q.front()] > 1) q.pop();
//          q.empty() ? -1 : q.front()
```

M1 — sliding window max — topic 09, Pattern 4 (deque).

M2 — number of recent calls

```
class RecentCounter {
    queue<int> q;
public:
    int ping(int t) {
        q.push(t);
        while (q.front() < t - 3000) q.pop();
        return q.size();
    }
};
```

M3 — rotting oranges

```
int orangesRotting(vector<vector<int>>& g) {
    int R = g.size(), C = g[0].size(), fresh = 0;
    queue<pair<int,int>> q;
    for (int r = 0; r < R; ++r) for (int c = 0; c < C; ++c) {
        if (g[r][c] == 2) q.push({r, c});
        else if (g[r][c] == 1) fresh++;
    }
    int dr[] = {1,-1,0,0}, dc[] = {0,0,1,-1}, minutes = 0;
    while (!q.empty() && fresh > 0) {
        minutes++;
        int sz = q.size();
        while (sz-- > 0) {
            auto [r, c] = q.front(); q.pop();
```

```

        for (int k = 0; k < 4; ++k) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr >= 0 && nr < R && nc >= 0 && nc < C && g[nr][nc] == 1) {
                g[nr][nc] = 2; fresh--; q.push({nr, nc});
            }
        }
    }
    return fresh == 0 ? minutes : -1;
}

```

C1 — circular deque — implement with a `vector<int>` + head/tail indices + modulo (topic 12's circular-buffer pattern, extended to four ops).

C2 — shortest path in binary matrix — BFS from (0,0); 8 directions; return `dist[target] + 1` (or -1). Standard BFS on a grid.

C3 — zero-one matrix — multi-source BFS from all zero cells; distances = levels.

13 — Recursion

Hints

- H1 (C2): recursion over split points — `s[l..i]` is a palindrome → recurse on `i+1`.
- H2 (C3): `pow(x, n) = pow(x, n/2)2`; handle negative `n` via `1/pow(x, -n)` with long long.

E1 — sum of digits

```
int sumDigits(int n) { return n == 0 ? 0 : n % 10 + sumDigits(n / 10); }
```

E2 — reverse string (indices)

```

void rev(string& s, int l, int r) {
    if (l >= r) return;
    swap(s[l], s[r]);
    rev(s, l + 1, r - 1);
}

```

E3 — climbing stairs

```

int climbStairs(int n) { // = fib(n+1); DP version in topic 22
    if (n <= 2) return n;
    return climbStairs(n - 1) + climbStairs(n - 2); // memoize for n > ~30!
}

```

M1 — recursive merge two sorted lists

```

ListNode* mergeTwo(ListNode* a, ListNode* b) {
    if (!a) return b;
    if (!b) return a;
    if (a->val <= b->val) { a->next = mergeTwo(a->next, b); return a; }
    else { b->next = mergeTwo(a, b->next); return b; }
}

```

M2 — powerset recursively — topic 14, Pattern 1 (include/exclude).

M3 — symmetric tree

```

bool isMirror(TreeNode* a, TreeNode* b) {
    if (!a || !b) return a == b;
    return a->val == b->val && isMirror(a->left, b->right) && isMirror(a->right, b->left);
}
bool isSymmetric(TreeNode* root) { return isMirror(root->left, root->right); }

```

C1 — Tower of Hanoi

```

void hanoi(int n, char from, char to, char aux) {
    if (n == 0) return;
    hanoi(n - 1, from, aux, to);
    cout << "Move disk " << n << " from " << from << " to " << to << "\n";
    hanoi(n - 1, aux, to, from);
}
// Total moves = 2^n - 1

```

C2 — palindrome partitioning

```

vector<vector<string>> partition(string s) {
    vector<vector<string>> res; vector<string> cur;
    function<void(int)> dfs = [&](int l) {
        if (l == (int)s.size()) { res.push_back(cur); return; }
        for (int i = l; i < (int)s.size(); ++i) {
            if (isPalindrome(s.substr(l, i - l + 1))) { // prune: only palindromic cuts
                cur.push_back(s.substr(l, i - l + 1));
                dfs(i + 1);
                cur.pop_back();
            }
        }
    };
    dfs(0);
    return res;
}

```

C3 — fast exponentiation

```

long long powFast(long long x, long long n) { // n ≥ 0
    if (n == 0) return 1;
    long long half = powFast(x, n / 2);
    return (n % 2 == 0) ? half * half : half * half * x;
}
// O(log n). For negative n: 1.0 / powFast(x, -n) with long long n (careful with INT_MIN).

```

14 — Backtracking

Hints

- H1 (E1/E2): duplicates → sort first, skip $a[i] == a[i-1]$ at the same recursion level.
- H2 (M1): sort candidates; in the loop, break once $candidates[i] > target$ (prune);

allow reuse → recurse with the SAME index i .

- H3 (C3): sort descending (big pieces fill fast), skip duplicates, prune $sum > target$ and leftover-space checks.

E1 — subsets with duplicates

```

void gen(vector<int>& a, int i, vector<int>& cur, vector<vector<int>>& res) {
    res.push_back(cur); // every prefix is a subset
    for (int j = i; j < (int)a.size(); ++j) {
        if (j > i && a[j] == a[j - 1]) continue; // skip duplicates at this level
        cur.push_back(a[j]);
        gen(a, j + 1, cur, res);
        cur.pop_back();
    }
}

vector<vector<int>> subsetsWithDup(vector<int>& a) {
    sort(a.begin(), a.end());
    vector<vector<int>> res; vector<int> cur;
    gen(a, 0, cur, res);
    return res;
}

```

E2 — permutations with duplicates

```
void genPerm(vector<int>& a, int pos, vector<vector<int>>& res) {
    if (pos == (int)a.size()) { res.push_back(a); return; }
    unordered_set<int> used; // per-level dedupe
    for (int i = pos; i < (int)a.size(); ++i) {
        if (used.count(a[i])) continue;
        used.insert(a[i]);
        swap(a[pos], a[i]);
        genPerm(a, pos + 1, res);
        swap(a[pos], a[i]);
    }
}
```

M1 — combination sum

```
void combo(vector<int>& cand, int target, int i, vector<int>& cur, vector<vector<int>>& res) {
    if (target == 0) { res.push_back(cur); return; }
    for (int j = i; j < (int)cand.size(); ++j) {
        if (cand[j] > target) break; // sorted → prune
        cur.push_back(cand[j]);
        combo(cand, target - cand[j], j, cur, res); // j (not j+1) → reuse allowed
        cur.pop_back();
    }
}

vector<vector<int>> combinationSum(vector<int>& cand, int target) {
    sort(cand.begin(), cand.end());
    vector<vector<int>> res; vector<int> cur;
    combo(cand, target, 0, cur, res);
    return res;
}
```

M2 — letter combinations of a phone number

```
vector<string> letterCombinations(string digits) {
    if (digits.empty()) return {};
    const vector<string> mp = {"", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv",
"wxzyz"};
    vector<string> res; string cur;
    function<void(int)> dfs = [&](int i) {
        if (i == (int)digits.size()) { res.push_back(cur); return; }
        for (char c : mp[digits[i] - '0']) { cur.push_back(c); dfs(i + 1); cur.pop_back(); }
    };
    dfs(0);
    return res;
}
```

M3 — N-Queens — topic 14, Pattern 3.

M4 — word search — topic 14, Pattern 4.

C1 — Sudoku solver: for each empty cell, try digits 1-9 with O(1) row/col/box checks (row[r][d], col[c][d], box[(r/3)*3 + c/3][d]); backtrack; first completed board is the answer.

C2 — palindrome partitioning — 13-C2.

C3 — partition to k equal subsets:

```
bool canPartitionKSubsets(vector<int>& a, int k) {
    int sum = accumulate(a.begin(), a.end(), 0);
    if (sum % k) return false;
    int target = sum / k;
    sort(a.rbegin(), a.rend()); // big first — prune faster
    if (a[0] > target) return false;
    vector<int> buckets(k, 0);
    function<bool(int)> dfs = [&](int i) {
        if (i == (int)a.size()) return true;
        for (int b = 0; b < k; ++b) {
```

```

        if (buckets[b] + a[i] > target) continue;
        if (b > 0 && buckets[b] == buckets[b - 1]) continue;    // skip identical states
        buckets[b] += a[i];
        if (dfs(i + 1)) return true;
        buckets[b] -= a[i];
        if (buckets[b] == 0) break;                            // pruning: don't leave a bucket empty
    }
    return false;
};
return dfs(0);
}

```

15 — Trees

Hints

- H1 (E2): subtree = for each node of big tree, isSameTree — $O(n \cdot m)$; optimize with hash or serialization.
- H2 (M4): height-recursion returns height, carries a "balanced?" flag (or return -1 on imbalance).
- H3 (C2): serialize with null markers ("#"), deserialize by consuming a stream.
- H4 (C3): like diameter, but track max path *sum* with negatives — return the best one-sided branch, update answer with the two-sided path.

E1 — max depth — topic 15, Pattern 1.

E2 — same tree / subtree — isSameTree in topic 15; subtree: any node of s where isSameTree(node, t).

E3 — inorder (recursive + iterative)

```

vector<int> inorderIter(TreeNode* root) {
    vector<int> res; stack<TreeNode*> st; TreeNode* cur = root;
    while (cur || !st.empty()) {
        while (cur) { st.push(cur); cur = cur->left; }    // go left as far as possible
        cur = st.top(); st.pop();
        res.push_back(cur->val);
        cur = cur->right;
    }
    return res;
}

```

M1 — diameter — topic 15, Pattern 2.

M2 — level order — topic 12/15.

M3 — LCA — topic 15, Pattern 3.

M4 — balanced tree

```

bool isBalanced(TreeNode* root) {
    function<int(TreeNode*)> height = [&](TreeNode* n) -> int {
        if (!n) return 0;
        int L = height(n->left), R = height(n->right);
        if (L == -1 || R == -1 || abs(L - R) > 1) return -1;    // imbalance marker
        return 1 + max(L, R);
    };
    return height(root) != -1;
}

```

C1 — build from preorder + inorder

```

TreeNode* build(vector<int>& pre, vector<int>& in, int& pIdx, int l, int r,
                unordered_map<int,int>& inPos) {
    if (l > r) return nullptr;
    int val = pre[pIdx++];
    TreeNode* node = new TreeNode(val);
    int m = inPos[val];
    node->left = build(pre, in, pIdx, l, m - 1, inPos);
    node->right = build(pre, in, pIdx, m + 1, r, inPos);
    return node;
}
// O(n) with the hash map for O(1) index lookup.

```

C2 — serialize / deserialize

```

void ser(TreeNode* n, ostream& os) {
    if (!n) { os << "# "; return; }
    os << n->val << " ";
    ser(n->left, os); ser(n->right, os);
}
TreeNode* deser(istream& is) {
    string t; is >> t;
    if (t == "#") return nullptr;
    TreeNode* n = new TreeNode(stoi(t));
    n->left = deser(is); n->right = deser(is);
    return n;
}

```

C3 — maximum path sum

```

int maxPathSum(TreeNode* root) {
    int best = INT_MIN;
    function<int(TreeNode*)> dfs = [&](TreeNode* n) -> int {
        if (!n) return 0;
        int L = max(0, dfs(n->left)); // negative branches contribute nothing
        int R = max(0, dfs(n->right));
        best = max(best, L + R + n->val); // path through this node
        return n->val + max(L, R); // best one-sided branch upward
    };
    dfs(root);
    return best;
}

```

16 — BST

Hints

- H1 (M2): three cases — leaf, one child, two children (inorder successor).
- H2 (C2): inorder traversal wiring `prev->right = cur`; `cur->left = prev`.
- H3 (C3): inorder gives (almost) sorted order; find the two positions where order breaks.

E1 — search (iterative)

```

TreeNode* searchBST(TreeNode* root, int key) {
    while (root && root->val != key) root = key < root->val ? root->left : root->right;
    return root;
}

```

E2 — insert — topic 16.

E3 — min/max

```

int minVal(TreeNode* root) { while (root->left) root = root->left; return root->val; }
int maxVal(TreeNode* root) { while (root->right) root = root->right; return root->val; }

```

M1 — validate — topic 16 (range method).

M2 — delete — topic 16.

M3 — k-th smallest — topic 16, Pattern.

M4 — sorted array → balanced BST

```
TreeNode* sortedToBST(vector<int>& a, int l, int r) {
    if (l > r) return nullptr;
    int m = (l + r) / 2;
    TreeNode* n = new TreeNode(a[m]);
    n->left = sortedToBST(a, l, m - 1);
    n->right = sortedToBST(a, m + 1, r);
    return n;
}
```

C1 — LCA in BST

```
TreeNode* lcaBST(TreeNode* root, TreeNode* p, TreeNode* q) {
    while (root) {
        if (p->val < root->val && q->val < root->val) root = root->left;
        else if (p->val > root->val && q->val > root->val) root = root->right;
        else return root; // split point (or one is the root)
    }
    return nullptr;
}
```

C2 — BST → sorted doubly linked list

```
TreeNode* treeToDLL(TreeNode* root) {
    TreeNode* head = nullptr, *prev = nullptr;
    function<void(TreeNode*)> go = [&](TreeNode* n) {
        if (!n) return;
        go(n->left);
        if (!prev) head = n; else { prev->right = n; n->left = prev; }
        prev = n;
        go(n->right);
    };
    go(root);
    return head;
}
```

C3 — recover swapped BST: inorder; the first "break" gives the first wrong node, the last "break" gives the second; swap their values.

17 — Heaps

Hints

- H1 (C3): math version — $(\text{maxFreq} - 1) * (n + 1) + \text{countOfMax}$ VS `tasks.size()`; take max.
- H2 (C4): two heaps + lazy deletion — delete by marking a "dead" map, skip when popping.

E1 — k-th largest in a stream

```
class KthLargest {
public:
    KthLargest(int k, vector<int>& a) : k(k) {
        for (int x : a) add(x);
    }
    int add(int x) {
        pq.push(x);
        if ((int)pq.size() > k) pq.pop();
        return pq.top();
    }
private:
    priority_queue<int, vector<int>, greater<int>> pq; // min-heap of size k
    int k;
}
```

```
};
```

E2 — k largest elements — topic 17, Pattern 1.

M1 — top K frequent — 01-M1.

M2 — merge k sorted lists — heap over heads (list version of 17-M2).

M3 — k closest points

```
vector<vector<int>> kClosest(vector<vector<int>>& pts, int k) {
    priority_queue<pair<long long, int>> pq;           // max-heap on distance
    for (int i = 0; i < (int)pts.size(); ++i) {
        long long d = 1LL * pts[i][0] * pts[i][0] + 1LL * pts[i][1] * pts[i][1];
        pq.push({d, i});
        if ((int)pq.size() > k) pq.pop();           // evict the farthest
    }
    vector<vector<int>> res;
    while (!pq.empty()) { res.push_back(pts[pq.top().second]); pq.pop(); }
    return res;
}
```

C1 — median of stream — topic 17, Pattern 3.

C2 — top k frequent words: freq map; heap with comparator: fewer count → "worse" (on top of a min-heap); ties → lexicographically larger is worse (so smaller pops last).

C3 — task scheduler

```
int leastInterval(vector<char>& tasks, int n) {
    vector<int> cnt(26, 0);
    for (char c : tasks) cnt[c - 'A']++;
    int mx = *max_element(cnt.begin(), cnt.end());
    int mxCount = count(cnt.begin(), cnt.end(), mx);
    return max((int)tasks.size(), (mx - 1) * (n + 1) + mxCount);
}
// Intuition: the most frequent task forces (mx-1) full cycles of n+1 slots, plus the
// mxCount tasks sharing the final row. If other tasks fill the gaps, total = tasks.size().
```

C4 — sliding window median: two heaps + a deleted frequency map; addNum/removeNum both $O(\log n)$ with lazy deletion: when the top of a heap is marked dead, pop it.

18 — Greedy

Hints

- H1 (E1): keep maxReach; fail when $i > \text{maxReach}$.
- H2 (M2): two passes — left→right then right→left, element-wise max.
- H3 (M3): $\text{total} < 0 \rightarrow$ impossible; else start after the running minimum.
- H4 (C3): sort by height desc, then k asc; insert at index k.

E1 — jump game

```
bool canJump(vector<int>& a) {
    int reach = 0;
    for (int i = 0; i < (int)a.size(); ++i) {
        if (i > reach) return false;
        reach = max(reach, i + a[i]);
    }
    return true;
}
```

E2 — assign cookies

```
int findContentChildren(vector<int>& g, vector<int>& s) {
    sort(g.begin(), g.end()); sort(s.begin(), s.end());
    int i = 0;
    for (int c : s) if (i < (int)g.size() && c >= g[i]) i++;
    return i;
}
```

M1 — max non-overlapping intervals — topic 18, Pattern 1. "Min to remove" = $n - \text{answer}$.

M2 — candy

```
int candy(vector<int>& r) {
    int n = r.size();
    vector<int> c(n, 1);
    for (int i = 1; i < n; ++i) if (r[i] > r[i-1]) c[i] = c[i-1] + 1; // left pass
    for (int i = n - 2; i >= 0; --i) if (r[i] > r[i+1]) c[i] = max(c[i], c[i+1] + 1);
    return accumulate(c.begin(), c.end(), 0);
}
```

M3 — gas station

```
int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
    int total = 0, cur = 0, start = 0;
    for (int i = 0; i < (int)gas.size(); ++i) {
        int diff = gas[i] - cost[i];
        total += diff; cur += diff;
        if (cur < 0) { start = i + 1; cur = 0; } // restart from the next station
    }
    return total < 0 ? -1 : start;
}
```

M4 — burst balloons with min arrows

```
int findMinArrowShots(vector<vector<int>>& points) {
    sort(points.begin(), points.end(), [](auto& a, auto& b) { return a[1] < b[1]; });
    int arrows = 1, end = points[0][1];
    for (auto& p : points)
        if (p[0] > end) { arrows++; end = p[1]; } // new balloon starts past current arrow
    return arrows;
}
```

C1 — Huffman coding — topic 18, Pattern 3.

C2 — task scheduler — 17-C3.

C3 — queue reconstruction by height

```
vector<vector<int>> reconstructQueue(vector<vector<int>>& people) {
    sort(people.begin(), people.end(), [](auto& a, auto& b) {
        return a[0] != b[0] ? a[0] > b[0] : a[1] < b[1]; // taller first, then fewer in front
    });
    vector<vector<int>> res;
    for (auto& p : people) res.insert(res.begin() + p[1], p); // insert at position k
    return res;
}
// Inserting shorter people first never disturbs taller people's k.  $O(n^2)$  worst (insert).
```

19 — Graphs

Hints

- H1 (M2): BFS/DFS copying with `unordered_map<Node*, Node*>`; copy on first visit.
- H2 (M3): directed cycle detection — Kahn (emit $< n \rightarrow \text{cycle}$) or three-color DFS.
- H3 (C1): BFS over words — neighbors = words differing by one char; build a graph

lazily, or mutate each position over 'a'..'z' and look up in a set.

- H4 (C2): reverse the flow — BFS from both oceans' borders *upward.

E1 — number of islands — topic 19, grid DFS.

E2 — path exists — BFS/DFS from source, return whether target was visited.

E3 — max depth of n-ary tree

```
int maxDepth(Node* root) {
    if (!root) return 0;
    int d = 0;
    for (Node* c : root->children) d = max(d, maxDepth(c));
    return 1 + d;
}
```

M1 — rotting oranges — 12-M3.

M2 — clone graph

```
Node* cloneGraph(Node* node) {
    if (!node) return nullptr;
    unordered_map<Node*, Node*> m;
    function<Node*(Node*)> dfs = [&](Node* u) {
        if (m.count(u)) return m[u];
        Node* copy = new Node(u->val);
        m[u] = copy;
        for (Node* v : u->neighbors) copy->neighbors.push_back(dfs(v));
        return copy;
    };
    return dfs(node);
}
```

M3 — course schedule

```
bool canFinish(int n, vector<vector<int>>& prereq) {
    vector<vector<int>> adj(n);
    vector<int> indeg(n, 0);
    for (auto& p : prereq) { adj[p[1]].push_back(p[0]); indeg[p[0]]++; }
    queue<int> q;
    for (int i = 0; i < n; ++i) if (indeg[i] == 0) q.push(i);
    int done = 0;
    while (!q.empty()) { int u = q.front(); q.pop(); done++;
        for (int v : adj[u]) if (--indeg[v] == 0) q.push(v); }
    return done == n; // cycle → can't finish
}
```

M4 — shortest path in binary matrix — BFS with 8 directions from (0,0); -1 if the start/end is blocked or unreachable.

C1 — word ladder — BFS; generate neighbors by replacing one char; if neighbor ∈ word set and unvisited, enqueue with distance+1; return distance when reaching endWord.

C2 — Pacific Atlantic water flow: reverse BFS from each ocean's border cells (move to cells with height ≥ current); a cell belongs to both if both BFSs reach it.

C3 — accounts merge — DSU (25-M1): map email→owner index; union on shared emails; group by root; sort each group.

20 — Shortest Paths

Hints

- H1 (M2): Bellman-Ford but stop after k+1 iterations (k stops = k+1 edges); careful: use

a snapshot of `dist` per iteration so one leg doesn't chain within the same round.

- H2 (M3): "maximize product of probabilities" = maximize sum of log-probabilities → Dijkstra with a max-heap (or negate logs for min-heap).
- H3 (C1): "minimize the maximum edge on the path" — Dijkstra where the heap key is the max edge so far.

E1 — BFS min steps in grid — standard grid BFS (12-C2 pattern), count levels.

M1 — network delay time

```
int networkDelayTime(vector<vector<int>>& times, int n, int k) {
    vector<vector<pair<int,int>>> adj(n + 1);
    for (auto& t : times) adj[t[0]].push_back({t[1], t[2]});
    const long long INF = 4e18;
    vector<long long> dist(n + 1, INF); dist[k] = 0;
    priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;
    pq.push({0, k});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : adj[u]) if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w; pq.push({dist[v], v});
        }
    }
    long long mx = 0;
    for (int i = 1; i <= n; ++i) mx = max(mx, dist[i]);
    return mx >= INF ? -1 : (int)mx;
}
```

M2 — cheapest flights within k stops

```
int findCheapestPrice(int n, vector<vector<int>>& flights, int src, int dst, int k) {
    vector<long long> dist(n, 4e18); dist[src] = 0;
    for (int i = 0; i <= k; ++i) {
        vector<long long> nxt = dist; // snapshot per round!
        for (auto& f : flights)
            if (dist[f[0]] + f[2] < nxt[f[1]])
                nxt[f[1]] = dist[f[0]] + f[2];
        dist = nxt;
    }
    return dist[dst] >= 4e18 ? -1 : (int)dist[dst];
}
```

M3 — path with maximum probability — max-heap Dijkstra over probabilities; `prob[v] = max(prob[v], prob[u] * w)`.

C1 — swim in rising water — Dijkstra with "max edge so far" as the key: `dist[v] = min(dist[v], max(dist[u], grid[v]))`.

C2 — word ladder II — BFS to record distances, then DFS/backtracking along the distance-decreasing neighbors to enumerate all shortest paths.

C3 — grid with k removals — BFS over state `(r, c, kLeft)`; `visited[][][]` 3D.

C4 — city with fewest reachable neighbors — Floyd-Warshall then count per city.

21 — MST & Topo

Hints

- H1 (M2): process in topological order; `dist[v] = max(dist[v], dist[u] + w)`.

- H2 (C1): bridges — topic 28 (Tarjan low-link).
- H3 (C2): DSU on swappable positions; sort each component's characters.

E1 — topo order — topic 21, Kahn's.

E2 — directed cycle detection — Kahn (21): if emitted $< n \rightarrow$ cycle. Or three-color DFS.

M1 — course schedule I/II — 19-M3 (and return order instead of just done == n).

M2 — longest path in a DAG

```
int longestPathDAG(int n, vector<vector<pair<int,int>>>& adj) {
    // topo order via Kahn (topic 21)...
    vector<int> dist(n, 0);
    for (int u : topo)                // process in topo order
        for (auto [v, w] : adj[u])
            dist[v] = max(dist[v], dist[u] + w);
    return *max_element(dist.begin(), dist.end());
}
```

M3 — min cost to connect cities — Kruskal (21): sort edges, DSU; return total or -1 if MST has $< n-1$ edges.

C1 — critical connections — bridges code in topic 28.

C2 — smallest string with swaps

```
string smallestStringWithSwaps(string s, vector<vector<int>>& pairs) {
    int n = s.size();
    DSU dsu(n);                // 25-DSU
    for (auto& p : pairs) dsu.unite(p[0], p[1]);
    unordered_map<int, vector<int>> groups;
    for (int i = 0; i < n; ++i) groups[dsu.find(i)].push_back(i);
    for (auto& [root, idxs] : groups) {
        string chars;
        for (int i : idxs) chars += s[i];
        sort(chars.begin(), chars.end());
        sort(idxs.begin(), idxs.end());
        for (int j = 0; j < (int)idxs.size(); ++j) s[idxs[j]] = chars[j];
    }
    return s;
}
```

C3 — second-best MST: for each edge e of the MST, find the MST of the graph without e — take the minimum total. $O(E \cdot (\text{sort} + \text{DSU}))$ — acceptable for small graphs; mention that more advanced approaches (lowest non-tree edge per tree path) exist.

22 — DP

Hints

- H1 (E3): $dp[i] = \text{cost}[i] + \min(dp[i-1], dp[i-2])$, answer $\min(dp[n-1], dp[n-2])$.
- H2 (M2): $O(n^2)$ $dp[i] = 1 + \max dp[j]$; $O(n \log n)$ via tails + lower_bound.
- H3 (M5): $dp[i][j]$ — delete/insert/replace transitions, base row/column = lengths.
- H4 (C1): $dp[i]$ = can prefix $s[0..i-1]$ be segmented; check all dictionary words ending at i (or dp on word lengths).
- H5 (C3): $dp[k][i]$ = max profit with at most k transactions ending by day i .

E1 — climbing stairs — 13-E3 with memo/DP (bottom-up in topic 22).

E2 — house robber — topic 22, Pattern 1.

E3 — min cost climbing stairs

```
int minCostClimbingStairs(vector<int>& cost) {
    int n = cost.size();
    vector<int> dp(n + 1, 0);           // dp[i] = min cost to REACH step i
    for (int i = 2; i <= n; ++i)
        dp[i] = min(dp[i-1] + cost[i-1], dp[i-2] + cost[i-2]);
    return dp[n];
}
```

M1 — knapsack + chosen items — topic 22, Pattern 3 (keep the 2D table and walk it back to recover the items).

M2 — LIS (both versions)

```
// O(n2):
int lis(vector<int>& a) {
    vector<int> dp(a.size(), 1); int best = 1;
    for (int i = 0; i < (int)a.size(); ++i)
        for (int j = 0; j < i; ++j)
            if (a[j] < a[i]) dp[i] = max(dp[i], dp[j] + 1), best = max(best, dp[i]);
    return best;
}
// O(n log n): tails + lower_bound (06-C2 sketch).
```

M3 — coin change (min & ways)

```
int coinChange(vector<int>& coins, int amount) {           // min coins — topic 22, Pattern 4 }
int change(int amount, vector<int>& coins) {              // number of ways
    vector<long long> dp(amount + 1, 0); dp[0] = 1;
    for (int c : coins)                                  // coin OUTER loop → combos (not perms)
        for (int a = c; a <= amount; ++a)
            dp[a] += dp[a - c];
    return (int)dp[amount];
}
```

M4 — LCS

```
int lcs(string a, string b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j)
            dp[i][j] = (a[i-1] == b[j-1]) ? dp[i-1][j-1] + 1
                                           : max(dp[i-1][j], dp[i][j-1]);
    return dp[n][m];
}
```

M5 — edit distance

```
int minDistance(string a, string b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 0; i <= n; ++i) dp[i][0] = i;           // delete all
    for (int j = 0; j <= m; ++j) dp[0][j] = j;           // insert all
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j)
            if (a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1];
            else dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]}); // del, ins, rep
    return dp[n][m];
}
```

C1 — word break

```
bool wordBreak(string s, vector<string>& dict) {
    unordered_set<string> d(dict.begin(), dict.end());
    vector<bool> dp(s.size() + 1, false); dp[0] = true;
    for (int i = 1; i <= (int)s.size(); ++i)
```

```

        for (int j = 0; j < i; ++j)
            if (dp[j] && d.count(s.substr(j, i - j))) { dp[i] = true; break; }
    return dp[s.size()];
}

```

C2 — partition equal subset sum

```

bool canPartition(vector<int>& a) {
    int sum = accumulate(a.begin(), a.end(), 0);
    if (sum & 1) return false;
    int target = sum / 2;
    vector<bool> dp(target + 1, false); dp[0] = true;
    for (int x : a)
        for (int s = target; s >= x; --s) // backwards → 0/1
            dp[s] = dp[s] || dp[s - x];
    return dp[target];
}

```

C3 — stock with k transactions — $dp[k][i]$; optimize the $\max(dp[k-1][j] - price[j])$ term incrementally for $O(n \cdot k)$.

C4 — regex matching (. and *)

```

bool isMatch(string s, string p) {
    int n = s.size(), m = p.size();
    vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));
    dp[0][0] = true;
    for (int j = 2; j <= m; ++j) if (p[j-1] == '*') dp[0][j] = dp[0][j-2]; // empty stars
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j) {
            if (p[j-1] == '*')
                dp[i][j] = dp[i][j-2] || // star matches
                    ((p[j-2] == '.' || p[j-2] == s[i-1]) && dp[i-1][j]); // or one more
            else
                dp[i][j] = (p[j-1] == '.' || p[j-1] == s[i-1]) && dp[i-1][j-1];
        }
    return dp[n][m];
}

```

23 — Tries

Hints

- H1 (E2): sort words, compare first & last — or walk the trie while it has one child.
- H2 (M3): insert every suffix; count nodes created (each node = one distinct substring).
- H3 (C3): counts in nodes; walk until a node's count == 1.

E1 — trie implementation — topic 23.

E2 — longest common prefix — sort + compare first/last, or trie single-child walk.

M1 — replace words — walk the trie; stop at the first node with `isWord`; return the prefix found (or the original word if none).

M2 — word search II

```

// Build a trie of the dictionary. For each grid cell, DFS with backtracking;
// prune: stop when the current node has no child for the next char OR the node is a
// word end (record it). The trie makes the search  $O(4^L)$  per word instead of  $O(\text{words} \cdot 4^L)$ .

```

M3 — count distinct substrings

```

int countDistinctSubstrings(string s) {
    int cnt = 0;

```

```

struct Node { unordered_map<char, Node*> next; };
Node* root = new Node();
for (int i = 0; i < (int)s.size(); ++i) {      // insert each suffix
    Node* cur = root;
    for (int j = i; j < (int)s.size(); ++j) {
        if (!cur->next.count(s[j])) { cur->next[s[j]] = new Node(); cnt++; }
        cur = cur->next[s[j]];
    }
}
return cnt;                                     // every trie node = a distinct substring
}

```

C1 — max XOR of two numbers — topic 23, Pattern 4.

C2 — autocomplete system — trie with count per node + a min-heap of size 3 at query time (or store top-3 per node).

C3 — shortest unique prefix — insert with counts; for each word, walk and stop at the first node where count == 1 (that prefix is unique to this word).

24 — Bit Manipulation

Hints

- H1 (M2): XOR all of 0..n with the array — the missing number remains. Or sum-difference.
- H2 (C1): count each bit across all numbers mod 3.
- H3 (C4): sum = a ^ b; carry = (a & b) << 1; repeat until carry == 0.

E1 — single number — topic 24, Pattern 1.

E2 — popcount

```

int hammingWeight(int n) {
    int cnt = 0;
    while (n) { n &= (n - 1); cnt++; }      // clears lowest set bit each time
    return cnt;
}

```

E3 — power of two — topic 24, Pattern 4.

M1 — reverse bits

```

uint32_t reverseBits(uint32_t n) {
    uint32_t res = 0;
    for (int i = 0; i < 32; ++i) { res = (res << 1) | (n & 1); n >>= 1; }
    return res;
}

```

M2 — missing number

```

int missingNumber(vector<int>& a) {
    int x = a.size();
    for (int i = 0; i < (int)a.size(); ++i) x ^= i ^ a[i];
    return x;
}

```

M3 — count bits 0..n — topic 24, Pattern 2.

C1 — single number II

```

int singleNumberII(vector<int>& a) {
    int ones = 0, twos = 0;
    for (int x : a) {
        ones = (ones ^ x) & ~twos;      // bits that appeared 1 mod 3
        twos = (twos ^ x) & ~ones;      // bits that appeared 2 mod 3
    }
}

```

```

    }
    return ones;
}

```

C2 — max XOR — 23-C1 (bit trie).

C3 — subsets by mask — topic 24, Pattern 3.

C4 — sum without + / -

```

int getSum(int a, int b) {
    while (b) {
        int carry = (unsigned)(a & b) << 1;    // carry bits (unsigned shift!)
        a ^= b;                                // sum without carry
        b = carry;
    }
    return a;
}

```

25 — DSU

Hints

- H1 (M2): each add unions with up to 4 existing land neighbors; decrement count per successful merge.
- H2 (C1): scan edges in order; the first edge whose endpoints are already connected is the redundant one.
- H3 (C3): map each cell to 4 sub-triangle ids; union adjacent ones (including across cell boundaries); answer = number of distinct sets – 1 (the outside region).

E1 — number of provinces

```

int findCircleNum(vector<vector<int>>& isConnected) {
    int n = isConnected.size();
    DSU dsu(n);
    int comps = n;
    for (int i = 0; i < n; ++i)
        for (int j = i + 1; j < n; ++j)
            if (isConnected[i][j] && dsu.find(i) != dsu.find(j)) { dsu.unite(i, j); comps--; }
    return comps;
}

```

E2 — cycle detection (undirected)

```

bool hasCycle(int n, vector<pair<int,int>>& edges) {
    DSU dsu(n);
    for (auto& [u, v] : edges) {
        if (dsu.find(u) == dsu.find(v)) return true;    // already connected → cycle
        dsu.unite(u, v);
    }
    return false;
}

```

M1 — accounts merge — map email→id, union on shared email, group by root, sort each group's emails (name first).

M2 — number of islands II

```

vector<int> numIslands2(int m, int n, vector<vector<int>>& pos) {
    DSU dsu(m * n);
    vector<bool> land(m * n, false);
    int count = 0;
}

```

```

vector<int> res;
int dr[] = {1,-1,0,0}, dc[] = {0,0,1,-1};
for (auto& p : pos) {
    int r = p[0], c = p[1], id = r * n + c;
    if (!land[id]) { land[id] = true; count++; }
    for (int k = 0; k < 4; ++k) {
        int nr = r + dr[k], nc = c + dc[k];
        if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
        int nid = nr * n + nc;
        if (land[nid] && dsu.find(id) != dsu.find(nid)) { dsu.unite(id, nid); count--; }
    }
    res.push_back(count);
}
return res;
}

```

M3 — smallest string with swaps — 21-C2.

C1 — redundant connection — E2 loop; return the first edge where find already equal.

C2 — longest consecutive sequence via DSU: map value→index; for each x, union with x−1 and x+1 if present; track max size. (The set-based O(n) solution is simpler — do both.)

C3 — regions cut by slashes — H3 above; each cell gets ids 0..3 for its 4 triangles; union within the cell (according to the slash), and union across neighboring cells' touching triangles.

26 — Fenwick

Hints

- H1 (C1): scan right→left; `sum(id-1)` counts smaller values already seen on the right.
- H2 (C2): coordinate-compress `2·a[j]` too (or use merge sort); `count pref - sum(id-1)`.
- H3 (C3): Fenwick over counts + binary lifting (kth).

E1 — range sum with point updates

```

// Fenwick from topic 26: add(i, delta); sum(i); rangeSum(l, r) = sum(r) - sum(l-1).
// Point update + range query in O(log n) each.

```

M1 — inversion count — topic 26, full solution.

M2 — range add, point query — difference BIT: `add(l, x); add(r+1, -x); value at i = sum(i)`.

M3 — range add, range sum — two BITs: `B1.add(l, x); B1.add(r+1, -x); B2.add(l, x*(l-1)); B2.add(r+1, -x*r); prefix(r) = B1.sum(r)*r - B2.sum(r)`.

C1 — count of smaller numbers after self

```

vector<int> countSmaller(vector<int>& a) {
    vector<int> vals = a;
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    Fenwick bit(vals.size());
    vector<int> res(a.size());
    for (int i = a.size() - 1; i >= 0; --i) {
        int idx = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin() + 1;
        res[i] = bit.sum(idx - 1);
        bit.add(idx, 1);
    }
    return res;
}

```

C2 — reverse pairs — same as C1, but count elements $\leq a[i]/2$ (careful with $2·a[j] \geq a[i]$ boundary — use `lower_bound(vals, a[i]/2.0)` style or compress $2·a[j]$ values).

C3 — k-th order statistic with insert/delete — Fenwick counts + k th (topic 26, Pattern 3); insert = add(+1), delete = add(-1), query = k th(k).

27 — Segment Trees

Hints

- H1 (M1): lazy rangeAdd; push before descending, apply on full cover.
- H2 (C1): lazy with a hasAssignment flag; assignment overwrites pending additions.
- H3 (C2): sweep line — sort vertical edges, segment tree over y with "covered length".

E1 — range sum point update — topic 27's SegTree as-is.

E2 — range min point update — same tree, change combine to min and no-overlap return to +INF.

M1 — range add, range sum (lazy) — topic 27's LazySeg.

M2 — count of values in a range — segment tree where each node holds the *sorted* values of its range (merge sort tree): query = for each covered node, upper_bound - lower_bound on its sorted vector. $O(\log^2 n)$ per query.

M3 — GCD range queries — combine = `std::gcd`, no-overlap = 0.

C1 — assignment + range sum: lazyAssign[node] + hasAssign[node]; on apply, set tree = val * len, mark assignment; push propagates the assignment flag (not additive lazy).

C2 — rectangle area union: sweep line — sort edges by x; for each edge, update the segment tree's covered length over y; area += covered_length × dx.

C3 — k-th smallest in range with updates — merge sort tree + point updates, or a segment tree over the value domain with count BITS (wavelet-tree-like). Sketch: each leaf = a value; query range counts per node; descend by counts.

28 — Advanced Graphs

Hints

- H1 (M2): Kosaraju — two passes (original for finish order, reversed for components).
- H2 (C2): after condensation, the answer = nodes with in-degree 0 in the condensed DAG.
- H3 (C4): Kuhn's algorithm; reset seen per left vertex.

M1 — critical connections — bridges (topic 28, code given).

M2 — SCC count — topic 28, Kosaraju (return `sccs.size()`).

M3 — course schedule IV — topo order, then bitset DP: `reachable[u]` = OR of `reachable[v]` over successors + bits for direct edges.

C1 — reconstruct itinerary — Hierholzer: DFS, take edges in lexical order (use a multiset or sorted vector per node + pointer), push nodes on postorder, reverse.

C2 — min vertices to reach all nodes — SCC-condense; count nodes with in-degree 0 in the condensation (they can't be reached from elsewhere).

C3 — 2-SAT — build implication graph ($x \rightarrow \neg y$ for each clause); SCC; unsatisfiable iff x and $\neg x$ share an SCC; otherwise assign by SCC condensation order.

C4 — bipartite matching — Kuhn's algorithm (topic 28, code given); model each problem as left/right bipartition.

C5 — max flow — build residual graph; Edmonds-Karp BFS (or Dinic for speed); answer = total flow. Min-cut = nodes reachable from s in the residual graph after max flow.

Closing notes

- If you solved an E/M problem without hints — great. Re-attempt the same topic's Challenging ones now.
- If you needed hints, re-solve the problem from scratch in 2-3 days (spaced repetition).
- For every solution you wrote, state its time & space complexity out loud. That's the habit that makes all of this stick.
- Solutions here are reference implementations — yours can be different and better. The goal is the **reasoning, not the code*.

31 — DSA Checklist & Complexity Cheat Sheet

Part A — Mastery checklist (tick off honestly)

Fundamentals

- ☐ STL: vector, map/unordered_map, set/unordered_set, priority_queue, stack, queue, deque — know each op's complexity
- ☐ Big-O: analyze nested loops, recursion (Master theorem light), hidden costs (string concat, vector insert)
- ☐ Arrays: prefix sums, difference array, Kadane's, in-place rotation, two-pointer basics
- ☐ Strings: frequency counting, anagram check, palindrome (expand around center), rolling hash idea

Core algorithms

- ☐ Binary search: classic, lower/upper bound, binary search on answer — safe-mid and keep-vs-discard rules
- ☐ Sorting: merge & quick by hand; stability; custom comparators; inversion count via merge
- ☐ Hashing: frequency, complement lookup, prefix-sum + hash subarray problems
- ☐ Two pointers: opposite & same direction, merge two sorted sequences
- ☐ Sliding window: fixed, variable, condition-counter, monotonic deque

Data structures

- ☐ Linked lists: reverse (iterative+recursive), dummy node, Floyd's cycle, k-from-end, merge
- ☐ Stacks: parentheses, postfix eval, monotonic stack (next greater, histogram)
- ☐ Queues/deques: circular array, level-order, sliding window max, multi-source BFS
- ☐ Trees: 4 traversals (recursive + iterative), height/diameter/LCA
- ☐ BST: search/insert/delete (3 cases), validate with ranges, k-th smallest, sorted→balanced
- ☐ Heaps: push/pop, heapify $O(n)$, top-K, k-way merge, two-heap median
- ☐ Tries: insert/search/startsWith, autocomplete, bit trie for max XOR
- ☐ DSU: find w/ path compression, union by size, components count, Kruskal's cycle check
- ☐ Fenwick: point update + prefix query, range update trick, inversion count, k-th via lifting
- ☐ Segment trees: build, point update, range query, lazy propagation, identity values

Techniques

- ☐ Recursion: base case + trust, call-stack cost, divide & conquer
- ☐ Backtracking: choose-explore-unchoose, pruning, duplicate handling, N-Queens, word search
- ☐ Greedy: exchange-argument intuition; know where greedy fails (0/1 knapsack, coin change)
- ☐ BFS/DFS on graphs & grids: components, cycles (undirected & directed), bipartite, flood fill
- ☐ Topological sort: Kahn + DFS, cycle detection, longest path in DAG
- ☐ Shortest paths: Dijkstra (stale-entry skip!), Bellman-Ford (neg cycles), Floyd-Warshall (k, i, j), 0-1 BFS
- ☐ MST: Kruskal (DSU) + Prim (heap), cut property
- ☐ DP: top-down vs bottom-up, 1D/2D states, knapsack (0/1 vs unbounded direction), LCS, LIS, coin change, space optimization

- Bit manipulation: XOR tricks, masks, popcount, power-of-two, subset enumeration

Advanced (stretch goals)

- SCC (Kosaraju/Tarjan), bridges, articulation points
- Euler path/circuit (Hierholzer)
- Bipartite matching (Kuhn) and max-flow concept (residual edges, min-cut theorem)
- Binary lifting for LCA

Part B — Complexity cheat sheet

Common operations

Operation	Complexity
Array / vector random access	$O(1)$
vector <code>push_back</code> / <code>pop_back</code>	$O(1)$ amortized
vector insert/erase at front or middle	$O(n)$
sort / <code>stable_sort</code>	$O(n \log n)$
<code>binary_search</code> / lower_bound on sorted	$O(\log n)$
set / map (insert, erase, find)	$O(\log n)$
unordered_set/map (average)	$O(1)$
unordered_set/map (worst case)	$O(n)$
priority_queue push / pop	$O(\log n)$; top $O(1)$
stack / queue / deque (ends)	$O(1)$
string concatenation <code>s = s + c</code> in loop	$O(n^2)$ total — use <code>+=</code>
<code>s.substr(k, len)</code>	$O(len)$
<code>std::gcd</code> , <code>__builtin_popcount</code>	$O(\log \min)$ / $O(1)$

Sorting

Algorithm	Best	Average	Worst	Space	Stable
Bubble / Selection / Insertion	$O(n)$ / $O(n^2)$ / $O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓ / ✗ / ✓
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓
Quick (introsort)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✗
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	✓

Graph algorithms (V vertices, E edges)

Algorithm	Time	Space
BFS / DFS	$O(V + E)$	$O(V)$
Dijkstra (heap)	$O((V + E) \log V)$	$O(V + E)$
0-1 BFS	$O(V + E)$	$O(V)$
Bellman-Ford	$O(V \cdot E)$	$O(V)$
Floyd-Warshall	$O(V^3)$	$O(V^2)$

Algorithm	Time	Space
Topological (Kahn / DFS)	$O(V + E)$	$O(V)$
Kruskal MST	$O(E \log E)$	$O(E)$
Prim MST (heap)	$O((V + E) \log V)$	$O(V + E)$
SCC (Kosaraju / Tarjan)	$O(V + E)$	$O(V)$
Bridges / Articulation	$O(V + E)$	$O(V)$
Kuhn bipartite matching	$O(V \cdot E)$	$O(E)$
Dinic max flow	$O(V^2 \cdot E)$	$O(V + E)$

Data structures (per operation)

Structure	Insert	Delete	Search	Extra
Unsorted array	$O(1)$ end	$O(n)$	$O(n)$	$O(1)$ access
Sorted array	$O(n)$	$O(n)$	$O(\log n)$	binary search
Linked list	$O(1)$ at head	$O(1)$ at head	$O(n)$	$O(1)$ with iterator
Hash table	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	worst $O(n)$
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	inorder sorted
Heap	$O(\log n)$	$O(\log n)$	$O(n)$	$O(1)$ peek
Fenwick	$O(\log n)$ update	—	$O(\log n)$ prefix	range sums
Segment tree	$O(\log n)$ update	—	$O(\log n)$ range	lazy: $O(\log n)$ range update

DP quick reference

Problem	States	Transition	Time
Fibonacci	$dp[i]$	$dp[i-1] + dp[i-2]$	$O(n)$
Climbing stairs	$dp[i]$	$dp[i-1] + dp[i-2]$	$O(n)$
House robber	$dp[i]$	$\max(dp[i-1], dp[i-2] + a[i])$	$O(n)$
0/1 knapsack	$dp[c]$	$\max(\text{skip}, dp[c-w] + v)$ — capacity backwards	$O(n \cdot W)$
Unbounded knapsack / coin change	$dp[a]$	$\min(dp[a-c] + 1)$ — forwards	$O(W \cdot \# \text{coins})$
LCS	$dp[i][j]$	equal $\rightarrow +1$ else $\max$ of neighbors	$O(n \cdot m)$
LIS	$dp[i]$	$1 + \max(dp[j] \mid j < i, a[j] < a[i])$	$O(n^2)$, $O(n \log n)$ tails
Edit distance	$dp[i][j]$	$\min(\text{insert}, \text{delete}, \text{replace}))$	$O(n \cdot m)$
Unique paths grid	$dp[i][j]$	$dp[i-1][j] + dp[i][j-1]$	$O(n \cdot m)$

Part C — Final exam survival card (one screen)

Read constraints $\rightarrow$ complexity budget $\rightarrow$ pattern $\rightarrow$ simplify $\rightarrow$ validate.

- $n \leq 10$: brute force / permutations
- $n \leq 20$: backtracking / bitmask DP
- $n \leq 500$: $O(n^3)$ (Floyd)
- $n \leq 5000$: $O(n^2)$ DP / two pointers
- $n \leq 10^5$ - 10^6 : $O(n \log n)$ / $O(n)$
- 10^8+ : $O(\log n)$ / math

Signals:

- subarray + window $\rightarrow$ sliding window; subarray + exact sum $\rightarrow$ prefix + hash

- pairs on sorted → two pointers; pairs on unsorted → hash
- next greater / histogram → monotonic stack
- min steps in graph/grid → BFS; weighted → Dijkstra; negative → Bellman-Ford
- DAG order / longest path in DAG → topological sort
- count ways / maximize / minimize / possible → DP (greedy only if provable)
- all combos / perms / puzzles, small n → backtracking (prune!)
- connectivity merging → DSU; range query + updates → Fenwick/segment tree
- prefix strings → trie; XOR/masks → bit manipulation
- median stream → two heaps; LRU → list + hash; autocomplete → trie + counts